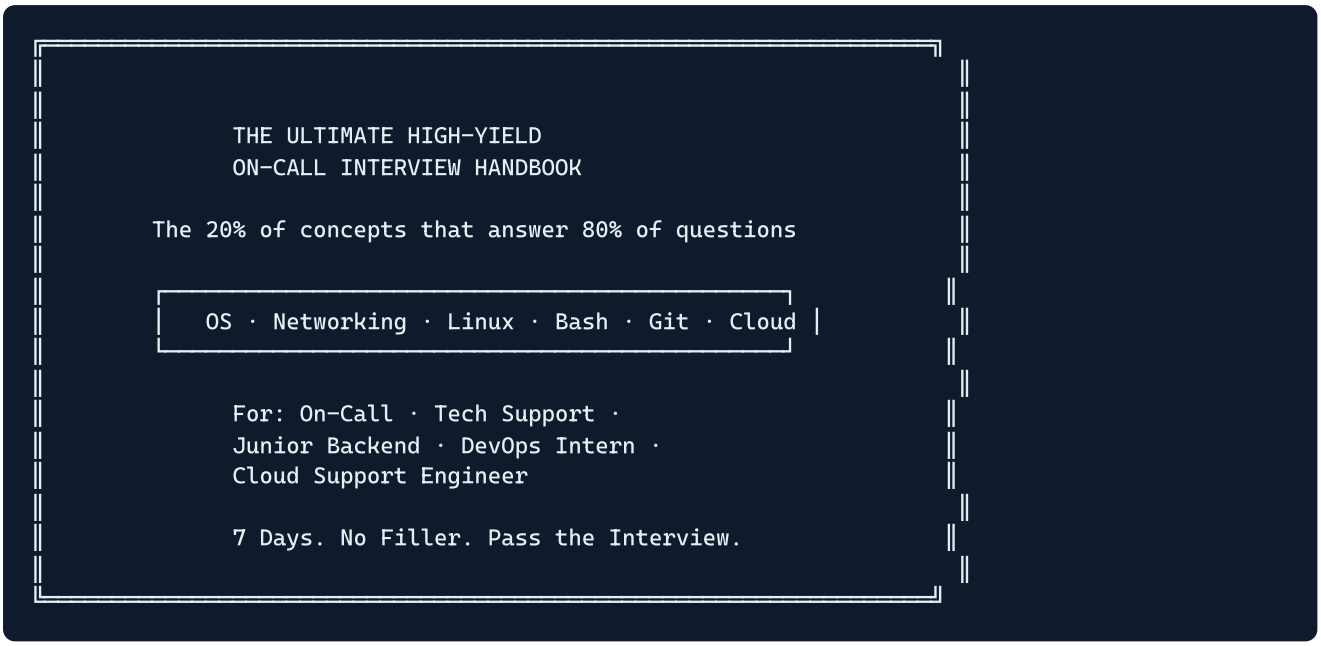
On-Call Interview Handbook



The Ultimate High-Yield On-Call Interview Handbook

A senior engineer's crash course for the week before your interview.

How to read this book (and the one rule)

I wrote this the way I'd brief a junior engineer I actually liked, one week before a big interview. I'm not going to teach you everything about operating systems or networking — a university degree already tried that. Instead, every section here earns its place by answering a single question:

> **"If the interview were tomorrow, what is the most valuable thing you could know?"**

That's the whole philosophy. The **Pareto Principle**: roughly 20% of the concepts show up in about 80% of interviews for these roles. We spend our time there and ruthlessly ignore the rest.

The star system

Every topic is ranked by how often it actually comes up in real interviews:

Rating	Meaning	What to do with 7 days
★★★★★	Asked in almost every interview	**Master it. Be able to teach it.**
★★★★☆	Very common	Know it cold
★★★☆☆	Common	Understand it, have an example
★★☆☆☆	Occasional	Skim, recognize it
★☆☆☆☆	Rare	Read once, move on

If you are short on time, **study the five-star and four-star topics first.** That is the highest-yield path through this book.

The callout boxes you'll see

Throughout the book, watch for these:

> 💬 **Interviewers usually ask…** — the exact question, phrased the way it's really asked.

> ✅ **Model answer** — how a strong candidate answers out loud, in about a minute.

> ⚠️ **Common mistake** — the trap that makes junior candidates look junior.

> 🧠 **Memory trick** — a way to remember it under pressure.

> 🏭 **In real production…** — where this shows up on the job, so your answer sounds like experience, not memorization.

> 🐧 **Linux example** — a command you can run right now to see the concept live.

Your 7-day study plan

You don't have time to read this front-to-back three times. Here's how to spend the week. Each "day" is roughly 2–3 focused hours.

Day	Focus	Chapters
Day 1	Operating Systems	Chapter 1
Day 2	Networking (part 1)	Chapter 2 (OSI → DNS/HTTP)
Day 3	Networking (part 2)	Chapter 2 (HTTPS → Proxies) + review
Day 4	Linux commands	Chapter 3
Day 5	Bash + text processing	Chapters 4 & 5
Day 6	Git + Cloud	Chapters 6 & 7
Day 7	Revision, exam, mocks	Chapters 8–11

> 💡 **Tip:** On Day 7, do the mock interviews **out loud**. Reading an answer and *saying* an answer are two different skills, and the interview tests the second one.

Table of Contents

Chapter 11 — Final Exam — 100 MCQ + 50 short + 20 practical Linux + 20 networking scenarios + answer key

---

> **Final note before you start:** This is not a textbook. It's an interview weapon. Don't try to memorize every word — try to *understand* each concept well enough to explain it to a friend. If you can teach it, you can pass an interview on it.

Let's get to work. Turn to Chapter 1.

Chapter 1 — Operating Systems

> **Why this chapter matters:** OS questions are the great equalizer. Every interviewer, from a scrappy startup to a cloud giant, reaches for "process vs thread" or "what's a deadlock" because they instantly separate people who *understand* computers from people who only *use* them. Get this chapter right and you'll sound like an engineer, not a graduate.

The topics are ordered roughly by interview frequency. If you only have one evening, study everything marked ★★★★★ and ★★★★☆.

1.1 What is an Operating System? ★★★☆☆

Definition. An operating system (OS) is the software that sits between your programs and the physical hardware. It manages the CPU, memory, disk, and devices, and hands those resources out to programs so they don't have to fight over them.

Why it exists. Imagine 200 programs all wanting the CPU, RAM, and disk at the same time. Without a referee, they'd corrupt each other's memory and crash constantly. The OS is that referee. Its three big jobs:

1. **Resource management** — who gets the CPU, how much memory, which files.
2. **Abstraction** — programs say "open this file" instead of "move the disk head to cylinder 4."
3. **Isolation & protection** — one crashing program shouldn't take down the whole machine.

> 🌍 **Analogy.** The OS is the manager of an apartment building. Tenants (programs) don't run their own water and electricity — they just turn a tap. The manager handles the plumbing, stops tenants from wandering into each other's flats, and settles disputes over shared laundry.

> 💬 **Interviewers usually ask:** "What does an operating system actually do?"
>
> ✅ **Model answer:** "It's the layer between applications and hardware. It manages resources — CPU time, memory, disk, devices — and shares them safely among many programs. It also gives programs a simple, consistent interface so they don't talk to hardware directly, and it isolates programs so one misbehaving process can't corrupt another."

> 🧠 **Memory trick:** OS = **M.A.I.** — **M**anages resources, **A**bstracts hardware, **I**solates programs.

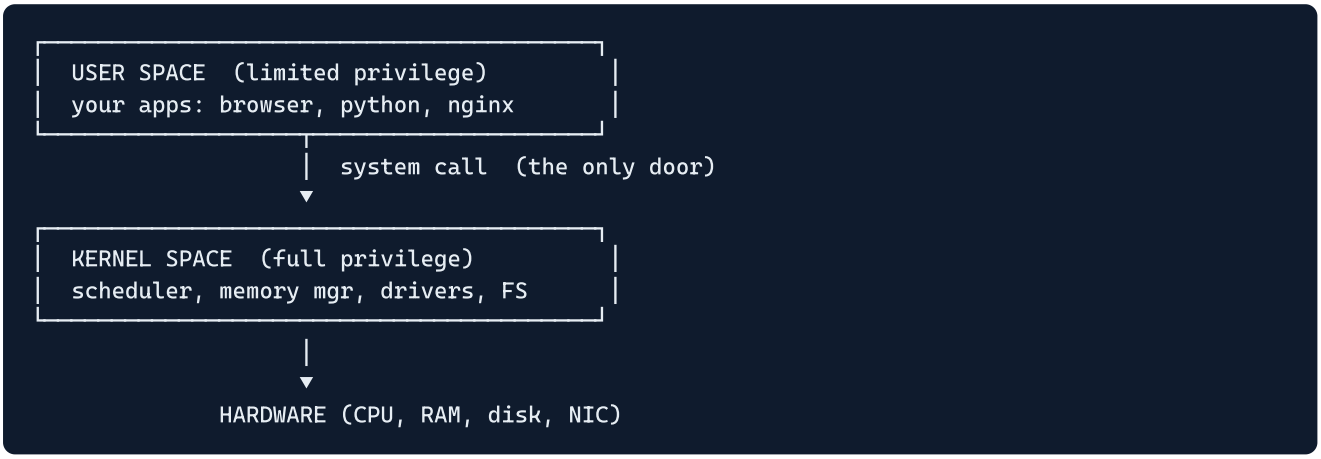
1.2 Kernel ★★★★☆

Definition. The **kernel** is the core of the OS — the part that's always running, with full control over the hardware. It manages memory, schedules processes, handles system calls, and talks to device drivers.

Why it exists. You want *one* trusted piece of code with god-mode access to the hardware, and everything else running with limited privileges. That trusted core is the kernel. If any app could touch hardware directly, a single bug could brick the machine.

> 🌍 **Analogy.** The kernel is the engine room of a ship. Passengers (apps) never go down there. The crew (kernel) runs the engines and only exposes controls — "faster," "slower," "turn" — through the bridge.

Key idea — two privilege levels (know this):



> 💬 **Interviewers usually ask:** "What's the difference between kernel space and user space?"
>
> ✅ **Model answer:** "Kernel space is privileged — code there can touch hardware directly and access any memory. User space is where normal applications run, sandboxed and restricted. When an app needs something privileged, like reading a file or opening a network socket, it can't do it itself — it asks the kernel through a **system call**. That boundary is what keeps a buggy app from crashing the whole system."

> ⚠️ **Common mistake:** Saying "Linux is the OS." Technically **Linux is the kernel**; the full OS is the kernel *plus* the surrounding tools (a distribution like Ubuntu). Interviewers notice when you get this right.

> 🧠 **Memory trick:** Kernel = **KING**. It's the only one allowed in the throne room (hardware).

1.3 User Space ★★★☆☆

Definition. User space is the memory region and privilege level where normal programs run. Code here cannot touch hardware or other processes' memory directly — it must go through the kernel.

Why it exists. Safety and stability. If your text editor crashes, the kernel and every other program keep running because the editor was fenced into its own user-space sandbox.

🏭 **In real production:** When you see a process using 100% CPU in `top`, it's almost always burning that time in user space (your app's own code) or in kernel space (system calls). Tools show these separately as `%us` (user) and `%sy` (system) — a high `%sy` often means the app is hammering the kernel with syscalls, like excessive disk or network I/O.

🧠 **Memory trick: User space = the playground; kernel space = the control room.** Kids play in the playground; only staff enter the control room.

1.4 System Calls ★★★★☆

Definition. A **system call (syscall)** is the controlled way a user-space program requests a service from the kernel — opening a file, allocating memory, creating a process, sending network data.

Why it exists. It's the *one door* between the untrusted playground and the privileged control room. The program can't just grab hardware; it politely asks, the kernel checks permissions, does the work, and returns the result.

The flow (worth being able to draw):

```
your code:  fd = open("data.txt", O_RDONLY);
                    |
                    ▼   (CPU switches to kernel mode)
kernel:      checks permissions → talks to filesystem/driver
                    |
                    ▼   (CPU switches back to user mode)
your code:  gets back a file descriptor (or an error)
```

Common syscalls to name-drop: `open`, `read`, `write`, `close`, `fork`, `exec`, `wait`, `mmap`, `socket`.

🌎 **Analogy.** A syscall is ordering at a restaurant. You don't walk into the kitchen (hardware). You tell the waiter (syscall interface) what you want; the kitchen (kernel) makes it and brings it out.

💬 **Interviewers usually ask:** "What happens when a program reads a file?"

✅ **Model answer:** "The program makes a `read` system call. The CPU switches from user mode to kernel mode, the kernel checks the process is allowed to read that file, talks to the filesystem and disk driver to fetch the data, copies it into the program's buffer, then switches back to user mode and returns. The mode switch is what makes it a *system* call rather than a normal function call."

🐧 **Linux example:** Run `strace ls` — you'll see the *actual* system calls `ls` makes (`openat`, `read`, `write`, `close`). This one command makes syscalls tangible; try it before your interview.

🧠 **Memory trick:** Syscall = the **waiter**. Only the waiter crosses into the kitchen.

1.5 Process vs Thread ★★★★★

This is the single most-asked OS question. Know it cold.

Definition. - A **process** is a program in execution — it has its own private memory space (code, heap, stack), its own file descriptors, and at least one thread. - A **thread** is a unit of execution *inside* a process. Threads of the same process **share** that process's memory and resources but each has its own stack and program counter.

Why threads exist. Creating a whole new process is expensive and processes can't easily share data. Threads are lightweight and share memory, so they're great for doing several things at once inside one program (e.g., one thread handles the UI while another downloads a file).

> 🌍 **Analogy.** A **process is a house**; **threads are the people living in it**. They share the kitchen and fridge (memory), which is convenient — but if two people grab the last egg at the same moment, you get a conflict (a race condition). Different houses (processes) don't share a fridge, so they're safer but have to phone each other to exchange anything (inter-process communication).

The comparison table interviewers love:

	Process	Thread
Memory	Own private space	Shared within the process
Creation cost	Heavy (slow)	Light (fast)
Communication	Hard — needs IPC (pipes, sockets)	Easy — shared memory
Crash impact	Isolated; one crash ≠ others die	One thread crashing can take down the whole process
Owns	Code, heap, its threads	Its own stack + registers only

> 💬 **Interviewers usually ask:** "What's the difference between a process and a thread?"
>
> ✅ **Model answer:** "A process is an independent program with its own isolated memory. A thread is a lighter unit of execution inside a process, and threads of the same process share memory. That sharing makes threads fast to create and easy to communicate between, but it also means they can step on each other's data — so you need synchronization like mutexes. Processes are isolated and safer but heavier, and they need explicit IPC to talk to each other."

> ⚠️ **Common mistake:** Saying "threads are just faster processes." The real distinction is **shared memory** — that's what makes threads both powerful *and* dangerous (race conditions).

> 🏭 **In real production:** A web server like Nginx uses multiple processes; a Java app server often uses many threads inside one process. When someone says "the app is leaking memory," you need to know whether it's one bloated process or many.

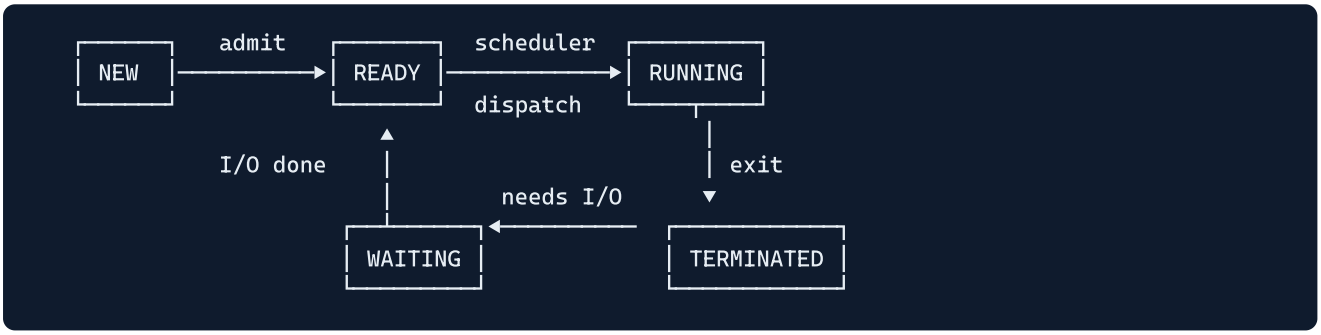
> 🧠 **Memory trick: P**rocess = **P**rivate memory. **T**hread = **T**ogether (shared) memory.

1.6 Process States ★★★★☆

Definition. As a process runs, it moves through a small set of states. The classic ones:

- **New** — being created.
- **Ready** — able to run, waiting for the CPU.
- **Running** — currently executing on the CPU.
- **Waiting / Blocked** — paused, waiting for something (disk, network, user input).
- **Terminated** — finished.

The state diagram (be able to sketch this):



Why it matters. The whole point of an OS is juggling processes between these states so the CPU is never idle while there's work to do. When a process is **Waiting** on disk, the OS runs someone else.

> 🌍 **Analogy.** A doctor's office. **Ready** = patients in the waiting room. **Running** = the one patient with the doctor. **Waiting** = someone sent off for a blood test; the doctor sees other patients meanwhile and calls them back when results are in.

> 💬 **Interviewers usually ask:** "Walk me through the states a process goes through."
>
> ✅ **Model answer:** "New when it's created, Ready when it's waiting for a CPU, Running when it's on the CPU. If it needs I/O — say, reading a file — it moves to Waiting and gives up the CPU so someone else can run. When the I/O completes it goes back to Ready, not straight to Running, because the CPU might be busy. Finally it Terminates. The OS constantly shuffles processes between Ready and Running to keep the CPU busy."

> 🐧 **Linux example:** In `top` or `ps`, the `STAT` column shows state: `R` running/runnable, `S` sleeping (interruptible wait), `D` uninterruptible sleep (usually disk I/O — a stuck `D` process is a classic sign of storage trouble), `Z` zombie, `T` stopped.

> 🧠 **Memory trick: Ready → Running → Waiting** is a loop. A process bounces around that loop many times before it Terminates.

1.7 Context Switching ★★★★☆

Definition. A **context switch** is when the CPU stops running one process (or thread) and starts running another. The OS must save the first one's state (registers, program counter, stack pointer) and load the second one's.

Why it exists. You have far more processes than CPU cores. To create the illusion that everything runs "at once," the OS rapidly switches between them — thousands of times per second.

Why interviewers care. Context switches aren't free. Saving and restoring state takes time, and the CPU caches get cold. Too many context switches = wasted CPU = a real production performance problem.

> 🌍 **Analogy.** A chef cooking five dishes on one stove. Every time they switch pots, they have to remember exactly where they left each one (heat level, timer, what's next). That remembering-and-reloading is the context switch. Switch too often and you spend more time remembering than cooking.

> 💬 **Interviewers usually ask:** "What is a context switch and why is it expensive?"
>
> ✅ **Model answer:** "It's when the CPU swaps from one process or thread to another. The OS saves the current one's CPU state — registers, program counter — and restores the next one's. It's expensive because it's pure overhead: no useful work happens during the switch, and afterward the CPU cache is full of the *old* process's data, so the new one runs slower until the cache warms up. If a system does too many context switches, you see high CPU with low actual throughput."

> 🏭 **In real production:** High context-switch rates (visible in `vmstat` under `cs`) often point to too many threads fighting over too few cores, or lock contention. It's a real thing SREs investigate.

> 🧠 **Memory trick:** Context switch = **save game, load different save.** The saving/loading is overhead; only the playing is progress.

1.8 CPU Scheduling ★★★★☆

Definition. CPU scheduling is how the OS decides *which* ready process runs next. The piece of the kernel that decides is the **scheduler**.

Why it exists. Many processes are Ready; there's one CPU (per core). Someone has to choose. Different algorithms optimize for different goals — fairness, throughput, or responsiveness.

Interviewers usually want three algorithms. Here they are with a worked example.

Setup: Three jobs arrive at time 0. Burst times (how long each needs the CPU): **P1 = 8, P2 = 4, P3 = 2.**

FCFS — First Come, First Served ★★★☆☆

Run them in arrival order. Simple, but a big job blocks everyone behind it (the **convoy effect**).

```
| P1 (8) ......... | P2 (4) .... | P3 (2) .. |
0                 8             12          14
Wait times: P1=0, P2=8, P3=12   → avg wait = 20/3 ≈ 6.67
```

> 🌍 **Analogy.** A single supermarket queue. If the person in front has a full trolley, everyone behind waits — even the person holding one apple.

SJF — Shortest Job First ★★★☆☆

Run the shortest job first. Gives the **minimum average wait time**, but long jobs can **starve** if short ones keep arriving.

```
| P3 (2) .. | P2 (4) .... | P1 (8) ......... |
0           2             6                 14
Wait times: P3=0, P2=2, P1=6   → avg wait = 8/3 ≈ 2.67  ← better!
```

> 🌍 **Analogy.** The "10 items or less" express lane — quick shoppers get out fast, but if express shoppers keep coming, the person with a full trolley never gets served (starvation).

Round Robin ★★★★☆

Each process gets a fixed **time slice (quantum)**, then goes to the back of the queue. Fair and responsive — the algorithm behind interactive, time-sharing systems. This is the one to know best.

```
Quantum = 2:
| P1 | P2 | P3 | P1 | P2 | P1 | P1 |
0    2    4    6    8   10   12   14
Everyone gets a turn quickly; no one is starved.
```

> 🌍 **Analogy.** A teacher giving every student 2 minutes to speak, round the room, again and again. Nobody hogs the floor; nobody is ignored.

The trade-off table:

Algorithm	Optimizes for	Weakness
FCFS	Simplicity	Convoy effect (big job blocks all)
SJF	Lowest avg wait	Starvation of long jobs; needs to know burst time
Round Robin	Fairness & responsiveness	Overhead from many context switches if quantum too small

> 💬 **Interviewers usually ask:** "Which scheduling algorithm would you use for an interactive system, and why?"
>
> ✅ **Model answer:** "Round Robin. Each process gets a small, fixed time slice, so no single process can hog the CPU and every user gets a quick response — which is exactly what you want for interactive workloads. The trade-off is the time slice size: too large and it degrades into FCFS; too small and you waste CPU on context switching."

> 🧠 **Memory trick:** **F**CFS = **F**irst in line. **S**JF = **S**hortest wins. **R**R = **R**otate everyone.

1.9 Mutex vs Semaphore ★★★★☆

When threads share memory (§1.5), two of them touching the same data at once causes a **race condition** — the result depends on luck of timing, and it corrupts data. Mutexes and semaphores are the tools that prevent it.

Race condition, concretely: two threads both do `balance = balance + 100` on a shared account. If they read the old value at the same time, one update is lost. Classic interview setup.

Mutex (Mutual Exclusion) ★★★★☆

A **lock** that lets exactly **one** thread into a critical section at a time. You **lock** it, do the sensitive work, then **unlock**. Crucially, the thread that locks it is the one that must unlock it — it's like a key you're holding.

```
Thread A: lock(m) → [update balance] → unlock(m)
Thread B: lock(m) ...waits... → [update balance] → unlock(m)
```

> 🌍 **Analogy.** A single toilet with one key. Only one person inside at a time. You take the key, go in, come out, hand the key back.

Semaphore ★★★☆☆

A **counter** that allows up to **N** threads through at once. `wait()` decrements (and blocks at 0); `signal()` increments. A semaphore with N=1 is called a **binary semaphore** and behaves much like a mutex — but any thread can signal it, so it's also used for *signaling between* threads, not just locking.

> 🌍 **Analogy.** A parking lot with 3 spaces and a counter at the gate. Cars enter while spaces remain; when full, new cars wait; each car leaving frees a space.

The distinction interviewers want:

	Mutex	Semaphore
Purpose	Locking (protect a resource)	Signaling + limiting to N
Allowed at once	Exactly 1	Up to N
Ownership	Owned — locker must unlock	Not owned — anyone can signal

> 💬 **Interviewers usually ask:** "Difference between a mutex and a semaphore?"
>
> ✅ **Model answer:** "A mutex is a lock that allows exactly one thread into a critical section, and only the thread that locked it can unlock it — it's about mutual exclusion. A semaphore is a counter that allows up to N threads through, so it's used both for limiting concurrent access to a pool of resources and for signaling between threads. A binary semaphore looks like a mutex but has no ownership — any thread can signal it."

> 🧠 **Memory trick: Mut**ex = **Mut**ually exclusive = **1** at a time (a key). **Sem**aphore = counts (like traffic **sem**aphore signals) = up to **N**.

1.10 Deadlock ★★★★★

Very high frequency. Know the four conditions.

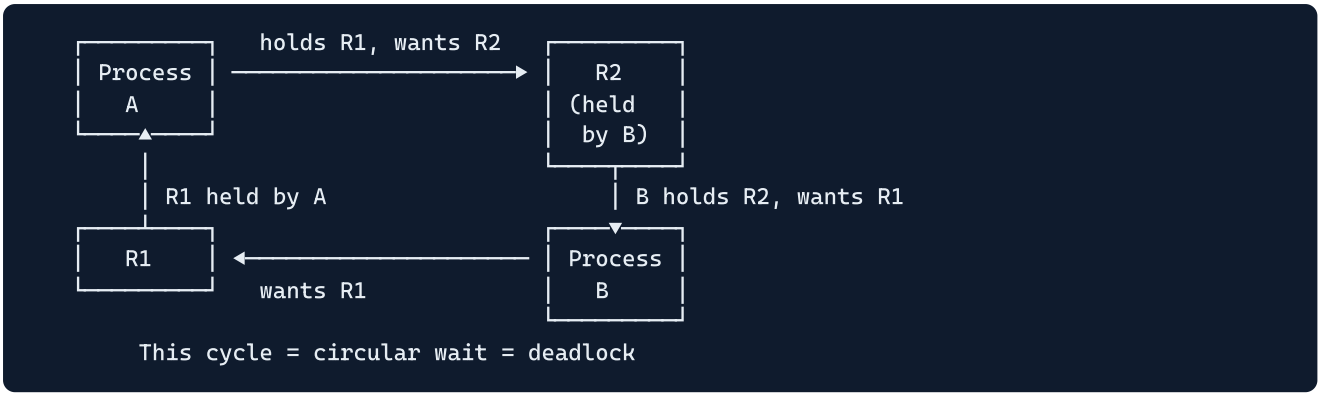
Definition. A **deadlock** is when two or more processes are each waiting for a resource the other holds, so none can ever proceed. Everyone's stuck forever.

> 🌍 **Analogy.** Two people at a narrow doorway, each insisting "you first — no, you first," neither moving. Or: Alice holds a fork and waits for a knife; Bob holds the knife and waits for the fork. Neither eats.

The four Coffman conditions (all four must hold for deadlock). This is the money answer:

1. **Mutual exclusion** — a resource can't be shared; only one holder at a time.
2. **Hold and wait** — a process holds one resource while waiting for another.
3. **No preemption** — you can't forcibly take a resource away; it must be released voluntarily.
4. **Circular wait** — a cycle of processes each waiting on the next.

Break **any one** of these and deadlock is impossible.



💬 **Interviewers usually ask:** "What is a deadlock and how do you prevent it?"

✅ **Model answer:** "A deadlock is when processes wait on each other in a cycle, each holding a resource the other needs, so none can proceed. It requires four conditions at once — mutual exclusion, hold-and-wait, no preemption, and circular wait. To prevent it you break one condition. The most practical is breaking circular wait by imposing a global order on locks — always acquire lock A before lock B — so a cycle can't form. You can also avoid hold-and-wait by grabbing all locks at once, or use timeouts to detect and recover."

⚠️ **Common mistake:** Confusing deadlock with **starvation**. Deadlock = everyone stuck forever in a cycle. Starvation = a process keeps getting skipped (e.g., a long job under SJF) but the system as a whole is still making progress.

🏭 **In real production:** Deadlocks are common in databases when two transactions lock rows in opposite order. Databases like MySQL/Postgres *detect* the cycle and kill one transaction with a "deadlock detected" error — your app should retry it.

🧠 **Memory trick:** The four conditions spell an easy story: **"Mine, held, no-take-backs, in a circle."** Or acronym **M-H-N-C**.

1.11 Virtual Memory ★★★★☆

Definition. Virtual memory is an abstraction where each process gets its own large, private, contiguous-looking address space, which the OS maps onto real physical RAM (and disk) behind the scenes. Programs use *virtual* addresses; the hardware (MMU) translates them to *physical* addresses.

Why it exists — three big wins: 1. **Isolation** — each process thinks it has the whole memory to itself and can't see others'. Security + stability. 2. **More memory than you have** — rarely used pages can be pushed to disk (**swap**), so you can run programs bigger than physical RAM. 3. **Simplicity for programmers** — every program is written as if it starts at address 0 with a clean, continuous space.

🌎 **Analogy.** Everyone's home address is "123 Main St" as far as they're concerned, but the postal service (MMU) maps each to a real, unique location. Nobody needs to know the physical layout; the mapping handles it.

💬 **Interviewers usually ask:** "What is virtual memory and why is it useful?"

✅ **Model answer:** "Virtual memory gives each process its own private address space that the OS maps to physical RAM. It gives you isolation — processes can't touch each other's memory — and it lets the system use disk as an overflow via swap, so programs can be larger than physical RAM. It also simplifies programming because every process sees a clean, continuous address space starting at zero, even though the real physical layout is fragmented."

⚠️ **Common mistake:** Thinking virtual memory *is* the swap file. Swap is just one part — the disk overflow. Virtual memory is the whole addressing-and-mapping system.

🏭 **In real production:** When a server "starts swapping," performance falls off a cliff because disk is ~1000× slower than RAM. Watch for it in `free -m` (swap used climbing) and `vmstat` (`si` / `so` = swap in/out).

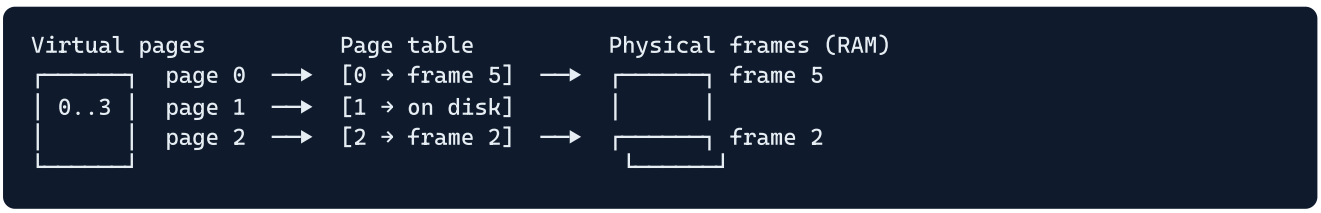
🧠 **Memory trick:** Virtual memory = **"everyone gets their own imaginary house on Main St; the OS knows the real map."**

1.12 Paging ★★★★☆

Definition. Paging is the mechanism that makes virtual memory work. Memory is divided into fixed-size blocks: **pages** in virtual memory and **frames** in physical memory (both commonly 4 KB). A **page table** maps each virtual page to a physical frame.

Why it exists. Fixed-size blocks eliminate **external fragmentation** — any page fits in any free frame, like same-sized LEGO bricks. And you only need to keep the *actively used* pages in RAM; the rest can live on disk.

Page fault (know this term): when a program accesses a page that isn't currently in RAM, the CPU raises a **page fault**, the OS fetches the page from disk into a frame, updates the page table, and resumes the program. A few page faults are normal; *constant* page faults (**thrashing**) means the machine is out of RAM and spending all its time shuffling pages.



🌍 **Analogy.** A library where books (pages) are stored in fixed-size shelf slots (frames). The catalog (page table) tells you which slot each book is in. Rarely read books go to off-site storage (disk); fetching one back is a page fault.

💬 **Interviewers usually ask:** "What is paging and what's a page fault?"

✅ **Model answer:** "Paging splits memory into fixed-size pages in virtual memory and frames in physical memory, with a page table mapping between them. Fixed sizes remove external fragmentation and let the OS keep only active pages in RAM. A page fault happens when a program touches a page that isn't in RAM — the OS pauses the program, loads the page from disk, updates the table, and resumes. Occasional faults are fine; constant faulting is thrashing, and it means you need more RAM."

🧠 **Memory trick: Page = virtual, Frame = physical.** Same size, so they slot together like LEGO.

1.13 Segmentation ★★☆☆☆

Definition. Segmentation divides memory by *logical* meaning rather than fixed size — separate variable-length segments for code, stack, heap, and data. Each segment has a base and a limit.

Paging vs Segmentation — the one comparison they want:

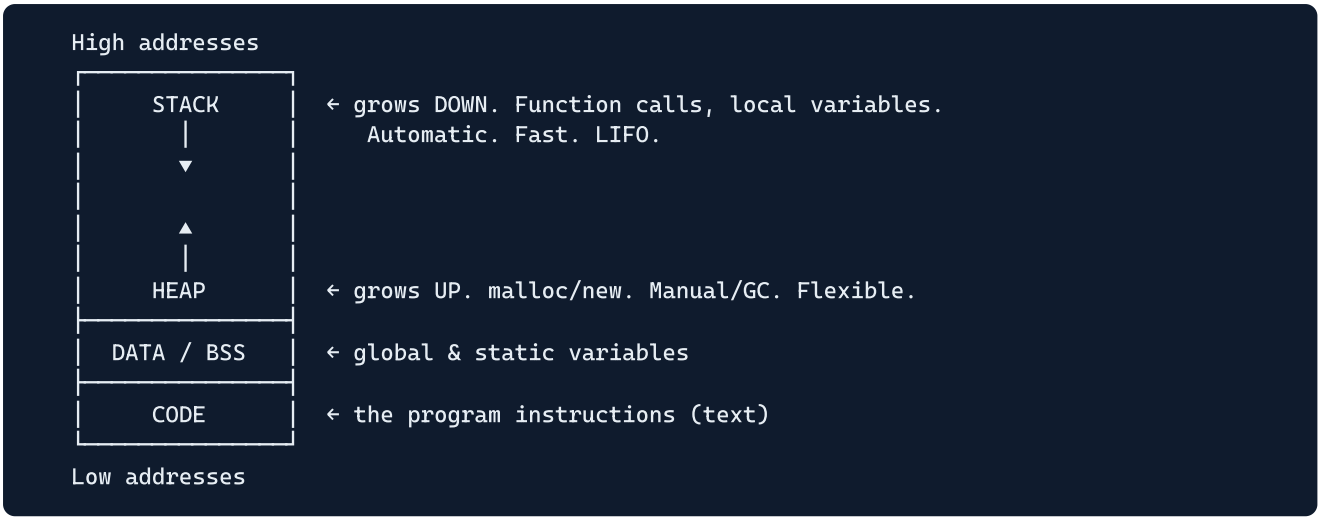
	Paging	Segmentation
Block size	Fixed (e.g., 4 KB)	Variable (logical units)
Divided by	Convenience (equal chunks)	Meaning (code / stack / heap)
Fragmentation	Internal (last page partly empty)	External (variable gaps)
Visible to programmer	No	Somewhat (logical)

🧠 **Memory trick: Pa**ging = **Pa**rts of equal size. **Seg**mentation = **Seg**ments by meaning. Modern systems mostly use paging; pure segmentation is rare, so don't over-study this one.

1.14 Heap vs Stack ★★★★★

Extremely common, especially for backend roles. This is about memory *inside* a single process.

Every process's memory has these regions:



Stack. Stores local variables and function-call bookkeeping (return addresses, parameters). Managed **automatically** — when a function is called, a "stack frame" is pushed; when it returns, the frame is popped. Fast, but **limited in size**.

Heap. Memory you request explicitly at runtime (`malloc` in C, `new` in Java/C++, or via the language's allocator). Lives until you free it (C) or the garbage collector reclaims it (Java/Python/Go). Flexible and large, but slower and needs management.

The comparison table:

	Stack	Heap
Stores	Local variables, call frames	Dynamically allocated objects
Managed by	Automatically (compiler)	Manually or by garbage collector
Speed	Fast	Slower
Size	Small, fixed limit	Large
Lifetime	Until function returns	Until freed / GC'd
Typical error	**Stack overflow** (e.g., infinite recursion)	**Memory leak** (never freed)

> 🌎 **Analogy.** The **stack** is a stack of plates — you add and remove only from the top, in strict order (LIFO), and it's quick. The **heap** is a big warehouse — you can request storage of any size anywhere, but you (or a cleanup crew) must remember to return it, or it fills up.

> 💬 **Interviewers usually ask:** "What's the difference between the stack and the heap?"
>
> ✅ **Model answer:** "Both live in a process's memory. The stack holds local variables and function-call frames; it's managed automatically in last-in-first-out order and it's fast but small. The heap is for dynamically allocated memory that you request at runtime — it's large and flexible but slower, and it lasts until it's explicitly freed or garbage collected. Too-deep recursion overflows the stack; forgetting to free heap memory causes a memory leak."

⚠️ **Common mistake:** Saying the stack and heap are separate programs or on separate machines. They're two **regions of the same process's memory**.

🧠 **Memory trick: Stack** = automatic, **st**rict order, **st**ack of plates. **Heap** = a big messy **heap** you manage yourself.

1.15 Zombie Process ★★★★☆

Definition. A **zombie** (or "defunct") process is one that has **finished executing** but still has an entry in the process table because its **parent hasn't read its exit status yet** (hasn't "reaped" it via `wait()`).

Why it exists. When a child dies, the OS keeps a tiny record — mainly the exit code — so the parent can find out how the child ended. Until the parent calls `wait()`, that record lingers. The zombie uses no CPU or memory; it's just a table entry.

🌍 **Analogy.** A zombie is like a finished delivery that's done but still sitting in the "awaiting signature" list. The package is delivered (process is dead); it just needs the manager (parent) to sign it off so the record clears.

Why interviewers care. A few zombies are harmless, but if a parent *never* reaps its children, zombies pile up and can exhaust the process table (you run out of PIDs) — a real bug in badly written services.

💬 **Interviewers usually ask:** "What is a zombie process?"

✅ **Model answer:** "It's a process that has finished but still appears in the process table because its parent hasn't collected its exit status with `wait()`. It holds no CPU or memory — just a table entry storing the exit code. One or two are fine, but if a parent never reaps its children, zombies accumulate and can exhaust the process table. You fix it by making the parent call `wait()`, or by killing the parent, in which case `init` (PID 1) adopts and reaps the zombies."

🐧 **Linux example:** In `ps aux`, a zombie shows state `Z` and often `<defunct>` in its name. You can't `kill -9` a zombie — it's already dead; you have to deal with the *parent*.

🧠 **Memory trick: Zombie = dead but not buried.** The parent has to "bury" it by reaping the exit status.

1.16 Orphan Process ★★★☆☆

Definition. An **orphan** is a process whose **parent has terminated** while the child is still running. The child keeps running; it's just lost its parent.

What happens to it. The orphan is **adopted by `init` / `systemd` (PID 1)**, which becomes its new parent and will reap it when it eventually finishes. So orphans, unlike zombies, get cleaned up properly.

> 🌍 **Analogy.** A child whose parent leaves is taken in by an orphanage (PID 1). Life goes on; someone still looks after the paperwork when the time comes.

Zombie vs Orphan — the pairing interviewers love:

	Zombie	Orphan
Who died?	The **child** (parent still alive)	The **parent** (child still alive)
Still running?	No — already dead	Yes — still running
Problem?	Yes if they pile up (table entries)	Not really — PID 1 adopts & reaps

> 💬 **Interviewers usually ask:** "Difference between a zombie and an orphan process?"
>
> ✅ **Model answer:** "In a zombie, the child has finished but the parent hasn't reaped its exit status, so a dead entry lingers in the process table. In an orphan, it's the reverse — the parent died while the child is still running. The orphan gets adopted by init, PID 1, which becomes its parent and reaps it later. So orphans are handled cleanly, whereas zombies can be a problem if they accumulate."

> 🧠 **Memory trick: Z**ombie = **child** is dead (**Z** for the child's zzz's). **Orphan** = **parent** is gone (an orphan lost their parent). Match the word to who died.

Chapter 1 — Key Takeaways

- **OS** = referee between programs and hardware: **M**anages, **A**bstracts, **I**solates.
- **Kernel** runs privileged in **kernel space**; your apps run in **user space** and cross the boundary via **system calls** (the one door).
- **Process = private memory; Thread = shared memory.** Sharing makes threads fast but race-prone. *(★★★★★ — nail this.)*
- Processes cycle **Ready → Running → Waiting**; swapping between them is a **context switch** (pure overhead).
- Scheduling: **FCFS** (simple, convoy effect), **SJF** (best avg wait, starvation), **Round Robin** (fair, interactive).
- **Mutex** = 1 at a time, owned. **Semaphore** = up to N, not owned.
- **Deadlock** needs all four: **mutual exclusion, hold-and-wait, no preemption, circular wait.** Break one to prevent it. *(★★★★★)*
- **Virtual memory** gives each process a private space, backed by RAM + swap, implemented with **paging** (fixed pages ↔ frames; a miss = **page fault**).
- **Stack** = auto, fast, small, LIFO (overflow). **Heap** = manual/GC, large, flexible (leaks). *(★★★★★)*
- **Zombie** = child dead, not reaped. **Orphan** = parent dead, child adopted by PID 1.

Next: Chapter 2 — Networking. It's the biggest chapter, and "what happens when you type google.com?" is the most famous interview question of all. Take a break, then let's go.

Chapter 2 — Networking

> **Why this chapter matters most:** For On-Call, Support, and Cloud roles, networking *is* the job. When a service is down at 3 a.m., you're tracing a request from a browser to a server to a database, figuring out where it broke. Interviewers know this, so networking gets the most questions — and the most famous one of all lives here: *"What happens when you type google.com and press Enter?"* We'll build up to answering it perfectly.

Study the ★★★★★ topics until you can explain them to a non-technical friend. That's the bar.

2.1 The OSI Model ★★★★☆

Definition. The **OSI (Open Systems Interconnection) model** is a 7-layer framework describing how data moves across a network. Each layer has one job and talks only to the layers directly above and below it.

Why it exists. It's a shared vocabulary. When someone says "that's a Layer 7 load balancer" or "it's a Layer 4 problem," everyone instantly knows what part of the stack they mean. You rarely implement OSI directly, but you *talk* in its terms daily.

The 7 layers (top to bottom):

#	Layer	Job	Real-world example
7	**Application**	What the user interacts with	HTTP, DNS, SSH
6	**Presentation**	Formatting, encryption, compression	TLS/SSL, JPEG
5	**Session**	Opens/keeps/closes conversations	Sessions, sockets
4	**Transport**	Reliable/unreliable delivery, ports	**TCP, UDP**
3	**Network**	Routing between networks, addressing	**IP**, routers
2	**Data Link**	Node-to-node on the same network	Ethernet, MAC, switches
1	**Physical**	Actual bits on the wire/air	Cables, Wi-Fi radio

> 🧠 **Memory trick (top→bottom): "All People Seem To Need Data Processing"** — **A**pplication, **P**resentation, **S**ession, **T**ransport, **N**etwork, **D**ata link, **P**hysical.

> 🌍 **Analogy.** Sending a letter: you write it (Application), put in a standard envelope (Presentation), address it (Network), the post office guarantees delivery or not (Transport), trucks carry it road by road (Data Link), on physical roads (Physical).

The layers that actually come up: In practice, interviewers care most about **Layer 7 (Application — HTTP)**, **Layer 4 (Transport — TCP/UDP/ports)**, and **Layer 3 (Network — IP)**. If you're short on time, know those three deeply and the rest by name.

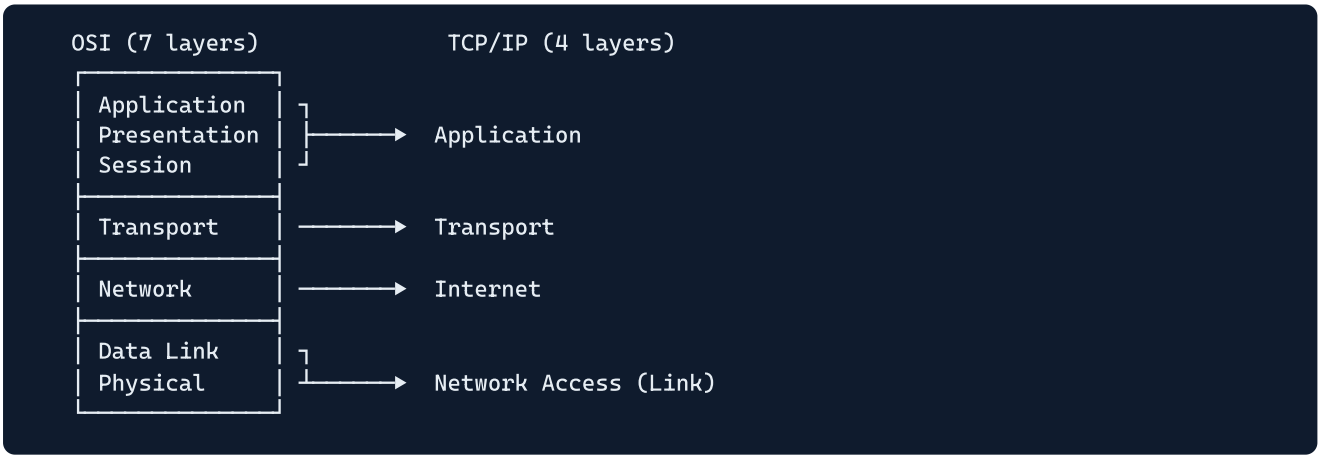
> 💬 **Interviewers usually ask:** "What layer does a load balancer / firewall / router work at?"
>
> ✅ **Model answer:** "A router works at Layer 3 — it routes by IP. A basic firewall filters at Layer 3/4 by IP and port. A load balancer can be Layer 4 (routing by IP/port, fast, no visibility into content) or Layer 7 (routing by HTTP details like URL path or headers, smarter but heavier)."

> ⚠️ **Common mistake:** Mixing up the order. Use the mnemonic. Also: TCP/UDP live at **Layer 4**, IP at **Layer 3** — a frequent slip.

2.2 TCP/IP Model ★★★☆☆

Definition. The **TCP/IP model** is the practical 4-layer model the actual internet runs on. It's a condensed version of OSI.

OSI vs TCP/IP mapping:



> ✅ **One-liner for interviews:** "OSI is the 7-layer *teaching* model; TCP/IP is the 4-layer *practical* model the internet actually uses. They map onto each other — OSI's top three collapse into TCP/IP's Application layer, and the bottom two into its Link layer."

2.3 TCP vs UDP ★★★★★

One of the most-asked networking questions. Master it.

Both are **Layer 4 (Transport)** protocols — they move data between applications. The difference is *reliability vs speed*.

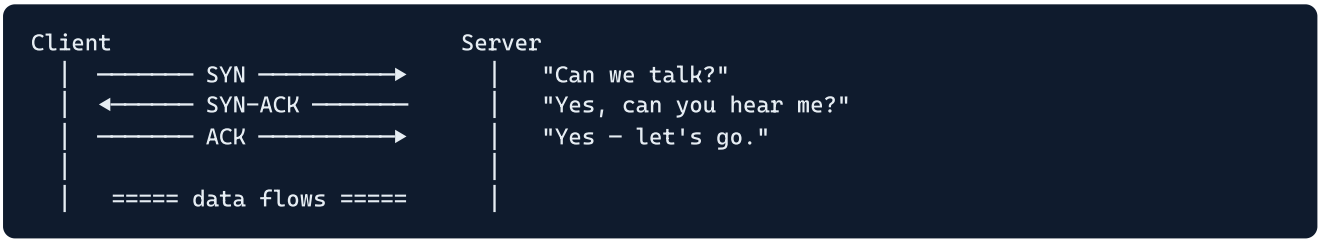
TCP — Transmission Control Protocol

Reliable, ordered, connection-based. It guarantees your data arrives, complete and in order, or you get an error. It does this with: - A **3-way handshake** to set up the connection (see below). - **Acknowledgements** — the receiver confirms each chunk; unconfirmed data is resent. - **Ordering** — segments are numbered and reassembled in order. - **Flow & congestion control** — slows down if the network is congested.

UDP — User Datagram Protocol

Fast, connectionless, "fire and forget." No handshake, no acknowledgements, no ordering guarantees. You send packets and hope they arrive. Less overhead = lower latency.

The 3-way handshake (TCP) — draw this:



The comparison table:

	TCP	UDP
Connection	Yes (handshake)	No
Reliable?	Yes — resends lost data	No — best effort
Ordered?	Yes	No
Speed	Slower (more overhead)	Faster (less overhead)
Use when	Correctness matters	Speed matters, some loss is OK
Examples	Web (HTTP/S), email, SSH, file transfer	Video calls, gaming, live streaming, DNS

> 🌎 **Analogy. TCP is a phone call** — you confirm the other person is there ("Hello?" "Yes, go ahead"), and if they miss a word they say "sorry, repeat that." **UDP is shouting across a crowded room** — fast, but if they miss a word, oh well, you've moved on.

> 💬 **Interviewers usually ask:** "TCP or UDP for a live video call, and why?"
>
> ✅ **Model answer:** "UDP. In a live call, a packet that arrives late is useless — you'd rather drop it and keep the conversation real-time than stall waiting for a retransmission. A dropped frame is a tiny glitch; a two-second delay to recover it ruins the call. TCP's guarantees are the wrong trade-off here. For something like a file download, it's the opposite — you'd use TCP because every byte must be correct."

> ⚠️ **Common mistake:** Saying "UDP is unreliable so nobody uses it." It's used constantly — DNS, video, gaming, VoIP — precisely *because* speed beats guaranteed delivery there.

> 🧠 **Memory trick: TCP = Trusted, Careful, Perfect. UDP = Unreliable, Direct, Prompt.**

2.4 Ports ★★★★☆

Definition. A **port** is a number (0–65535) that identifies a specific application or service on a machine. The **IP address gets you to the right computer; the port gets you to the right program on it.**

Why it exists. One server runs many services — a website, a database, SSH — all on the same IP. Ports let the OS deliver incoming data to the correct one.

> 🌍 **Analogy.** The IP address is a building's street address; the port is the apartment number. The mail truck finds the building by address, then the specific flat by number.

The well-known ports you MUST memorize:

Port	Service	Notes
22	SSH	Remote login
25	SMTP	Sending email
53	DNS	Name resolution (uses UDP mostly)
80	HTTP	Web (unencrypted)
443	HTTPS	Web (encrypted) — the one that matters most today
3306	MySQL	Database
5432	PostgreSQL	Database
6379	Redis	Cache
27017	MongoDB	Database

> 💬 **Interviewers usually ask:** "What port does HTTPS use? SSH? MySQL?"
>
> ✅ **Model answer:** "HTTPS is 443, HTTP is 80, SSH is 22, DNS is 53, MySQL is 3306, PostgreSQL is 5432. The IP address routes to the machine and the port routes to the specific service on it."

> 🐧 **Linux example:** `ss -tulnp` (or `netstat -tulnp`) lists which ports are open and which process owns each — the first thing you check when "the service won't connect."

> 🧠 **Memory trick: 22 SSH, 80 HTTP, 443 HTTPS, 53 DNS** — chant these four until automatic. They cover most questions.

2.5 DNS ★★★★★

Extremely common, and central to the "type google.com" question.

Definition. DNS (Domain Name System) translates human-friendly domain names (`google.com`) into machine IP addresses (`142.250.x.x`). It's often called "the phonebook of the internet."

Why it exists. Humans remember names; computers route by numbers. DNS bridges the two so you never have to memorize IP addresses.

How a DNS lookup works (know this flow):

```
You type google.com
      |
      ▼
1. Browser cache?  ── hit? done.
      | miss
      ▼
2. OS cache?       ── hit? done.
      | miss
      ▼
3. Recursive resolver (your ISP / 8.8.8.8)
      |
      ▼
4. Root server:  "ask the .com server" ──►
5. TLD (.com) server: "ask Google's nameserver" ──►
6. Authoritative server: "google.com = 142.250.1.2"
      |
      ▼
   Resolver caches it (TTL) and returns the IP to you
```

Common record types to know:

Record	Maps…	Example
A	name → IPv4	google.com → 142.250.1.2
AAAA	name → IPv6	→ 2607:f8b0::…
CNAME	name → another name (alias)	www → google.com
MX	domain → mail server	email routing
NS	domain → its nameservers	delegation

🌍 **Analogy.** DNS is asking directory assistance for a phone number. You know the *name* ("Joe's Pizza"); they give you the *number* to actually call. Your phone remembers it for a while (caching) so it doesn't ask again every time.

💬 **Interviewers usually ask:** "Walk me through what DNS does when you visit a website."

✅ **Model answer:** "DNS turns a domain name into an IP address. Your browser first checks its own cache, then the OS cache. If it's not cached, it asks a recursive resolver — usually your ISP's or something like 8.8.8.8. The resolver walks the hierarchy: it asks a root server, which points to the `.com` TLD server, which points to the domain's authoritative nameserver, which returns the actual IP. The resolver caches the answer for its TTL so the next lookup is instant. Then your browser can finally open a connection to that IP."

⚠️ **Common mistake:** Thinking DNS *transfers the web page*. It only returns the **IP address**. The page comes later, over HTTP, once you have the IP.

🐧 **Linux example:** `dig google.com` or `nslookup google.com` performs a lookup and shows the answer. `dig +trace google.com` shows the full root → TLD → authoritative walk.

🧠 **Memory trick:** DNS = **"Domain Name → System that finds the number."** Phonebook. Name in, number out.

2.6 DHCP ★★★☆☆

Definition. DHCP (Dynamic Host Configuration Protocol) automatically assigns a device an IP address (plus subnet mask, gateway, and DNS server) when it joins a network — so you don't configure each device by hand.

How it works — the DORA sequence:

```
Device ⟶ DISCOVER  ("Is there a DHCP server? I need an IP.")
Server ⟶ OFFER     ("Here's 192.168.1.42, want it?")
Device ⟶ REQUEST   ("Yes, I'll take it.")
Server ⟶ ACK       ("It's yours, lease = 24h. Here's your gateway + DNS.")
```

🌎 **Analogy.** Checking into a hotel. The front desk (DHCP server) assigns you a room number (IP) for the length of your stay (lease). You don't pick your own room; when you leave, it's freed for the next guest.

💬 **Interviewers usually ask:** "How does your laptop get an IP address on Wi-Fi?"

✅ **Model answer:** "Through DHCP. When it joins, it broadcasts a DISCOVER, a DHCP server responds with an OFFER of an available IP, the laptop sends a REQUEST for it, and the server ACKs with a lease. That handshake — Discover, Offer, Request, Ack — also hands over the subnet mask, default gateway, and DNS server addresses. It's leased, not permanent, so it can be reclaimed."

🧠 **Memory trick: DORA** explores and finds an address — **D**iscover, **O**ffer, **R**equest, **A**ck.

2.7 HTTP ★★★★★

Definition. HTTP (HyperText Transfer Protocol) is the Application-layer (Layer 7) protocol browsers and servers use to exchange web content. It's a **request/response** protocol: the client sends a request, the server sends a response.

Key property — HTTP is stateless. Each request is independent; the server doesn't remember previous requests on its own. (That's *why* cookies, sessions, and tokens exist — see §2.11–2.13.)

Anatomy of a request/response:

```
REQUEST                          RESPONSE
GET /index.html HTTP/1.1         HTTP/1.1 200 OK
Host: example.com                Content-Type: text/html
User-Agent: Chrome               Content-Length: 1024
Accept: text/html
                                 <html>...</html>
(method) (path) (version)        (version) (status) (body)
```

🌎 **Analogy.** HTTP is ordering by mail-order catalog. You send a slip saying exactly what you want (request); they mail back the item or a "sold out" notice (response). Each order is separate — the company doesn't remember you unless you include a membership card (cookie).

💬 **Interviewers usually ask:** "What does it mean that HTTP is stateless?"

✅ **Model answer:** "Every HTTP request is independent — the server doesn't inherently remember anything about previous requests from the same client. That keeps servers simple and scalable, but it means that to build something like a logged-in session, you need to carry state yourself, usually with a cookie holding a session ID or a token that you send with every request."

🧠 **Memory trick:** HTTP = **request in, response out, forgets you immediately** (stateless).

2.8 HTTPS, SSL & TLS ★★★★★

Definition. - **HTTPS** = HTTP + encryption. The same HTTP, but the connection is secured. - **TLS (Transport Layer Security)** is the protocol that does the encrypting. **SSL** is its older, now-deprecated predecessor — people still say "SSL" but mean TLS.

What HTTPS gives you (three things): 1. **Encryption** — eavesdroppers see scrambled data, not your password. 2. **Integrity** — data can't be tampered with in transit undetected. 3. **Authentication** — the certificate proves you're really talking to `google.com`, not an impostor.

How the TLS handshake works (conceptually — this comes up a lot):

```
1. Client: "Hello, here are the ciphers I support."
2. Server: "Hello, let's use this cipher. Here's my certificate."
3. Client: verifies the certificate against trusted Certificate Authorities (CAs).
4. Both: use asymmetric crypto to agree on a shared SECRET (session key).
5. From now on: fast SYMMETRIC encryption using that shared key.
```

The key insight interviewers want: HTTPS uses **asymmetric encryption** (slow, public/private keys) *only at the start* to safely agree on a shared **symmetric** key, then uses fast **symmetric** encryption for the actual data. Best of both worlds.

> 🌍 **Analogy.** Two people want to talk privately in public. They use a slow, secure method (asymmetric) to agree on a secret code word, then switch to speaking quickly in that code (symmetric) for the rest of the conversation. The certificate is like a government-issued ID proving one party is who they claim to be.

> 💬 **Interviewers usually ask:** "How does HTTPS work?" or "What's the difference between symmetric and asymmetric encryption in HTTPS?"
>
> ✅ **Model answer:** "HTTPS is HTTP over TLS. When you connect, there's a TLS handshake: the server presents a certificate that your browser verifies against trusted Certificate Authorities, which proves the server's identity. During the handshake they use asymmetric encryption — a public/private key pair — to securely agree on a shared session key. That asymmetric step is secure but slow, so it's only used to exchange the key. After that, both sides switch to fast symmetric encryption with the shared key for all the actual data. So you get authentication, encryption, and integrity, without the performance cost of asymmetric crypto for everything."

> ⚠️ **Common mistake:** Saying "HTTPS uses asymmetric encryption for everything." No — asymmetric is only for the handshake/key exchange; the bulk data is symmetric.

> 🧠 **Memory trick:** **A**symmetric = **A**greement on the key (once). **S**ymmetric = **S**peedy data (the rest). Certificate = **ID card**.

2.9 SSL vs TLS (quick clarity) ★★☆☆☆

> ✅ **One-liner:** "SSL is the old, insecure predecessor; TLS is its modern replacement. All SSL versions are deprecated and shouldn't be used. When people say 'SSL certificate,' they almost always mean a TLS certificate — the name just stuck."

2.10 Cookies ★★★★☆

Definition. A **cookie** is a small piece of data the server sends to the browser, which the browser stores and **sends back with every subsequent request** to that site. Cookies are how a stateless HTTP world remembers you.

How it flows:

```
1. You log in.
2. Server responds:  Set-Cookie: session_id=abc123
3. Browser stores it.
4. Every later request:  Cookie: session_id=abc123
5. Server sees the cookie → knows it's you → "Welcome back."
```

Cookie attributes worth naming: `HttpOnly` (JavaScript can't read it — protects against XSS), `Secure` (only sent over HTTPS), `Expires` / `Max-Age` (lifetime), `SameSite` (limits cross-site sending — CSRF protection).

> 🌎 **Analogy.** A cookie is the hand-stamp at a club. They stamp your hand on entry; every time you come back to the door, you show the stamp and they let you in without re-checking your ID.

> 💬 **Interviewers usually ask:** "What is a cookie and what is it used for?"
>
> ✅ **Model answer:** "A cookie is a small piece of data the server tells the browser to store and send back on every request to that site. Because HTTP is stateless, cookies are how the server recognizes a returning user — the classic use is holding a session ID after login so you stay logged in. Security-wise you flag them HttpOnly so JavaScript can't steal them, and Secure so they only travel over HTTPS."

> 🧠 **Memory trick:** Cookie = **the site's memory living in your browser.** Server sets it once, browser hands it back every time.

2.11 Sessions ★★★★☆

Definition. A **session** is server-side memory of a logged-in user. The server stores the user's data (who they are, their cart) and gives the browser only a **session ID** (usually in a cookie). On each request, the server looks up that ID to find the user's data.

Cookie vs Session — a key distinction:

```
COOKIE-based (data on client)      SESSION-based (data on server)
Browser holds actual data          Browser holds only an ID
                                    Server holds the real data
[user=joe; role=admin]             [session=abc] ⟶ server: {abc: joe, admin}
```

> 🌍 **Analogy.** A coat check. Instead of carrying your coat around (all the data), you get a numbered ticket (session ID). The cloakroom (server) holds the coat; you just show the ticket to get it back.

> 💬 **Interviewers usually ask:** "What's the difference between a cookie and a session?"
>
> ✅ **Model answer:** "A cookie is client-side storage in the browser; a session is server-side storage. Usually they work together: the server keeps the user's real data in a session and sends the browser a cookie containing just the session ID. On each request, the browser sends the ID back and the server looks up the session. Sessions keep sensitive data on the server, which is more secure, but they use server memory — which is why big systems sometimes move to stateless tokens like JWT instead."

> 🧠 **Memory trick: Cookie = client-side note. Session = server-side memory.** The cookie usually just carries the session's ticket number.

2.12 JWT (JSON Web Token) ★★★☆☆

Definition. A **JWT** is a self-contained, digitally-signed token that carries the user's identity and claims. Unlike a session, the server doesn't need to store anything — the token itself proves who you are because it's **signed** by the server.

Structure — three dot-separated parts:

```
header . payload . signature
eyJhbGc.eyJzdWIi.SflKxwRJ

header    = algorithm & type
payload   = claims (user id, role, expiry)  ← readable, NOT secret
signature = proves it wasn't tampered with  ← only server can create
```

Session vs JWT — the modern trade-off:

	Session	JWT
State stored	On server	In the token (stateless)
Scales across servers	Needs shared store	Easy — any server can verify
Revoke instantly	Yes (delete session)	Hard (valid until expiry)
Size	Tiny ID	Bigger token

⚠️ **Common mistake:** Thinking a JWT is *encrypted*. By default it's **signed, not encrypted** — anyone can read the payload (it's just Base64). The signature guarantees it wasn't *changed*, not that it's *secret*. Never put passwords in a JWT payload.

💬 **Interviewers usually ask:** "Session vs JWT — when would you use each?"

✅ **Model answer:** "A session stores state on the server and gives the client an ID; a JWT is stateless — the token itself carries the user's identity, signed by the server so it can't be forged. JWTs shine in distributed systems and APIs because any server can verify the signature without a shared session store. The downside is you can't easily revoke a JWT before it expires, and it's larger. Sessions are easier to revoke but need a shared store to scale. Also, a JWT is signed, not encrypted, so you never put secrets in the payload."

🧠 **Memory trick:** JWT = **a signed ID badge you carry.** The guard doesn't need a guest list (server state) — the badge's signature proves it's genuine.

2.13 REST APIs ★★★★☆

Definition. REST (Representational State Transfer) is a style for designing web APIs around **resources** (things like users, orders) identified by URLs, manipulated with standard **HTTP methods**. A REST API is how programs talk to a server over HTTP.

Core principles (name a couple): - **Resource-based URLs:** `/users/42`, not `/getUser?id=42`. - **HTTP methods carry the action:** GET to read, POST to create, etc. - **Stateless:** each request contains everything the server needs. - Returns data, usually **JSON**.

Good REST design example:

```
GET    /users        → list all users
GET    /users/42     → get user 42
POST   /users        → create a new user
PUT    /users/42     → replace/update user 42
DELETE /users/42     → delete user 42
```

🌎 **Analogy.** REST is a well-organized restaurant menu. Each dish (resource) has a clear name (URL), and you use standard actions — order (GET), add (POST), change (PUT), cancel (DELETE). Everyone understands the conventions without explanation.

💬 **Interviewers usually ask:** "What makes an API RESTful?"

✅ **Model answer:** "It's organized around resources addressed by URLs, and it uses standard HTTP methods for actions — GET to read, POST to create, PUT to update, DELETE to remove. It's stateless, so each request carries all the context the server needs, and it typically exchanges JSON. The idea is predictability: if you know the resource, you already know how to interact with it."

🧠 **Memory trick:** REST = **nouns in the URL, verbs in the HTTP method.** `/users/42` + `DELETE`.

2.14 HTTP Methods ★★★★☆

The verbs. Know what each does and two properties: **safe** (read-only) and **idempotent** (repeating it has the same effect as doing it once).

Method	Purpose	Idempotent?	Safe?
GET	Read data	Yes	Yes (read-only)
POST	Create new resource	**No** (repeats create duplicates)	No
PUT	Replace/update a resource	Yes	No
PATCH	Partially update	Not necessarily	No
DELETE	Remove a resource	Yes	No

⚠️ **Common mistake:** Saying POST and PUT are the same. **POST creates (not idempotent — hit it twice, get two records); PUT updates/replaces (idempotent — hit it twice, same result).** This distinction is a favorite.

💬 **Interviewers usually ask:** "What's the difference between PUT and POST?" or "What does idempotent mean?"

✅ **Model answer:** "POST creates a new resource and isn't idempotent — sending it twice creates two resources, which is why double-clicking 'submit' can double-charge you. PUT replaces a resource at a known URL and *is* idempotent — sending it twice leaves the same final state. Idempotent means repeating the request has no additional effect beyond the first, which matters for safe retries when a network call times out."

> 🧠 **Memory trick: GET reads, POST creates, PUT replaces, PATCH tweaks, DELETE removes.**
> POST is the odd one out — not idempotent.

2.15 HTTP Status Codes ★★★★★

Guaranteed to come up in support/on-call interviews. You must know the categories and the famous individual codes.

The five categories — memorize the pattern:

Range	Meaning	Memory hook
1xx	Informational	"hold on"
2xx	Success	"here you go"
3xx	Redirection	"go look over there"
4xx	Client error	"**you** messed up"
5xx	Server error	"**I** messed up"

The specific codes you must know:

Code	Name	Meaning
200	OK	Success
201	Created	Resource created (after POST)
301	Moved Permanently	Permanent redirect
302	Found	Temporary redirect
304	Not Modified	Use your cached copy
400	Bad Request	Malformed request
401	Unauthorized	You're not authenticated (log in)
403	Forbidden	Authenticated, but not allowed
404	Not Found	Resource doesn't exist
429	Too Many Requests	Rate limited
500	Internal Server Error	Generic server crash
502	Bad Gateway	Upstream server gave a bad response
503	Service Unavailable	Server overloaded / down
504	Gateway Timeout	Upstream server didn't respond in time

⚠️ **Common mistake:** Confusing **401 vs 403**. **401 = "I don't know who you are"** (not authenticated — log in). **403 = "I know who you are, and you can't do this"** (authenticated but not authorized). This exact question is asked constantly.

💬 **Interviewers usually ask:** "Difference between 401 and 403? What does a 502 mean?"

✅ **Model answer:** "401 Unauthorized means you're not authenticated — the server doesn't know who you are, so log in. 403 Forbidden means you *are* authenticated but you don't have permission for this resource. As for 5xx codes: 500 is a generic server error, 502 Bad Gateway means a proxy or load balancer got an invalid response from the upstream server behind it, 503 means the server is unavailable or overloaded, and 504 is a gateway timeout — the upstream didn't answer in time. In on-call, a wave of 502s or 504s usually points to a backend being down or slow, not the load balancer itself."

🧠 **Memory trick: 4xx = your fault (client), 5xx = server's fault.** And **401 = who are you? / 403 = no, not you.**

2.16 SSH ★★★★☆

Definition. SSH (Secure Shell) is an encrypted protocol for logging into and running commands on a remote machine over an untrusted network. Port **22**. It's how you actually get onto servers.

Password vs key-based auth (know this): You can log in with a password, but production uses **SSH key pairs** — a private key on your laptop and a public key on the server. You prove your identity with the private key without ever sending a password. More secure and scriptable.

```
ssh user@server.com              # log in
ssh -i mykey.pem ubuntu@1.2.3.4  # log in with a specific private key
scp file.txt user@server:/path/  # copy a file over SSH
```

🌍 **Analogy.** SSH key auth is a lock (public key, on the server's door) that only your specific physical key (private key, in your pocket) can open. You never hand over a copy of the key; you just prove you have it.

💬 **Interviewers usually ask:** "How do you securely connect to a remote server?"

✅ **Model answer:** "SSH, over port 22. It gives you an encrypted shell on the remote machine. In production you use key-based auth rather than passwords: you keep a private key locally and put the matching public key on the server, so you authenticate cryptographically without sending a password over the wire. That's both more secure and easy to automate. `scp` and `rsync` ride on top of SSH to copy files securely."

🧠 **Memory trick: SSH = Secure SHell, port 22, keys beat passwords.**

2.17 NAT ★★★☆☆

Definition. NAT (Network Address Translation) lets many devices on a private network share a single public IP address. Your router rewrites the source address of outgoing packets to its public IP, and remembers the mapping so replies get back to the right device.

Why it exists. There aren't enough IPv4 addresses for every device on Earth. NAT lets a whole home or office sit behind one public IP, using private ranges (like `192.168.x.x`) internally.

> 🌍 **Analogy.** NAT is a company receptionist. Outside callers all dial one public number. The receptionist (router) routes each call to the right internal extension (private IP) and remembers who's on which call so replies come back correctly.

> 💬 **Interviewers usually ask:** "How do many devices share one public IP?"
>
> ✅ **Model answer:** "NAT — Network Address Translation. Devices use private IPs internally, like 192.168.x.x. When they reach the internet, the router rewrites the source to its single public IP and keeps a table mapping each connection back to the right internal device, so responses are routed correctly. It's what lets a whole household browse the web through one public address, and it conserves the limited IPv4 space."

> 🧠 **Memory trick:** NAT = **one public face, many private people behind it** (the receptionist).

2.18 Firewall ★★★★☆

Definition. A **firewall** is a security barrier that controls network traffic based on rules — allowing or blocking packets by IP address, port, and protocol. It's the gatekeeper deciding what's allowed in and out.

Default-deny principle: good firewalls **block everything by default** and only allow explicitly permitted traffic. You open port 443 for your web server and 22 for SSH, and everything else stays shut.

> 🌍 **Analogy.** A firewall is a nightclub bouncer with a guest list. Only names/ports on the list get in; everyone else is turned away. Default-deny = "if you're not on the list, you're not coming in."

> 💬 **Interviewers usually ask:** "What does a firewall do?"
>
> ✅ **Model answer:** "It filters network traffic against a set of rules — typically by IP, port, and protocol — to allow or block connections. The best practice is default-deny: block everything, then open only the specific ports you need, like 443 for HTTPS and 22 for SSH. In the cloud, security groups are basically firewalls attached to your instances."

> 🧠 **Memory trick:** Firewall = **bouncer with a guest list. Default: not on the list = not getting in.**

2.19 Load Balancer ★★★★☆

Definition. A **load balancer** distributes incoming traffic across multiple backend servers, so no single server is overwhelmed. It also improves reliability — if one server dies, traffic goes to the healthy ones.

Why it exists. One server can't handle millions of users, and if it's your only server, it's a single point of failure. A load balancer gives you both **scalability** (spread the load) and **high availability** (survive server failures).

Common algorithms: Round Robin (rotate through servers evenly), **Least Connections** (send to the least busy), **IP Hash** (same client always to the same server).

Layer 4 vs Layer 7: A **L4** load balancer routes by IP/port — fast, but blind to content. An **L7** load balancer reads HTTP, so it can route by URL path or headers (e.g., `/api` → one pool, `/images` → another).

> 🌍 **Analogy.** A load balancer is the person at a bank directing customers to whichever teller is free. No single teller gets swamped, and if one goes on break, customers are simply sent to the others.

> 💬 **Interviewers usually ask:** "What's a load balancer and why do you need one?"
>
> ✅ **Model answer:** "It sits in front of a group of servers and spreads incoming requests across them, using an algorithm like round robin or least-connections. It gives you two things: scalability, because you can add servers to handle more load, and high availability, because it health-checks the backends and stops sending traffic to any that fail. A Layer 4 balancer routes by IP and port; a Layer 7 one understands HTTP and can route by things like the URL path."

> 🧠 **Memory trick:** Load balancer = **the traffic cop spreading cars across open lanes.** Also removes crashed lanes (health checks).

2.20 Proxy vs Reverse Proxy ★★★★☆

A favorite "do you really understand it" question. The difference is *which side it sits on*.

Forward proxy — sits in front of **clients**. Clients send requests through it; the server sees the proxy, not the real client. Used for anonymity, caching, and content filtering (e.g., a corporate proxy blocking sites).

Reverse proxy — sits in front of **servers**. Clients think they're talking to the reverse proxy; it forwards to the real backend servers. Used for load balancing, SSL termination, caching, and hiding the backend. (Nginx is the classic example.)

```
FORWARD PROXY (protects/represents CLIENTS)
[Clients] ──► [Forward Proxy] ──► Internet ──► [Server]
              server sees the proxy, not you

REVERSE PROXY (protects/represents SERVERS)
[Client] ──► Internet ──► [Reverse Proxy] ──► [Backend Servers]
             you see the proxy, not the real servers
```

🌍 **Analogy.** A **forward proxy** is like sending your assistant to make purchases on your behalf — the shop never sees you. A **reverse proxy** is like a company receptionist — you talk to reception, and they route you to whichever employee should handle it; you never see the org chart behind the desk.

💬 **Interviewers usually ask:** "What's the difference between a proxy and a reverse proxy?"

✅ **Model answer:** "It's about which side it protects. A forward proxy sits in front of clients — the client sends requests through it, so the destination server sees the proxy instead of the real client. It's used for caching, filtering, or anonymity. A reverse proxy sits in front of servers — the client thinks it's talking to the proxy, which forwards to backend servers behind it. Reverse proxies handle load balancing, SSL termination, caching, and hiding the backend. Nginx is the go-to example, and a reverse proxy is often also acting as a load balancer."

🧠 **Memory trick: Forward proxy = for the client. Reverse proxy = for the server.** Match the "R" in reveRse to seRveR.

2.21 VLAN ★★☆☆☆

Definition. A **VLAN (Virtual LAN)** logically splits one physical network into several isolated virtual networks. Devices on different VLANs can't talk directly, even on the same switch, without a router.

✅ **One-liner:** "A VLAN lets you carve one physical switch into multiple isolated networks — say, separating Guest Wi-Fi from the internal company network — for security and to reduce broadcast traffic, without buying separate hardware." *(Low frequency for these roles — know the one-liner and move on.)*

2.22 🎯 THE Big One: "What happens when you type google.com and press Enter?"

This is the most famous interview question in the industry. It ties together this entire chapter. A strong answer signals you understand the whole stack. Here's the full walkthrough — practice saying it end to end.

```
1. DNS RESOLUTION
   Browser cache → OS cache → resolver → root → .com TLD →
   authoritative server → returns google.com's IP.

2. TCP CONNECTION
   Browser opens a TCP connection to that IP on port 443,
   via the 3-way handshake (SYN, SYN-ACK, ACK).

3. TLS HANDSHAKE (because HTTPS)
   Server presents its certificate; browser verifies it against a CA;
   both agree on a symmetric session key. Connection is now encrypted.

4. HTTP REQUEST
   Browser sends: GET / HTTP/1.1, Host: google.com (encrypted).

5. SERVER PROCESSING
   Request may hit a load balancer → reverse proxy → app server →
   possibly a database → builds the response.

6. HTTP RESPONSE
   Server returns 200 OK plus the HTML.

7. BROWSER RENDERS
   Parses HTML, requests more resources (CSS, JS, images —
   each possibly its own DNS/TCP/TLS/HTTP), builds the page.
```

> ✅ **Model answer (say this out loud):** "First, DNS resolves google.com to an IP — the browser checks its cache, then the OS, then a recursive resolver walks from the root to the .com server to Google's authoritative nameserver. With the IP, the browser opens a TCP connection on port 443 using the three-way handshake. Because it's HTTPS, there's then a TLS handshake — the server sends a certificate the browser verifies against a trusted CA, and they agree on a symmetric session key, so everything after is encrypted. The browser sends an HTTP GET request. On the server side that may pass through a load balancer and a reverse proxy to an application server, maybe hitting a database, and comes back as a 200 OK with HTML. Finally the browser parses the HTML and fetches the CSS, JavaScript, and images it references — each possibly repeating that whole process — and renders the page."

> 🧠 **Memory trick — the sequence is: DNS → TCP → TLS → HTTP → (server) → response → render.** Chant it: *"Do The Task, Handle The Response, Render."*

Chapter 2 — Key Takeaways

- **OSI = 7 layers** ("All People Seem To Need Data Processing"); the ones that matter are **L7 HTTP, L4 TCP/UDP, L3 IP**.
- **TCP** = reliable/ordered/slow (web, email). **UDP** = fast/best-effort (video, DNS, gaming). *(★★★★★)*
- **Ports:** 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS, 3306 MySQL, 5432 Postgres.
- **DNS** turns names into IPs via a caching hierarchy (root → TLD → authoritative). It returns an **IP, not the page**. *(★★★★★)*
- **DHCP** auto-assigns IPs via **DORA**.

- **HTTP is stateless** → that's *why* cookies/sessions/JWT exist.
- **HTTPS** = HTTP + TLS: certificate proves identity; asymmetric agrees a key; symmetric encrypts the data. *(★★★★★)*
- **Cookie** = client-side; **session** = server-side (cookie carries the ID); **JWT** = stateless signed token (signed ≠ encrypted).
- **HTTP methods:** GET/POST/PUT/PATCH/DELETE. POST creates (not idempotent), PUT updates (idempotent).
- **Status codes:** 4xx = client's fault, 5xx = server's fault. **401 = who are you; 403 = not allowed.** *(★★★★★)*
- **NAT** = many private IPs behind one public. **Firewall** = default-deny gatekeeper. **Load balancer** = spread + survive. **Forward proxy = for clients; reverse proxy = for servers.**
- **The big question:** DNS → TCP → TLS → HTTP → server → response → render. Practice it out loud.

> **Next:** Chapter 3 — Linux. Now we go hands-on with the commands interviewers actually make you use.

Chapter 3 — Linux

> **Why this chapter matters:** For On-Call, Support, and Cloud roles, Linux is where you *live*. Interviewers won't ask you to recite man pages — they'll say "a server's disk is full, what do you do?" or "how do you find which process is eating the CPU?" and watch how you think. This chapter teaches the commands that answer those questions, grouped by the job they do, so you remember them by *purpose*, not by rote.

> 🐧 **Golden rule for the interview:** You don't need to memorize every flag. You need to know *which tool* solves *which problem*, and roughly how to invoke it. "I'd use `du -sh *` to find what's eating the disk" beats reciting options you'll never use.

3.1 The mental model: everything is a file

Before the commands, one idea that makes Linux click and impresses interviewers:

> **In Linux, almost everything is a file** — documents, directories, devices (your disk is `/dev/sda`), even some system info (`/proc`). This uniformity is why the same tools (`cat`, `grep`, redirection) work on nearly everything.

And the filesystem is a single tree starting at `/` (root):

```
/                 the root of everything
├── home/         users' home directories (/home/alice)
├── etc/          configuration files (/etc/nginx/…)
├── var/          variable data — logs live in /var/log
├── usr/          installed programs
├── tmp/          temporary files
├── bin/, sbin/   essential commands
└── dev/, proc/   devices & kernel info (virtual)
```

> 🧠 **Memory trick:** Config in `/etc`, logs in `/var/log`, your stuff in `/home`. Those three cover 90% of what you touch on the job.

3.2 Navigation & basics ★★★☆☆

Command	What it does	Example
`pwd`	**P**rint **w**orking **d**irectory (where am I?)	`pwd` → `/home/alice`
`ls`	List directory contents	`ls -la` (all + details)
`cd`	Change directory	`cd /var/log`, `cd ..` (up), `cd ~` (home)

`ls` flags worth knowing: `-l` long/detailed, `-a` show hidden (dotfiles), `-h` human-readable sizes, `-t` sort by time. Combine them: `ls -lath` = detailed, all files, human sizes, newest first.

```
$ ls -la
drwxr-xr-x  2 alice staff  4096 Jul 27 10:00 .
-rw-r--r--  1 alice staff  1024 Jul 27 09:00 notes.txt
#   ↑ permissions      ↑ size  ↑ modified    ↑ name
```

> 💬 **Interviewers usually ask:** "How do you see hidden files?" ✅ **Answer:** "`ls -a` — hidden files in Linux start with a dot, like `.bashrc`, and `-a` reveals them."

> 🧠 **Memory trick:** `pwd` = "**P**osition, **W**here's my **D**irectory." `cd ..` goes **up**; `cd ~` goes **home**.

3.3 Creating & managing files ★★★☆☆

Command	What it does	Example
`mkdir`	Make a directory	`mkdir logs`, `mkdir -p a/b/c` (nested)
`touch`	Create an empty file / update timestamp	`touch app.log`
`cp`	Copy	`cp file.txt backup.txt`, `cp -r dir/ dir2/` (recursive)
`mv`	Move **or rename**	`mv old.txt new.txt` (rename), `mv f.txt /tmp/` (move)
`rm`	Remove	`rm file.txt`, `rm -r dir/` (recursive)

> ⚠️ **Common mistake / interview trap:** `mv` does **both** moving and renaming — there's no separate rename command. And `rm -rf /` is the infamous "delete everything" disaster. **`rm` has no recycle bin** — deleted is gone.

> 💬 **Interviewers usually ask:** "How do you rename a file in Linux?" ✅ **Answer:** "`mv oldname newname`. Linux uses `mv` for both moving and renaming — renaming is just moving to a new name in the same directory."

> 🧠 **Memory trick:** `-r` / `-R` = **recursive** = "and everything inside it." Needed for directories with `cp` and `rm`.

3.4 Viewing file contents ★★★★☆

Support engineers read logs constantly. This group is high-yield.

Command	What it does	When to use
`cat`	Dump whole file to screen	Small files
`less`	Scrollable pager (q to quit)	Large files — the safe default
`head`	First N lines (default 10)	`head -20 file` — top of a file
`tail`	Last N lines (default 10)	`tail -50 file` — recent log entries
`tail -f`	**Follow** — stream new lines live	Watching a log in real time

`tail -f` is the star of on-call work:

```
tail -f /var/log/nginx/error.log     # watch errors as they happen
tail -100 /var/log/syslog            # last 100 lines
```

> 🚂 **In real production:** When you deploy a change and want to see if it breaks, you `tail -f` the log and watch. When an interviewer asks "how do you watch a log live?", the answer is `tail -f`. Knowing this one command signals real hands-on experience.

> 💬 **Interviewers usually ask:** "How do you view the last 50 lines of a log, and how do you watch it update live?" ✅ **Answer:** "`tail -50 file` for the last 50 lines. To watch it update in real time, `tail -f file` — the `-f` follows the file and prints new lines as they're written. That's my go-to when debugging a live service."

> 🧠 **Memory trick:** `head` = top, `tail` = bottom, **`tail -f` = follow the flow.** For big files, `less` is more (safer than `cat`).

3.5 Searching: find & grep ★★★★★

These two are asked in nearly every Linux interview.

`find` — locate files by name, type, size, age

```
find /var/log -name "*.log"          # all .log files under /var/log
find . -type f -name "config*"       # files named config* here & below
find / -size +100M                   # files bigger than 100 MB (disk hunting!)
find . -mtime -1                     # modified in the last 1 day
find . -name "*.tmp" -delete         # find and delete .tmp files
```

`grep` — search *inside* files for text

```
grep "error" app.log                 # lines containing "error"
grep -i "error" app.log              # case-insensitive
grep -r "TODO" .                     # recursive — search all files in tree
grep -n "error" app.log              # show line numbers
grep -v "debug" app.log              # invert — lines WITHOUT "debug"
grep -c "error" app.log              # count matching lines
```

🏭 **In real production:** The combo `grep -i error /var/log/app.log | tail -20` — "show me the last 20 error lines, case-insensitive" — is something you'll type a hundred times. (Chapter 5 goes deeper on grep.)

💬 **Interviewers usually ask:** "Difference between find and grep?" ✅ **Model answer:** "`find` searches for *files* by their attributes — name, type, size, modification time. `grep` searches for *text inside* files. So `find` answers 'where is the file?' and `grep` answers 'which files or lines contain this text?' You often combine them — `find` to locate files, then `grep` to search within them."

🧠 **Memory trick: find = find the file. grep = grab the text (inside).**

3.6 Permissions: chmod & chown ★★★★☆

Definition. Every file has an **owner**, a **group**, and permissions for three classes: owner (**u**ser), **g**roup, and **o**thers. Each class can have **r**ead, **w**rite, e**x**ecute.

Reading `ls -l` output:

```
-rwxr-xr--
|└┘└┘└┘
| |  |  └── others:  r--  (read only)
| |  └───── group:   r-x  (read + execute)
| └──────── owner:   rwx  (read + write + execute)
└─────────── type: - file, d directory, l link
```

The numeric (octal) system — this is what interviewers test:

Number	Permission	Binary
4	read (r)	100
2	write (w)	010
1	execute (x)	001

Add them up per class. So `chmod 755` = owner 7 (4+2+1 = rwx), group 5 (4+1 = r-x), others 5 (r-x). And `chmod 644` = owner rw-, group r–, others r– (the typical file default).

```
chmod 755 script.sh        # rwxr-xr-x — runnable script
chmod 644 notes.txt        # rw-r--r-- — normal file
chmod +x script.sh         # add execute (symbolic form)
chown alice file.txt       # change owner to alice
chown alice:devs file.txt  # change owner and group
```

> 💬 **Interviewers usually ask:** "What does chmod 755 mean?" or "How do you make a script executable?" ✅ **Model answer:** "755 sets owner to read-write-execute — that's 4+2+1 — and group and others to read-execute, which is 4+1. It's the standard for scripts and directories: the owner can modify and run it, everyone else can run and read it. To make a script executable specifically, I'd use `chmod +x script.sh`. And 644 — read-write for owner, read-only for everyone — is the typical setting for a plain data file."

> 🧠 **Memory trick: 4 = read, 2 = write, 1 = execute.** Full access = 7 (4+2+1). Read+execute = 5. **755 = "me full, everyone look-and-run."**

3.7 Processes: ps, top, kill ★★★★★

Core on-call skill: find the misbehaving process and deal with it.

`ps` — snapshot of running processes

```
ps aux                     # ALL processes, detailed (the one to remember)
ps aux | grep nginx        # find a specific process
```

Columns: `USER PID %CPU %MEM ... COMMAND`. **PID** (process ID) is what you need to kill it.

`top` (and `htop`) — live, updating process view

```
top                        # live dashboard, sorted by CPU. Press q to quit.
```

Shows CPU%, memory, load average, per-process usage — refreshing every few seconds. Your first stop when "the server is slow."

`kill` — send a signal to a process

```
kill 1234                # polite: SIGTERM — "please shut down cleanly"
kill -9 1234             # forceful: SIGKILL — "die now" (last resort)
killall nginx            # kill all processes named nginx
```

⚠️ **Common mistake / key interview point:** Reaching for `kill -9` first. `kill` **(SIGTERM) asks the process to shut down gracefully** — it can save state and clean up. `kill -9` **(SIGKILL) is un-ignorable and immediate** — it can leave corrupt files or orphaned resources. Always try `kill` first; use `-9` only when it won't die.

💬 **Interviewers usually ask:** "A process is stuck using 100% CPU — walk me through what you'd do." ✅ **Model answer:** "First I'd run `top` to confirm which process it is and grab its PID. I'd check whether it's genuinely stuck or just busy. To stop it, I'd start with `kill <PID>`, which sends SIGTERM and lets it shut down gracefully — flush data, close files. If it ignores that and stays stuck, then `kill -9 <PID>`, which is SIGKILL and can't be caught, but I use it as a last resort because it doesn't let the process clean up. Then I'd look at logs to find *why* it spun up in the first place."

🧠 **Memory trick: `kill` = ask nicely (SIGTERM 15). `kill -9` = force (SIGKILL). Ask before you force.**

3.8 Disk & memory: df, du, free ★★★★★

"The disk is full" is one of the most common real on-call pages. Know these three.

`df` — disk free: how full are the filesystems?

```
df -h                # -h = human-readable (GB/MB). Shows % used per mount.
```

`du` — disk usage: what's taking up the space?

```
du -sh *                        # size of each item in current dir, summarized
du -sh /var/log/*               # what's big inside /var/log
du -h / | sort -rh | head -20   # top 20 biggest things (classic combo)
```

`free` — memory usage

```
free -h              # RAM and swap, human-readable
```

🏭 **In real production:** "Disk full" alert → `df -h` to see *which* disk is full → `du -sh /var/*` (or wherever) to find the culprit, drilling down → usually it's runaway logs in `/var/log`. This exact drill is a top interview scenario.

💬 **Interviewers usually ask:** "The server's disk is full. How do you find what's using the space?" ✅ **Model answer:** "I'd start with `df -h` to see which filesystem is full and how full. Then I'd drill into that mount with `du -sh *`, moving into whichever directory is largest, to find the culprit — often it's runaway logs under `/var/log`. A handy one-liner is `du -h /path | sort -rh | head` to rank the biggest directories. Once I find it, I'd clear or rotate the offending files rather than blindly deleting, and I'd check *why* they grew — maybe log rotation is broken."

⚠️ **Common mistake:** Confusing `df` and `du`. **`df` = free space overview (the whole disk). `du` = usage of specific files/dirs (the detail).** You use `df` to spot the problem, `du` to hunt it down.

🧠 **Memory trick: d**isk **f**ree = **df** (big picture). **d**isk **u**sage = **du** (dig in). **free** = memory.

3.9 Services & logs: systemctl & journalctl ★★★★☆

Modern Linux manages services (background programs like web servers, databases) with **systemd**. Two commands run it.

`systemctl` — control services

```
systemctl status nginx      # is it running? recent logs? (most-used)
systemctl start nginx       # start it
systemctl stop nginx        # stop it
systemctl restart nginx     # restart (after a config change)
systemctl enable nginx      # start automatically on boot
```

`journalctl` — read systemd logs

```
journalctl -u nginx         # logs for the nginx service
journalctl -u nginx -f      # follow live (like tail -f)
journalctl -xe              # recent logs with errors (great for debugging)
journalctl --since "1 hour ago"
```

🏭 **In real production:** "The website is down." Step one: `systemctl status nginx` — is the service even running? If it crashed, `journalctl -u nginx -e` shows *why*. This two-command reflex is exactly what interviewers want to hear.

> 💬 **Interviewers usually ask:** "How do you check if a service is running, and how do you restart it?"
> ✅ **Model answer:** "`systemctl status <service>` tells me if it's active, when it last started, and shows recent log lines. To restart after a config change, `systemctl restart <service>`, or `reload` if I only changed config and don't want downtime. And `enable` makes it start on boot. If it won't start, I check the logs with `journalctl -u <service> -e` to see the error."

> 🧠 **Memory trick: systemctl = control the service. journalctl = read the service's diary (logs).** `status` first, always.

3.10 Other handy commands ★★★☆☆

Command	What it does	Example
`history`	Show your recent commands	`history | grep ssh`
`man`	Manual for a command	`man grep` (q to quit)
`chmod` / `chown`	Permissions/ownership	(see §3.6)
`wc`	Word/line/char count	`wc -l file` (count lines)
`ln -s`	Symbolic link	`ln -s /path/target linkname`
`which`	Where is a command?	`which python`
`sudo`	Run as superuser (root)	`sudo systemctl restart nginx`
`chmod +x`	Make executable	(see §3.6)

> 🧠 **Memory trick:** `sudo` = "**s**uper**u**ser **do**" = run this one command as root. Use it for anything system-level (services, installing software, editing `/etc`).

3.11 Piping & redirection — the Linux superpower ★★★★☆

This ties every command together, and interviewers love it. The Unix philosophy: small tools that each do one thing, combined with **pipes**.

The pipe `|` sends one command's output into the next command's input:

```
ps aux | grep nginx              # list processes, keep only nginx lines
cat access.log | grep 404 | wc -l   # count 404s in a log
du -sh * | sort -rh | head -5    # 5 biggest items here
```

Redirection sends output to (or input from) files:

```
echo "hello" > file.txt      # > overwrites file with output
echo "more" >> file.txt      # >> appends to file
command 2> errors.log        # 2> redirects error output (stderr)
command > out.log 2>&1       # send both output and errors to out.log
sort < unsorted.txt          # < feeds a file as input
```

🌍 **Analogy.** Pipes are a factory assembly line. Each machine (command) does one job and passes the product to the next. `ps aux | grep nginx | wc -l` = "list everything → keep nginx lines → count them" = "how many nginx processes are running?"

💬 **Interviewers usually ask:** "How would you count how many times 'error' appears in a log?" ✅ **Model answer:** "`grep -c error app.log` directly counts matching lines. If I wanted to build it up with a pipe, `cat app.log | grep error | wc -l` — read the file, filter to error lines, count them. Pipes let you chain small tools into exactly the query you need, which is the whole Unix philosophy."

🧠 **Memory trick: `|` passes data between commands; `>` writes to a file (overwrite), `>>` appends.** Pipe = assembly line.

Chapter 3 — Key Takeaways

- **Everything is a file**, filesystem is one tree from `/`. Config in `/etc`, logs in `/var/log`, users in `/home`.
- **Viewing logs:** `tail -f` to watch live, `head` / `tail` for ends, `less` for big files. *(★★★★☆)*
- **Searching:** `find` = locate *files*, `grep` = find *text inside*. *(★★★★★)*
- **Permissions:** `4 read, 2 write, 1 execute`. `755` = owner full, others read+run; `644` = normal file. `chmod +x` to make runnable. *(★★★★☆)*
- **Processes:** `ps aux` (snapshot), `top` (live), `kill` (SIGTERM, polite) vs `kill -9` (SIGKILL, force). *(★★★★★)*
- **Disk/memory:** `df -h` (which disk is full) → `du -sh *` (what's using it) → `free -h` (memory). *(★★★★★)*
- **Services:** `systemctl status/start/stop/restart`, `journalctl -u <svc>` for logs. `status` first. *(★★★★☆)*
- **Pipes & redirection:** `|` chains commands, `>` overwrites, `>>` appends. The glue that makes Linux powerful.

Next: Chapter 4 — Bash scripting. Now we combine these commands into scripts that automate the boring stuff.

Chapter 4 — Bash Scripting

> **Why this chapter matters:** Nobody expects a junior engineer to write elegant 500-line programs in Bash. But *every* on-call and support role expects you to automate small, repetitive tasks — check if a service is up, clean old logs, back up a folder, alert when disk is full. Interviewers ask Bash to see if you can turn a manual chore into a script. This chapter gives you the building blocks and 10 real scripts you could genuinely use on the job.

> 🧠 **The one thing to remember:** A Bash script is just **the same commands you'd type at the terminal, saved in a file and run top to bottom.** If you know the commands from Chapter 3, you're 80% there.

4.1 Your first script — the anatomy ★★★★☆

```
#!/bin/bash
# This line above is the "shebang" — it tells the OS to run this with bash.

echo "Hello, on-call world!"
```

To run it:

```
chmod +x hello.sh    # make it executable (remember chmod from Ch.3!)
./hello.sh           # run it
```

The shebang `#!/bin/bash` must be the first line. It tells the system which interpreter to use. Without it, the OS doesn't know the file is a Bash script.

> 💬 **Interviewers usually ask:** "What's the first line of a shell script for?" ✅ **Answer:** "The shebang, `#!/bin/bash`. It tells the operating system which interpreter should run the script — in this case Bash. Without it, the system doesn't know how to execute the file."

> 🧠 **Memory trick:** Shebang = **She-Bang!** (`#!`) — the bang that *launches* the right interpreter.

4.2 Variables ★★★★☆

```bash
#!/bin/bash
name="Alice"           # NO spaces around = (this is the #1 beginner error)
count=5

echo "Hello $name"     # use $ to read a variable
echo "Count is ${count}"   # ${} braces when you need clear boundaries

# Command substitution — capture a command's output into a variable:
today=$(date +%Y-%m-%d)
files=$(ls | wc -l)
echo "Today is $today, there are $files files here."
```

⚠️ **Common mistake #1 (interviewers love catching this):** Spaces around `=`. `name = "Alice"` is **wrong** — Bash thinks `name` is a command. It must be `name="Alice"` with no spaces.

⚠️ **Common mistake #2:** Forgetting the `$` when *reading* a variable. You assign with `name=...` but read with `$name`.

🧠 **Memory trick: Assign without `$`, read with `$`.** And **no spaces around `=`, ever.** `$(...)` captures command output.

4.3 User input & output ★★★☆☆

```bash
#!/bin/bash
echo "What's your name?"
read name                    # read input into $name
echo "Hello, $name!"

read -p "Enter a number: " num    # -p prints the prompt inline
echo "You typed $num"

read -sp "Password: " pass         # -s hides input (silent) for secrets
echo    # newline after hidden input
```

- `echo` prints output (add `-n` to skip the trailing newline).
- `read` gets input from the user. `-p` shows a prompt, `-s` hides typing (for passwords).

🧠 **Memory trick:** `echo` = **out**, `read` = **in**. `-p` = prompt, `-s` = secret.

4.4 Arguments ★★★★☆

Scripts take arguments from the command line — `./backup.sh /home/data` — accessed with special variables. **This is high-yield because real scripts are almost always parameterized.**

```
#!/bin/bash
echo "Script name: $0"       # the script's own name
echo "First arg:   $1"       # first argument
echo "Second arg:  $2"       # second argument
echo "All args:    $@"       # all arguments
echo "How many:    $#"       # count of arguments
```

Running `./greet.sh Alice Bob` gives: `$1`=Alice, `$2`=Bob, `$#`=2, `$@`=Alice Bob.

Always check arguments were provided:

```
if [ $# -eq 0 ]; then
    echo "Usage: $0 <filename>"
    exit 1                    # exit with error code 1
fi
```

> 💬 **Interviewers usually ask:** "How does a shell script access command-line arguments?" ✅
> **Answer:** "Positional parameters: `$1` is the first argument, `$2` the second, and so on. `$0` is the script name itself, `$@` is all arguments, and `$#` is the count. I always check `$#` at the top to make sure the user passed what's required, and print a usage message and `exit 1` if not."

> 🧠 **Memory trick:** `$1 $2 $3` = the args in order. `$#` = # of args (# = "number"). `$@` = @ll of them. `$0` = the script itself.

4.5 Conditions (if statements) ★★★★★

The most important control structure. The bracket syntax and operators trip up almost everyone — study this carefully.

```
#!/bin/bash
num=10

if [ $num -gt 5 ]; then
    echo "Greater than 5"
elif [ $num -eq 5 ]; then
    echo "Exactly 5"
else
    echo "Less than 5"
fi
```

Number comparisons (memorize — these are asked):

Operator	Meaning	Think
`-eq`	equal	**eq**ual
`-ne`	not equal	**n**ot **e**qual
`-gt`	greater than	**g**reater **t**han
`-lt`	less than	**l**ess **t**han
`-ge`	greater or equal	
`-le`	less or equal	

String comparisons: use `=`, `≠` (and quote your variables):

```
if [ "$name" = "admin" ]; then echo "Welcome admin"; fi
if [ -z "$name" ]; then echo "Name is empty"; fi   # -z = zero length
```

File tests (very common in ops scripts):

Test	True if...
`-f file`	file exists and is a regular file
`-d dir`	directory exists
`-e path`	path exists (file or dir)
`-r` / `-w` / `-x`	readable / writable / executable
`-z "$var"`	variable is empty
`-n "$var"`	variable is non-empty

```
if [ -f /etc/nginx/nginx.conf ]; then
    echo "Config exists"
else
    echo "Config missing!"
fi
```

⚠️ **Common mistakes:** (1) Forgetting **spaces inside the brackets** — `[$num -gt 5]` fails; it must be `[ $num -gt 5 ]`. (2) Using `>` for number comparison — that's redirection! Use `-gt`. (3) Not quoting variables — `[ $name = x ]` breaks if `$name` is empty or has spaces; use `[ "$name" = x ]`.

💬 **Interviewers usually ask:** "How do you check if a file exists in a Bash script?" ✅ **Answer:** "`if [ -f /path/to/file ]; then ... fi`. The `-f` test is true when the file exists and is a regular file; `-d` checks a directory and `-e` checks either. In ops scripts I use this constantly — for example, checking a config or log file exists before acting on it."

> 🧠 **Memory trick: Spaces inside `[ ]` are mandatory.** Numbers use `-eq -ne -gt -lt`; strings use `= ≠`. Files use `-f -d -e`.

4.6 Loops ★★★★☆

for loop — iterate over a list

```
for i in 1 2 3 4 5; do
    echo "Number $i"
done

for file in *.log; do          # loop over all .log files
    echo "Processing $file"
done

for i in $(seq 1 100); do      # 1 to 100
    echo $i
done
```

while loop — repeat while a condition holds

```
count=1
while [ $count -le 5 ]; do
    echo "Count: $count"
    count=$((count + 1))       # arithmetic: $(( ... ))
done
```

Reading a file line by line (a genuinely useful pattern):

```
while read line; do
    echo "Line: $line"
done < input.txt
```

> 🧠 **Memory trick:** `for` = "for each thing in a list." `while` = "keep going while true." Arithmetic goes in `$(( ))`.

4.7 Functions ★★★☆☆

```bash
#!/bin/bash

greet() {
    echo "Hello, $1!"          # functions use $1, $2 for THEIR arguments
}

check_service() {
    if systemctl is-active --quiet "$1"; then
        echo "$1 is running"
        return 0               # 0 = success
    else
        echo "$1 is DOWN"
        return 1               # non-zero = failure
    fi
}

greet "Alice"
check_service "nginx"
```

- Functions keep scripts organized and avoid repetition.
- They take arguments the same way scripts do: `$1`, `$2`.
- `return` gives an **exit code** (0 = success), not a value — capture actual output with `echo` + `$( ... )`.

> 🧠 **Memory trick:** Functions reuse the `$1 $2` argument system. **`return 0` = success**, like the rest of Unix (0 is good, non-zero is an error).

4.8 Arrays ★★☆☆☆

```bash
servers=("web1" "web2" "web3")

echo "${servers[0]}"        # first element → web1
echo "${servers[@]}"        # all elements
echo "${#servers[@]}"       # count → 3

for s in "${servers[@]}"; do
    echo "Pinging $s"
done
```

> 🧠 **Memory trick:** Arrays echo the `$#` / `$@` idea: **`[@]` = all, `${#arr[@]}` = count.** Index starts at 0. *(Lower frequency — know the basics and move on.)*

4.9 Exit codes — the Unix success/failure convention ★★★★☆

Every command returns an **exit code**: `0` **means success, anything non-zero means failure.** Check the last command's code with `$?`. This underpins error handling in scripts and comes up in interviews.

```
systemctl restart nginx
if [ $? -eq 0 ]; then
    echo "Restart succeeded"
else
    echo "Restart FAILED"
fi

# Chaining with && (and) and || (or):
mkdir /backup && echo "created"      # run 2nd only if 1st succeeded
ping -c1 google.com || echo "no network"   # run 2nd only if 1st failed
```

> 💬 **Interviewers usually ask:** "How do you know if the last command succeeded?" ✅ **Answer:** "Check `$?` — it holds the exit code of the last command, where 0 means success and non-zero means failure. I can branch on it with an `if`, or chain commands: `cmd1 && cmd2` runs the second only if the first succeeds, and `cmd1 || cmd2` runs the second only if the first fails."

> 🧠 **Memory trick: 0 = OK.** `$?` = "what was the result?" `&&` = and-then, `||` = or-else.

4.10 Cron jobs — scheduling scripts ★★★★☆

Cron runs scripts automatically on a schedule — nightly backups, hourly log cleanup, health checks. You edit your cron table with `crontab -e`.

The five time fields:

```
┌──── minute (0-59)
│ ┌──── hour (0-23)
│ │ ┌──── day of month (1-31)
│ │ │ ┌──── month (1-12)
│ │ │ │ ┌──── day of week (0-6, Sun=0)
│ │ │ │ │
* * * * *   command_to_run
```

Real examples:

```
0 2 * * *      /home/scripts/backup.sh      # every day at 2:00 AM
*/15 * * * *   /home/scripts/health.sh       # every 15 minutes
0 0 * * 0      /home/scripts/weekly.sh       # midnight every Sunday
30 3 1 * *     /home/scripts/monthly.sh      # 3:30 AM on the 1st
```

> 🧠 **Memory trick: "Minute, Hour, Day-of-month, Month, Day-of-week"** — remember the order **M H Dom M Dow**. A `*` means "every." `*/15` means "every 15." Use crontab.guru to check schedules.

> 💬 **Interviewers usually ask:** "How would you schedule a backup script to run every night?" ✅ **Answer:** "A cron job. I'd run `crontab -e` and add `0 2 * * * /path/backup.sh` to run it at 2 AM daily. The five fields are minute, hour, day of month, month, and day of week — so `0 2 * * *` is minute 0 of hour 2, every day. I'd also redirect output to a log so I can confirm it ran and debug failures."

4.11 Ten practical scripts

These are the kind of scripts you'd actually write on the job — and great to reference if asked "write a script that…" in an interview.

1. Check if a website is up

```
#!/bin/bash
URL="https://google.com"
if curl -s --head "$URL" | grep "200 OK" > /dev/null; then
    echo "$URL is UP"
else
    echo "$URL is DOWN"
fi
```

2. Alert when disk usage is high

```
#!/bin/bash
THRESHOLD=80
usage=$(df / | tail -1 | awk '{print $5}' | tr -d '%')
if [ "$usage" -gt "$THRESHOLD" ]; then
    echo "WARNING: disk is ${usage}% full!"
fi
```

3. Back up a directory with a timestamp

```
#!/bin/bash
SRC="/home/data"
DEST="/backup"
timestamp=$(date +%Y%m%d_%H%M%S)
tar -czf "$DEST/backup_$timestamp.tar.gz" "$SRC"
echo "Backup created: backup_$timestamp.tar.gz"
```

4. Check if a service is running, restart if not

```bash
#!/bin/bash
SERVICE="nginx"
if ! systemctl is-active --quiet "$SERVICE"; then
    echo "$SERVICE is down — restarting"
    systemctl restart "$SERVICE"
fi
```

5. Delete log files older than 7 days

```bash
#!/bin/bash
find /var/log/myapp -name "*.log" -mtime +7 -delete
echo "Old logs cleaned up"
```

6. Count error lines in today's log

```bash
#!/bin/bash
LOG="/var/log/app.log"
count=$(grep -i "error" "$LOG" | wc -l)
echo "Found $count errors in $LOG"
```

7. Ping a list of servers

```bash
#!/bin/bash
servers=("google.com" "github.com" "example.com")
for s in "${servers[@]}"; do
    if ping -c1 -W1 "$s" > /dev/null 2>&1; then
        echo "$s is reachable"
    else
        echo "$s is UNREACHABLE"
    fi
done
```

8. Rename all .txt files to .bak

```bash
#!/bin/bash
for file in *.txt; do
    mv "$file" "${file%.txt}.bak"   # %.txt strips the extension
done
echo "Renamed all .txt to .bak"
```

9. Show top 5 memory-hungry processes

```bash
#!/bin/bash
echo "Top 5 processes by memory:"
ps aux --sort=-%mem | head -6
```

10. Simple menu (case statement)

```bash
#!/bin/bash
echo "1) Disk  2) Memory  3) Uptime"
read -p "Choose: " choice
case $choice in
    1) df -h ;;
    2) free -h ;;
    3) uptime ;;
    *) echo "Invalid choice" ;;
esac
```

> 🏭 **In real production:** Scripts 2, 4, and 5 (disk alert, service watchdog, log cleanup) are the kind of thing that runs on a cron every few minutes on real servers. Being able to sketch one of these on a whiteboard is a strong signal you can do the job.

Chapter 4 — Key Takeaways

- Start with `#!/bin/bash` (shebang); `chmod +x` then `./script.sh` to run.
- **Variables:** `name="x"` (no spaces!), read with `$name`, capture commands with `$( ... )`. *(★★★★☆)*
- **Arguments:** `$1 $2` positional, `$#` count, `$@` all, `$0` script name. Always validate `$#`. *(★★★★☆)*
- **Conditions:** `if [ ... ]; then` — **spaces inside brackets required**. Numbers: `-eq -ne -gt -lt`. Files: `-f -d -e`. *(★★★★★)*
- **Loops:** `for x in list; do ... done`, `while [ cond ]; do ... done`. Arithmetic in `$(( ))`.
- **Exit codes:** 0 = success; check `$?`; chain with `&&` / `||`. *(★★★★☆)*
- **Cron:** `min hour day-of-month month day-of-week command`. `0 2 * * *` = 2 AM daily. *(★★★★☆)*
- You don't need to be fancy — you need to automate a real task: health check, disk alert, log cleanup, backup.

> **Next:** Chapter 5 — the text-processing power tools: grep, awk, sed, and tr. This is where log analysis gets serious.

Chapter 5 — grep, awk, sed, tr

> **Why this chapter matters:** Logs are text, and text is where on-call engineers spend their lives. "Here's a log file — find all the failed logins," or "extract every IP that hit a 500 error," is a *classic* live interview task. These four tools — grep, awk, sed, tr — are the Swiss Army knife for slicing text. You don't need mastery; you need to know **which tool for which job** and a handful of patterns.

The one-line mental model — memorize this, it answers half the questions:

Tool	Job in one word	Use it to…
grep	**Find**	Find lines that match a pattern
awk	**Columns**	Extract & compute on columns/fields
sed	**Edit**	Find-and-replace / edit text in a stream
tr	**Translate**	Replace or delete individual characters

> 🧠 **Memory trick: grep** grabs lines, **awk** works columns, **sed** substitutes, **tr** translates characters. *Grab, Columns, Substitute, Translate.*

We'll use this sample log throughout:

```
192.168.1.10 - GET /home 200
192.168.1.11 - POST /login 401
192.168.1.10 - GET /data 500
192.168.1.12 - GET /home 200
192.168.1.11 - POST /login 401
```

5.1 grep — find matching lines ★★★★★

The most-used of the four. Interviewers assume you know it.

Basic idea: `grep "pattern" file` prints every line containing the pattern.

The flags that matter (know these cold):

Flag	Does	Example
`-i`	case-**i**nsensitive	`grep -i error log` (matches Error, ERROR)
`-v`	in**v**ert — lines NOT matching	`grep -v 200 log` (non-200s)
`-r`	**r**ecursive through directories	`grep -r "TODO" .`
`-n`	show line **n**umbers	`grep -n error log`
`-c`	**c**ount matching lines	`grep -c 401 log`
`-w`	match whole **w**ord only	`grep -w "cat" file` (not "category")
`-E`	**E**xtended regex (or use `egrep`)	`grep -E "401|500" log`
`-o`	print **o**nly the matched part	`grep -oE "[0-9]+$" log`
`-A/-B/-C`	lines **A**fter/**B**efore/**C**ontext	`grep -A3 error log` (3 lines after)

Real examples on our log:

```
grep 500 access.log              # → the one 500 error line
grep -c 401 access.log           # → 2 (count of 401s)
grep -v 200 access.log           # → all non-200 lines (the errors)
grep -E "401|500" access.log     # → all 401 AND 500 lines
grep -A2 -B2 "500" access.log    # → the 500 line plus 2 lines around it
```

🏭 **In real production:** The single most common on-call move is `grep -i error /var/log/app.log`. Add `| tail -20` for the most recent, or `-A5` to see the stack trace *after* each error. This alone solves a huge share of "what broke?" questions.

💬 **Interviewers usually ask:** "Find all lines containing 'error', case-insensitive, and count them." ✅ **Answer:** "`grep -ic error file` — `-i` makes it case-insensitive so it catches Error and ERROR, and `-c` counts the matching lines instead of printing them. If I wanted to see them with line numbers instead, I'd use `grep -in error file`."

🧠 **Memory trick: i**gnore-case, in**v**ert, **r**ecursive, **n**umbers, **c**ount = **i, v, r, n, c**. These five cover almost everything.

5.2 awk — extract and compute on columns ★★★★★

awk shines when data is in columns (which logs almost always are). It automatically splits each line into fields: **`$1` is the first column, `$2` the second, `$0` the whole line.**

The core pattern: `awk '{print $N}'` prints column N.

```
awk '{print $1}' access.log       # → all IP addresses (first column)
awk '{print $1, $4}' access.log   # → IP and status code
awk '{print $NF}' access.log      # → NF = Number of Fields = LAST column
```

On our log, `awk '{print $1}'` gives:

```
192.168.1.10
192.168.1.11
192.168.1.10
192.168.1.12
192.168.1.11
```

Filtering with conditions (this is where awk beats grep):

```
awk '$4 == 500' access.log            # lines where column 4 is 500
awk '$4 >= 400 {print $1}' access.log  # IPs of any 4xx/5xx error
awk '$4 == 401 {print $1}' access.log  # who got 401s
```

Custom delimiter with `-F` (huge for CSVs and `/etc/passwd`):

```
awk -F',' '{print $2}' data.csv        # 2nd column of a CSV
awk -F':' '{print $1}' /etc/passwd     # all usernames (colon-separated)
```

Computing — sum, count, average:

```
awk '{sum += $3} END {print sum}' numbers.txt    # total a column
awk 'END {print NR}' access.log                  # NR = row count = line count
awk '{count[$1]++} END {for (ip in count) print ip, count[ip]}' access.log
#   ↑ count requests per IP — a genuine log-analysis one-liner
```

🏭 **In real production:** "Which IP is hammering us?" → `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head` — extract IPs, count each, sort by count. That pipeline is a rite of passage for support engineers.

💬 **Interviewers usually ask:** "From this log, print only the IP addresses of requests that returned a 500." ✅ **Answer:** "`awk '$4 == 500 {print $1}' access.log`. awk splits each line into fields automatically, so `$4` is the status code and `$1` is the IP. The condition `$4 == 500` filters to just those lines, and `{print $1}` prints the IP. That's the kind of thing awk does far more cleanly than grep, because grep matches whole lines while awk understands columns."

🧠 **Memory trick:** awk = **A**ll about columns. `$1 $2 $3` = columns, `$NF` = last field, `$0` = whole line, `-F` = field separator, `NR` = number of rows.

5.3 sed — stream editor (find & replace) ★★★★☆

sed edits text as it streams past — most famously for find-and-replace, without opening an editor.

The killer feature — substitution: `s/old/new/`

```
sed 's/error/ERROR/' log.txt        # replace FIRST "error" on each line
sed 's/error/ERROR/g' log.txt       # g = GLOBAL — replace ALL on each line
sed 's/error/ERROR/gi' log.txt      # g + i = all, case-insensitive
```

⚠️ **Common mistake:** Forgetting the `g` flag. Without it, `sed` only replaces the *first* match on each line. `g` = global = all matches on the line.

In-place editing (actually change the file):

```
sed -i 's/old/new/g' config.txt         # -i edits the file directly
sed -i.bak 's/old/new/g' config.txt      # -i.bak makes a backup first (safer!)
```

Other handy sed moves:

```
sed -n '5,10p' file.txt          # print only lines 5–10 (-n + p)
sed '2d' file.txt                # delete line 2
sed '/debug/d' file.txt          # delete every line containing "debug"
sed 's/^/> /' file.txt           # add "> " to the start of each line (^ = start)
```

🏭 **In real production:** Changing a config value across servers — `sed -i 's/port=8080/port=9090/' app.conf` — is a everyday sed use. The `-i.bak` habit (make a backup) has saved many engineers from a bad regex.

💬 **Interviewers usually ask:** "How would you replace every occurrence of 'foo' with 'bar' in a file?"
✅ **Answer:** "`sed -i 's/foo/bar/g' file`. The `s` command substitutes, the `g` flag makes it replace *every* occurrence on each line rather than just the first, and `-i` edits the file in place. I'd often use `-i.bak` instead so it keeps a backup — regex mistakes on a live config file are easy to make and hard to undo."

🧠 **Memory trick:** sed = **s**ubstitute: **`s/old/new/g`**. **`g` = global (all), `i` = ignore case, `-i` = in-place (edit the actual file).** Don't forget the `g`!

5.4 tr — translate/delete characters ★★★☆☆

tr works on individual characters, not patterns or words. It reads from standard input (use it in a pipe).

```
echo "hello" | tr 'a-z' 'A-Z'          # → HELLO (lowercase to uppercase)
echo "hello world" | tr ' ' '_'         # → hello_world (spaces to underscores)
echo "hello" | tr -d 'l'                # → heo (-d = delete chars)
echo "a,b,c" | tr ',' '\n'              # → each on its own line (comma → newline)
cat file | tr -s ' '                    # -s = squeeze repeated spaces into one
echo "Phone: 555-1234" | tr -cd '0-9'   # -c -d = keep ONLY digits → 5551234
```

The flags: `-d` delete, `-s` squeeze (collapse repeats), `-c` complement (invert the set — "everything except").

> 🏭 **In real production:** Cleaning messy data — `tr -d '\r'` to strip Windows carriage returns from a file, or `tr -s ' '` to squeeze irregular spacing before feeding to awk. Small but constantly useful.

> 💬 **Interviewers usually ask:** "How do you convert a file to uppercase / remove all digits?" ✅ **Answer:** "`tr 'a-z' 'A-Z'` to uppercase, piping the file in. To remove digits, `tr -d '0-9'`. tr operates on individual characters rather than words or patterns, so it's the right tool for character-level transforms — case changes, deleting or squeezing specific characters, swapping delimiters."

> 🧠 **Memory trick: tr = translate characters.** Two sets = swap; `-d` = delete; `-s` = squeeze; `-c` = complement (everything but).

5.5 Putting it together — the log-analysis pipelines ★★★★★

This is what separates a strong candidate. Real answers *combine* these tools with pipes, `sort`, and `uniq`. Two more tools to know:

- `sort` — order lines. `-n` numeric, `-r` reverse, `-rn` = biggest first.
- `uniq -c` — collapse duplicate *adjacent* lines and count them (**must `sort` first**).

The single most useful log pipeline (learn it by heart):

```
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head
```

Read it left to right: *extract IPs → sort them → count duplicates → sort by count descending → show the top few.* Result: your top traffic sources.

More real-world one-liners:

```
# Count how many of each HTTP status code:
awk '{print $4}' access.log | sort | uniq -c | sort -rn

# Top 10 URLs requested:
awk '{print $3}' access.log | sort | uniq -c | sort -rn | head

# All unique IPs that caused a 500 error:
awk '$4 == 500 {print $1}' access.log | sort | uniq

# Find failed logins, extract the IP, count offenders:
grep "401" access.log | awk '{print $1}' | sort | uniq -c | sort -rn
```

> 💬 **Interviewers usually ask:** "Given a web server log, find the top 5 IP addresses making requests."
> ✅ **Model answer:** "`awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5`. I use awk to pull the IP from the first column, sort so identical IPs are adjacent, `uniq -c` to collapse and count them — that's why the sort comes first, uniq only compares adjacent lines — then `sort -rn` to order by count descending, and `head -5` for the top five. It's a pipeline where each tool does one job: extract, group, count, rank."

> 🧠 **Memory trick — the golden pipeline: extract → sort → uniq -c → sort -rn → head.** Whenever you hear "top N" or "count each," reach for this shape.

5.6 grep vs awk vs sed — the decision guide

When you're not sure which to reach for:

```
Need to FIND lines matching a pattern?          →  grep
Need to EXTRACT a column / filter by a field?   →  awk
Need to REPLACE / edit text?                     →  sed
Need to change/delete individual characters?     →  tr
Need to COUNT occurrences of things?             →  awk … | sort | uniq -c
```

> ⚠️ **Common mistake:** Using the wrong tool and fighting it. grep can't easily pick "column 4"; awk isn't for find-and-replace across a file; sed isn't for math on columns. Match the tool to the job and the one-liner writes itself.

Chapter 5 — Key Takeaways

- **grep = find lines.** Flags: `-i` ignore case, `-v` invert, `-r` recursive, `-n` numbers, `-c` count. *(★★★★★)*
- **awk = columns.** `$1 $2 $NF`, filter with `$4 == 500`, `-F` sets the delimiter, `NR` counts rows. Best for extracting fields. *(★★★★★)*
- **sed = substitute.** `sed 's/old/new/g'` (don't forget `g`!), `-i` edits in place, `-i.bak` keeps a backup. *(★★★★☆)*

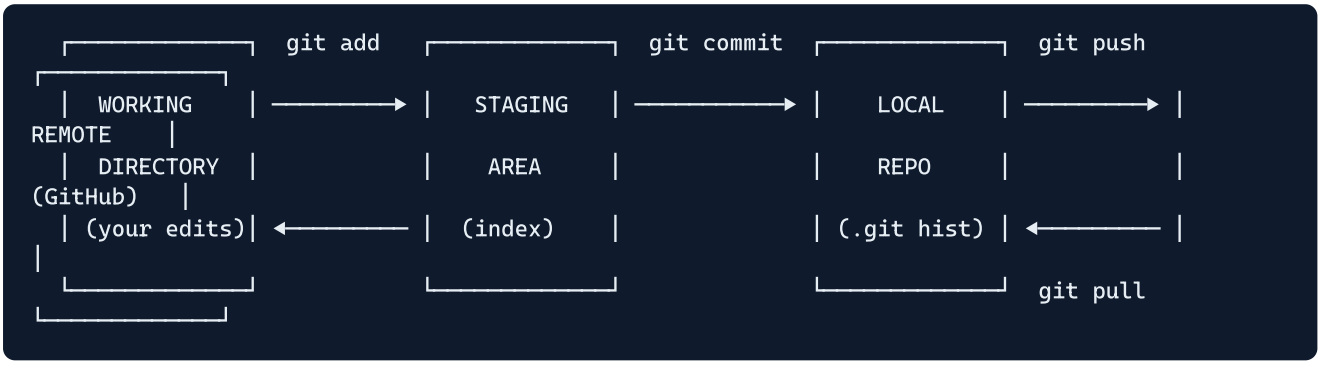
- **tr = translate characters.** Swap sets, `-d` delete, `-s` squeeze, `-c` complement. *(★★★☆☆)*
- **The golden pipeline:** `... | sort | uniq -c | sort -rn | head` for any "top N / count each" question. *(★★★★★)*
- Match the tool to the job: **find→grep, columns→awk, replace→sed, characters→tr.**

> **Next:** Chapter 6 — Git. Short but high-yield: the commands you use daily plus the merge-vs-rebase and conflict questions interviewers can't resist.

Chapter 6 — Git

> **Why this chapter matters:** Git is a baseline expectation. Nobody's testing whether you can implement Git internals — they want to know you can work on a team without breaking the shared codebase. The high-yield questions are always the same handful: the everyday workflow, **merge vs rebase**, and **how you resolve a conflict**. Nail those three and Git is handled.

First, the mental model — the four areas Git moves code between:



> 🧠 **Memory trick: Edit → `add` (stage) → `commit` (save locally) → `push` (share).** The staging area in the middle is Git's signature idea: you choose *exactly* what goes into each commit.

6.1 The everyday commands ★★★★★

Command	What it does
`git clone <url>`	Copy a remote repo to your machine (once, at the start)
`git status`	What's changed / staged? (check this constantly)
`git add <file>`	Stage a file for the next commit (`git add .` = all)
`git commit -m "msg"`	Save staged changes to local history
`git push`	Upload local commits to the remote
`git pull`	Download + merge remote changes into your branch
`git fetch`	Download remote changes **but don't merge** yet
`git log --oneline`	View commit history compactly
`git diff`	See exact line changes not yet staged

A typical day:

```
git pull                          # get latest before you start
# ... edit files ...
git status                        # what did I change?
git add .                         # stage everything
git commit -m "Fix login bug"     # save it locally
git push                          # share it
```

> 💬 **Interviewers usually ask:** "Walk me through your normal Git workflow." ✅ **Model answer:** "I pull the latest changes first so I'm working on current code. I make my edits, then `git status` to see what changed and `git diff` to review it. I stage with `git add`, commit with a clear message describing *why*, not just what, and push to the remote. If I'm working on a feature I do it on a branch and open a pull request so it's reviewed before merging into main."

> 🧠 **Memory trick: Pull first, push last.** Between them: status → add → commit.

6.2 fetch vs pull ★★★★☆

A classic "do you actually understand Git" question.

- `git fetch` downloads the latest commits from the remote **but does not change your working files.** It just updates your knowledge of what's on the remote. Safe — you can inspect before integrating.
- `git pull` = `git fetch` + `git merge`. It downloads *and* immediately merges into your current branch.

> ✅ **Model answer:** "`fetch` downloads remote changes but leaves my working branch untouched — I can review what came in before integrating. `pull` is `fetch` plus `merge`: it downloads and immediately merges into my current branch. So `pull` is the quick everyday command, but `fetch` is safer when I want to look before I leap."

> 🧠 **Memory trick: fetch = look, pull = look + apply.** Pull is fetch that doesn't wait.

6.3 Branching ★★★★☆

Branches let you work on a feature in isolation without touching the main codebase, then merge it back when it's ready.

```
git branch                      # list branches (* = current)
git branch feature-login        # create a branch
git checkout feature-login      # switch to it
git checkout -b feature-login   # create AND switch (the common shortcut)
git switch feature-login        # modern equivalent of checkout
git merge feature-login         # merge that branch into your current one
git branch -d feature-login     # delete a merged branch
```

🌍 **Analogy.** A branch is a parallel draft of a document. You experiment freely on your copy; when it's good, you merge your changes back into the master copy. Everyone else's work on `main` is unaffected while you tinker.

🏭 **In real production:** Teams almost never commit directly to `main`. You branch (`feature/…` or `fix/…`), push the branch, open a **pull request**, get it reviewed, and merge. Mentioning this workflow signals you've worked on a team.

🧠 **Memory trick: `checkout -b` = create + jump.** Branch = safe sandbox off of `main`.

6.4 Merge vs Rebase ★★★★★

The most-asked advanced Git question. Understand the difference in shape.

Both integrate changes from one branch into another. The difference is *history*.

Merge — combines the two branches with a **merge commit**, preserving the exact history of both. Non-destructive but the history looks branchy.

```
main:    A──B──C────────M   ← M is a merge commit
                  \     /
feature:           D──E
```

Rebase — **replays** your branch's commits on top of the latest main, creating a **straight, linear history** as if you'd started from the newest code. Cleaner, but it *rewrites* commit history.

```
Before:  main: A─B─C     feature: (from B) D─E
After rebase:  A─B─C─D'─E'   ← D,E replayed on top of C, linear
```

The comparison:

	Merge	Rebase
History	Preserved, shows branches + a merge commit	Rewritten, linear & clean
Safety	Safe — never changes existing commits	Rewrites history (new commit IDs)
Best for	Shared/public branches	Cleaning up *your own* local branch before merging

> ⚠️ **THE golden rule (interviewers love this): Never rebase a branch that others are working on / that's already pushed and shared.** Rebasing rewrites commit history, so it breaks everyone else who has the old commits. Rebase only your own local, un-shared work.

> 💬 **Interviewers usually ask:** "Difference between merge and rebase, and when would you use each?" ✅ **Model answer:** "Both integrate one branch's changes into another. Merge creates a merge commit and preserves the full branching history — it's safe and non-destructive. Rebase replays your commits on top of the target branch, giving a clean, linear history, but it rewrites commit history in the process. I use rebase to tidy up my own local feature branch before merging, so the history reads cleanly. I use merge for integrating into shared branches. The rule I never break: don't rebase commits that have already been pushed and shared, because rewriting shared history breaks everyone else's copy."

> 🧠 **Memory trick: Merge = keep the story (branchy + merge commit). Rebase = rewrite the story (linear).** Rebase = "re-base" your commits onto a new base. Never rewrite shared history.

6.5 Merge conflicts ★★★★★

Guaranteed question for any dev-adjacent role.

What it is. A conflict happens when two branches change the **same lines** of the same file differently, so Git can't automatically decide which version wins. It stops and asks *you*.

What it looks like in the file:

```
<<<<<<< HEAD
color = "blue"          ← your version (current branch)
=======
color = "green"         ← their version (incoming branch)
>>>>>>> feature-branch
```

How you resolve it (the steps to recite):

```
# 1. Git tells you which files conflict:
git status
# 2. Open each file, find the <<<< ==== >>>> markers,
#    edit to the correct final version, and DELETE the markers.
# 3. Stage the resolved files:
git add conflicted_file.txt
# 4. Complete the merge:
git commit          # (or: git rebase --continue if mid-rebase)
```

> 💬 **Interviewers usually ask:** "How do you resolve a merge conflict?" ✅ **Model answer:** "A conflict happens when two branches edited the same lines and Git can't auto-merge. Git marks the conflicting sections in the file with `<<<<<<<`, `=======`, and `>>>>>>>` markers showing my version and the incoming version. I open each conflicted file, decide what the correct final code should be — sometimes mine, sometimes theirs, sometimes a combination — remove the markers, then `git add` the resolved files and `git commit` to finish the merge. `git status` lists exactly which files still need resolving. The key is that I read both sides and think, rather than blindly picking one."

> ⚠️ **Common mistake:** Leaving the `<<<<<<<` markers in the file (they'll break your code), or blindly choosing one side without understanding both changes.

> 🧠 **Memory trick:** Conflict markers = **`<<<< mine ==== theirs >>>>`**. Edit to the truth, delete the markers, `add`, `commit`.

6.6 Undoing things: reset, revert, stash ★★★★☆

Interviewers test whether you can fix mistakes safely.

`git stash` — shelve changes temporarily

You're mid-edit and need to switch branches urgently. Stash saves your uncommitted work and gives you a clean slate.

```
git stash           # save & hide uncommitted changes
git stash pop       # bring them back
git stash list      # see stashed changes
```

> 🌍 Like sweeping your messy desk into a drawer to deal with something urgent, then tipping it back out later.

`git reset` vs `git revert` — the key distinction

- **`git revert <commit>`** creates a **new commit that undoes** a previous one. History is preserved. **Safe for shared/public branches.**

- `git reset` moves the branch pointer **back**, effectively erasing commits from the branch. **Rewrites history — dangerous on shared branches.**

```
git revert abc123          # safe undo: adds a new "undo" commit
git reset --soft HEAD~1    # undo last commit, KEEP changes staged
git reset --hard HEAD~1    # undo last commit AND discard changes (destructive!)
```

⚠️ **Common mistake:** Using `git reset --hard` on shared history. **Rule of thumb: `revert` for anything already pushed/shared; `reset` only for local, un-pushed commits.** And `--hard` throws work away permanently — treat it with respect.

💬 **Interviewers usually ask:** "Difference between git reset and git revert?" ✅ **Model answer:** "Both undo changes, but differently. `revert` creates a *new* commit that reverses an earlier one, so the history stays intact — that makes it safe on shared branches everyone else has pulled. `reset` moves the branch pointer backward and effectively removes commits, which rewrites history and is risky if those commits were pushed. So on a shared branch I always use `revert`; `reset` I reserve for cleaning up my own local commits before I've pushed. And `reset --hard` also discards the working changes, so I'm careful with it."

🧠 **Memory trick: re`v`ert = safe, adds a `v`ersion (new commit). re`s`et = `s`crubs history (dangerous on shared).** `stash` = drawer.

Chapter 6 — Key Takeaways

- **Flow:** edit → `add` (stage) → `commit` (local) → `push` (share). **Pull first, push last.** *(★★★★★)*
- **`fetch` = download only; `pull` = fetch + merge.** *(★★★★☆)*
- **Branches** isolate work; teams use feature branches + pull requests, not commits to `main`. `checkout -b` = create + switch.
- **Merge vs rebase:** merge keeps branchy history + a merge commit; rebase makes linear history but rewrites it. **Never rebase shared/pushed history.** *(★★★★★)*
- **Conflicts:** `<<<< mine ==== theirs >>>>` — edit to the correct version, delete markers, `add`, `commit`. *(★★★★★)*
- **Undo:** `stash` shelves work; `revert` safely undoes with a new commit (use on shared branches); `reset` rewrites history (local only); `reset --hard` is destructive. *(★★★★☆)*

Next: Chapter 7 — Cloud Computing. The final knowledge chapter, and increasingly the one that lands the offer for cloud-support and DevOps roles.

Chapter 7 — Cloud Computing

> **Why this chapter matters:** "Cloud Support Engineer" and "DevOps Intern" have *cloud* in the name, and even backend/on-call roles now assume you know the basics. The good news: at the junior level, interviewers want *conceptual clarity*, not deep AWS certification knowledge. If you can explain IaaS/PaaS/SaaS, the core AWS services, and what Docker and Kubernetes are *for*, you'll clear the bar. This chapter stays deliberately at interview level.

7.1 What is the Cloud? ★★★★☆

Definition. "The cloud" means renting computing resources — servers, storage, databases, networking — from a provider (like Amazon, Microsoft, or Google) over the internet, instead of buying and running your own hardware. You pay only for what you use.

Why it exists — the problem it solves. Traditionally, to launch an app you'd buy physical servers, rack them in a data center, and hope you guessed the capacity right. Too few and you crash under load; too many and you've wasted money. The cloud replaces that with resources you spin up in minutes and scale up or down on demand.

The key benefits (name these): - **Pay-as-you-go** — no huge upfront hardware cost; rent by the hour/second. - **Elasticity/scalability** — add capacity during a traffic spike, remove it after. - **No hardware management** — the provider handles the physical machines, power, cooling. - **Global reach** — deploy near your users in minutes. - **Reliability** — spread across multiple data centers ("availability zones").

> 🌍 **Analogy.** The cloud is **electricity from the grid vs. owning a generator.** You don't build a power plant to run your house — you plug in and pay for what you use. Need more power? The grid handles it. The cloud is compute from the grid.

> 💬 **Interviewers usually ask:** "What is cloud computing and why do companies use it?" ✅ **Model answer:** "It's renting computing resources — servers, storage, databases — over the internet from a provider, instead of owning hardware. Companies use it because it turns a big upfront capital cost into pay-as-you-go, and it's elastic: you scale up during busy periods and back down after, so you're not paying for idle machines. You also offload all the physical maintenance — power, cooling, hardware failures — to the provider, and you can deploy globally in minutes. The trade-off is ongoing cost and depending on the provider, but for most companies the flexibility wins."

> 🧠 **Memory trick:** Cloud = **someone else's computers, rented by the hour, scaled on demand.** Electricity, not a generator.

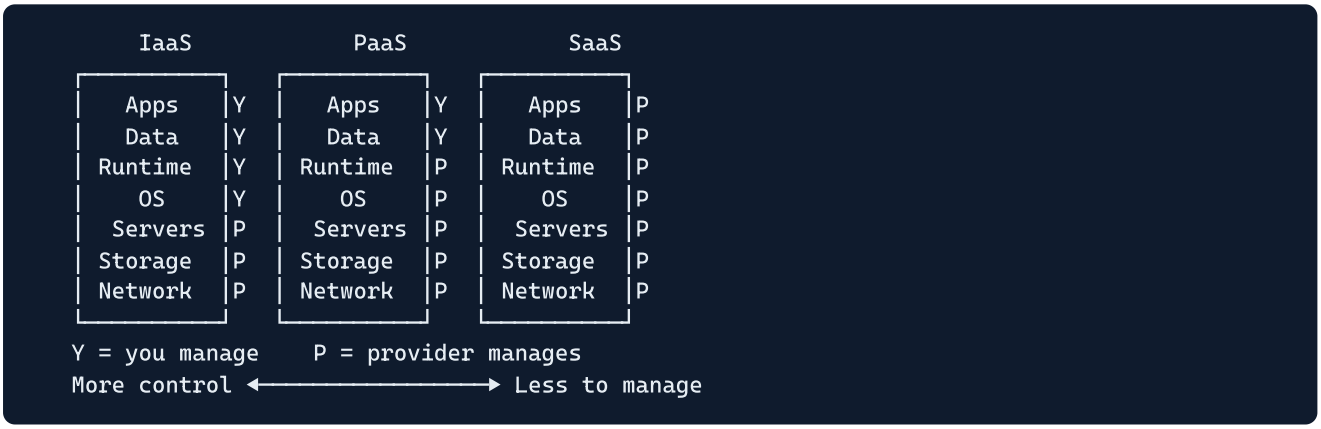
7.2 IaaS vs PaaS vs SaaS ★★★★★

One of the most-asked cloud questions. Know the layers and an example of each.

These are the three service models — they differ in **how much the provider manages vs. how much you manage.**

Model	You manage	Provider manages	Example
IaaS (Infrastructure)	OS, apps, data	Hardware, virtualization, network	AWS **EC2**, virtual machines
PaaS (Platform)	Just your app + data	OS, runtime, servers, scaling	Heroku, Google App Engine, AWS Elastic Beanstalk
SaaS (Software)	Nothing — just use it	Everything	Gmail, Dropbox, Salesforce

The layers, visualized:



🌎 **Analogy — pizza!** The classic one interviewers smile at: - **IaaS** = buying ingredients and making pizza in your kitchen (they give you the kitchen; you cook). - **PaaS** = a take-and-bake pizza — it's prepped, you just bake it (they handle the platform; you bring the app). - **SaaS** = ordering a pizza delivered — you just eat (everything done for you).

💬 **Interviewers usually ask:** "Explain IaaS, PaaS, and SaaS with examples." ✅ **Model answer:** "They're layers of how much the cloud provider manages. With IaaS — Infrastructure as a Service, like AWS EC2 — you rent raw virtual machines and manage the OS, runtime, and app yourself; you get the most control. With PaaS — Platform as a Service, like Heroku — the provider manages the OS, runtime, and scaling, and you just deploy your code. With SaaS — Software as a Service, like Gmail or Salesforce — the provider runs everything and you just use the finished application. The trade-off is control versus convenience: IaaS gives the most control and the most responsibility, SaaS the least of both."

> 🧠 **Memory trick: I-P-S = more managed as you go down.** IaaS = you cook, PaaS = you bake, SaaS = you eat. Or: **I**nfrastructure → **P**latform → **S**oftware, each one hands more to the provider.

7.3 The big three providers ★★★☆☆

Provider	Owner	Nickname
AWS (Amazon Web Services)	Amazon	The market leader, biggest service catalog
Azure	Microsoft	Strong in enterprise / Windows shops
GCP (Google Cloud Platform)	Google	Strong in data / Kubernetes (Google invented K8s)

> ✅ **One-liner:** "AWS is the market leader with the broadest set of services; Azure is popular in enterprises already using Microsoft; GCP is known for data analytics and Kubernetes. They offer largely equivalent building blocks under different names." *(You'll usually interview against one — know its core services below.)*

7.4 Core AWS services you must know ★★★★☆

Even if the role isn't AWS-specific, these four concepts (compute, storage, identity, networking) appear everywhere under different names.

EC2 — Elastic Compute Cloud (virtual servers) ★★★★☆

A virtual machine in the cloud. You pick an OS and size, and get a server you fully control. This is IaaS. You'd run your app, database, or anything else on it. > 🧠 **EC2 = a rented computer in the cloud.** "Compute" = a machine that runs things.

S3 — Simple Storage Service (object storage) ★★★★☆

Storage for files ("objects") — images, backups, videos, logs — organized into "buckets." Virtually unlimited, cheap, accessed over HTTP. **It is not a filesystem or a hard drive for your OS** — it's for storing and retrieving whole files. > 🧠 **S3 = infinite cloud folder for files.** Think Dropbox for applications. > ⚠️ **Common mistake:** Confusing S3 (object storage for files) with block storage (a virtual hard disk attached to EC2, called EBS on AWS). S3 = files by name; EBS = a disk for your server.

IAM — Identity and Access Management (permissions) ★★★★☆

Controls *who* can do *what*. IAM manages users, groups, roles, and policies that grant or deny access to resources. It's the security backbone of any cloud account. > 🧠 **IAM = the cloud's bouncer + rulebook.** Principle to name: **least privilege** — give each user/service only the permissions it actually needs, nothing more.

VPC — Virtual Private Cloud (your private network) ★★★☆☆

Your own isolated network within the cloud. You define IP ranges, subnets (public/private), and firewall rules (security groups) so your resources are networked and protected the way you want. > 🧠 **VPC = your private, walled-off network in AWS.** Public subnet = reachable from the internet; private subnet = internal only (e.g., your database).

> 💬 **Interviewers usually ask:** "What's the difference between EC2 and S3?" or "What is IAM for?" ✅ **Model answer:** "EC2 is compute — a virtual server you run your application on and control at the OS level. S3 is object storage — you put files like images, backups, or logs into buckets and retrieve them over HTTP; it's not a disk for the OS, it's for whole files. IAM is identity and access management: it defines who can do what across the account, using users, roles, and policies, and the guiding principle is least privilege — grant only the access that's actually needed. And a VPC is your isolated private network in the cloud where those resources live."

7.5 Containers & Docker ★★★★★

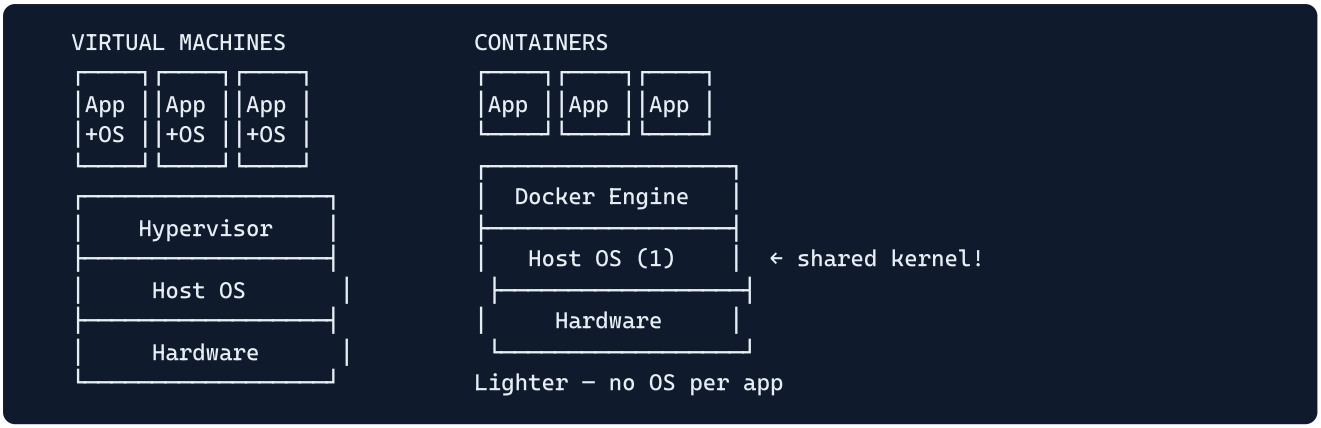
High-frequency for DevOps and backend roles. Understand the problem Docker solves.

The problem. "It works on my machine" — an app runs fine on your laptop but breaks on the server because of different OS versions, libraries, or configs. Setting up dependencies consistently across environments is painful.

Definition. A **container** packages an application *together with everything it needs to run* — code, runtime, libraries, settings — into one portable unit that runs identically anywhere. **Docker** is the most popular tool for building and running containers.

Containers vs Virtual Machines — the key comparison interviewers want:

	Virtual Machine	Container
Contains	Full guest OS + app	Just app + its dependencies
Size	Gigabytes	Megabytes
Startup	Minutes	Seconds
Overhead	Heavy (each has own OS)	Light (shares host OS kernel)
Isolation	Stronger	Lighter but sufficient



Docker vocabulary to know: - **Image** — the blueprint/template (built from a `Dockerfile`). - **Container** — a running instance of an image. - **Dockerfile** — the recipe describing how to build the image. > 🧠 Image is to container as class is to object (if you know OOP): the image is the template, the container is the live instance.

A few commands:

```
docker build -t myapp .        # build an image from a Dockerfile
docker run myapp               # run a container from the image
docker ps                      # list running containers
docker images                  # list images
```

> 🌍 **Analogy.** A container is a **shipping container**. Before standardized containers, loading a ship meant custom-packing every odd-shaped item. Standardized containers made cargo portable across ship, train, and truck without repacking. Docker does the same for software: package it once, run it anywhere.

> 💬 **Interviewers usually ask:** "What is Docker / a container, and how is it different from a VM?" ✅ **Model answer:** "A container packages an application with all its dependencies — libraries, runtime, config — so it runs the same way everywhere, which solves the classic 'works on my machine' problem. Docker is the standard tool for it. The big difference from a virtual machine is that a VM includes an entire guest operating system, so it's heavy — gigabytes and minutes to boot. Containers share the host's OS kernel and only bundle the app and its dependencies, so they're megabytes and start in seconds. You can pack far more containers onto a machine than VMs. VMs give stronger isolation; containers give speed and density, which is why they're the default for modern deployments."

> 🧠 **Memory trick: Container = app + its stuff, no OS, starts in seconds. VM = app + whole OS, heavy, starts in minutes.** Shipping container for software.

7.6 Kubernetes ★★★★☆

Definition. Kubernetes (K8s) is a system that automatically manages ("orchestrates") lots of containers across many machines — deploying them, scaling them up and down, restarting failed ones, and load-balancing traffic between them.

Why it exists. Running one container with Docker is easy. Running hundreds across dozens of servers — keeping them healthy, scaling on load, replacing crashed ones at 3 a.m. without a human — is not. Kubernetes automates that.

What it does for you (name a few): - **Self-healing** — restarts or replaces containers that crash. - **Auto-scaling** — adds more containers under load, removes them when quiet. - **Load balancing** — spreads traffic across container instances. - **Rolling updates** — deploys new versions with zero downtime, and can roll back.

A couple of terms: a **Pod** is the smallest unit (one or a few containers together); a **Cluster** is the set of machines K8s manages.

> 🌍 **Analogy.** If a container is a shipping container, **Kubernetes is the entire automated port** — the cranes, scheduling, and logistics that decide which container goes where, replace damaged ones, and add capacity when ships pile up. Docker packs the box; Kubernetes runs the port.

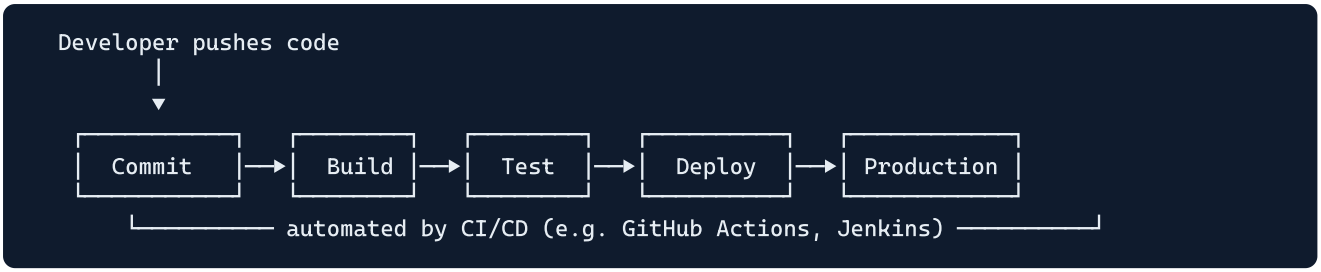
> 💬 **Interviewers usually ask:** "What is Kubernetes and why would you use it?" ✅ **Model answer:** "Kubernetes is a container orchestrator — it manages containers at scale across a cluster of machines. Docker is great for building and running individual containers, but once you have hundreds across many servers, you need automation to deploy them, scale them with demand, restart the ones that crash, and route traffic between them. Kubernetes does all that: it's self-healing, it auto-scales, it load-balances, and it can do rolling updates with zero downtime and roll back if something breaks. So Docker packages the app; Kubernetes runs and manages many of them in production."

> 🧠 **Memory trick: Docker builds & runs one box; Kubernetes manages the whole fleet.** K8s = self-healing, auto-scaling, orchestration.

7.7 CI/CD ★★★★☆

Definition. CI/CD automates the path from writing code to running it in production. - **CI — Continuous Integration:** every time someone pushes code, it's automatically built and **tested**, catching bugs early and keeping the main branch always working. - **CD — Continuous Delivery/Deployment:** the tested code is automatically **deployed** to staging or production, so releases are frequent, small, and low-risk.

The pipeline:



Why it matters. Instead of a scary, manual, once-a-quarter release, you ship small changes many times a day, automatically tested and deployed. Bugs are caught in minutes, not weeks, and rollbacks are easy.

Tools to name-drop: GitHub Actions, GitLab CI, Jenkins, CircleCI.

> 🌎 **Analogy.** CI/CD is a **factory assembly line with quality control at every station.** Each new part (code change) is automatically inspected (tested) and only moves forward if it passes, instead of assembling the whole car and hoping it works at the end.

> 💬 **Interviewers usually ask:** "What is CI/CD and why is it valuable?" ✅ **Model answer:** "CI/CD automates going from code to production. Continuous Integration means every push is automatically built and tested, so bugs are caught immediately and the main branch stays healthy. Continuous Delivery/Deployment means that once tests pass, the code is automatically deployed to staging or production. The value is that you ship small changes frequently and safely instead of doing big risky manual releases — problems surface in minutes and are easy to roll back. Common tools are GitHub Actions, GitLab CI, and Jenkins."

> 🧠 **Memory trick: CI = integrate + test automatically. CD = deliver/deploy automatically.** Push → build → test → deploy, all hands-free.

Chapter 7 — Key Takeaways

- **Cloud** = renting compute over the internet, pay-as-you-go, elastic. Electricity, not a generator. *(★★★★☆)*
- **IaaS/PaaS/SaaS** = increasing amounts managed by the provider. EC2 / Heroku / Gmail. You cook / you bake / you eat. *(★★★★★)*
- **Big three:** AWS (leader), Azure (enterprise), GCP (data/K8s).
- **AWS core:** EC2 = virtual server (compute); S3 = file/object storage (buckets); IAM = who-can-do-what (least privilege); VPC = your private network. *(★★★★☆)*
- **Docker/containers** = app + dependencies, no full OS, starts in seconds; lighter than VMs. Solves "works on my machine." *(★★★★★)*
- **Kubernetes** = orchestrates many containers: self-healing, auto-scaling, load-balancing, rolling updates. Docker packs the box, K8s runs the fleet. *(★★★★☆)*
- **CI/CD** = automate build → test (CI) → deploy (CD). Ship small, safe, often. *(★★★★☆)*

Next: Chapter 8 — Final Revision. Cheat sheets, every diagram, the ports and status-code tables, “things people confuse,” and interview one-liners — your last-hour, night-before reference.

Chapter 8 — Final Revision & Cheat Sheets

> **How to use this chapter:** This is your **night-before and morning-of** reference. Don't read it like prose — scan it, quiz yourself, and drill the tables. If you can reproduce these cheat sheets from memory, you're ready. Everything here is distilled from Chapters 1–7.

8.1 The "confusion killers" — things candidates mix up ★★★★★

Interviewers love these because getting them backwards instantly reveals shaky understanding. Master every row.

Pair	The difference in one line
Process vs Thread	Process = private memory; thread = shared memory (inside a process)
Stack vs Heap	Stack = auto, fast, small, LIFO (overflow); heap = manual/GC, large (leaks)
Zombie vs Orphan	Zombie = child dead, not reaped; orphan = parent dead, child adopted by PID 1
Deadlock vs Starvation	Deadlock = everyone stuck in a cycle forever; starvation = one keeps getting skipped
Mutex vs Semaphore	Mutex = 1 at a time, owned; semaphore = up to N, not owned
Paging vs Segmentation	Paging = fixed-size blocks; segmentation = variable, by logical meaning
TCP vs UDP	TCP = reliable/ordered/slow; UDP = fast/best-effort/no guarantees
401 vs 403	401 = not authenticated (who are you?); 403 = authenticated, not allowed
4xx vs 5xx	4xx = client's fault; 5xx = server's fault
Cookie vs Session	Cookie = client-side; session = server-side (cookie carries the ID)
Session vs JWT	Session = server-stored state; JWT = stateless signed token
GET vs POST	GET = read, idempotent; POST = create, not idempotent
PUT vs POST	PUT = update/replace (idempotent); POST = create (not idempotent)
Forward vs Reverse Proxy	Forward = for clients; reverse = for servers (reveRse = seRveR)
df vs du	df = free space (whole disk); du = usage (specific files/dirs)
kill vs kill -9	kill = SIGTERM (graceful); kill -9 = SIGKILL (forced, last resort)
find vs grep	find = locate files; grep = find text inside files
fetch vs pull	fetch = download only; pull = fetch + merge
merge vs rebase	merge = keeps history + merge commit; rebase = linear, rewrites history
reset vs revert	reset = rewrites history (local); revert = new undo commit (safe/shared)
VM vs Container	VM = full OS, heavy, minutes; container = shares kernel, light, seconds
IaaS vs PaaS vs SaaS	Increasing provider management: you cook / you bake / you eat
Docker vs Kubernetes	Docker = build/run a container; K8s = orchestrate many containers
Symmetric vs Asymmetric (TLS)	Asymmetric = agree the key (once); symmetric = encrypt the data (fast)

8.2 Ports cheat sheet ★★★★☆

Port	Service		Port	Service
22	SSH		**443**	HTTPS
25	SMTP (email out)		**3306**	MySQL
53	DNS		**5432**	PostgreSQL
80	HTTP		**6379**	Redis
110/143	POP3 / IMAP (email in)		**27017**	MongoDB

> **Drill:** 22 SSH · 53 DNS · 80 HTTP · 443 HTTPS · 3306 MySQL · 5432 Postgres. Say them until automatic.

8.3 HTTP status codes cheat sheet ★★★★★

```
1xx Informational   "hold on"
2xx Success         "here you go"     200 OK · 201 Created · 204 No Content
3xx Redirect        "look elsewhere"  301 Moved Perm · 302 Found · 304 Not Modified
4xx CLIENT error    "YOU messed up"   400 Bad Req · 401 Unauth · 403 Forbidden
                                      404 Not Found · 429 Too Many Requests
5xx SERVER error    "I messed up"     500 Internal · 502 Bad Gateway
                                      503 Unavailable · 504 Gateway Timeout
```

The five you'll be asked to distinguish: 200, 301 vs 302, 401 vs 403, 404, 500 vs 502 vs 503 vs 504.

8.4 Linux commands cheat sheet ★★★★★

```
NAVIGATION        pwd  ls -la  cd  cd ..  cd ~
FILES             mkdir  touch  cp -r  mv (move/rename)  rm -r
VIEW              cat  less  head -n  tail -n  tail -f (live!)
SEARCH            find /path -name "*.log"      grep -i "err" file
                  find . -size +100M            grep -rn "text" .
PERMISSIONS       chmod 755 file   chmod +x   chown user:group file
                  (4=read 2=write 1=execute; 755=rwxr-xr-x; 644=rw-r--r--)
PROCESSES         ps aux   ps aux | grep x   top   kill PID   kill -9 PID
DISK / MEM        df -h (which disk full)   du -sh *   free -h
SERVICES          systemctl status|start|stop|restart|enable  SERVICE
LOGS              journalctl -u SERVICE -f     tail -f /var/log/...
PIPES/REDIRECT    cmd1 | cmd2     > overwrite   >> append   2> errors
```

The three on-call reflexes:

```
"Disk full"      → df -h  →  du -sh /var/*  →  find & clear big files
"Service down"   → systemctl status X  →  journalctl -u X -e  →  restart
"Server slow"    → top  →  find the PID  →  investigate/kill  →  check logs
```

8.5 Text processing cheat sheet ★★★★★

```
grep    FIND lines       grep -i (ignore case)  -v (invert)  -r (recursive)
                         -n (line #)  -c (count)  -E "a|b" (regex)
awk     COLUMNS          awk '{print $1}'   $NF (last)   -F',' (delimiter)
                         awk '$4==500 {print $1}'   (filter by field)
sed     SUBSTITUTE       sed 's/old/new/g'   -i (in-place)   -i.bak (backup)
tr      CHARACTERS       tr 'a-z' 'A-Z'   -d (delete)   -s (squeeze)

THE GOLDEN PIPELINE (top-N / count-each):
    awk '{print $1}' log | sort | uniq -c | sort -rn | head
    → extract → sort → count → rank → top few
```

8.6 Bash syntax cheat sheet ★★★★☆

```
SHEBANG      #!/bin/bash
VARIABLES    name="x"   (no spaces!)   echo "$name"   x=$(command)
ARGUMENTS    $1 $2 (positional)  $# (count)  $@ (all)  $0 (script name)
CONDITIONS   if [ $n -gt 5 ]; then ... elif ... else ... fi
             numbers: -eq -ne -gt -lt -ge -le    strings: = != -z -n
             files:   -f (file) -d (dir) -e (exists) -r -w -x
LOOPS        for x in list; do ... done      while [ cond ]; do ... done
             arithmetic: $(( a + b ))         seq 1 10
FUNCTIONS    name() { echo "$1"; return 0; }
EXIT CODES   0 = success; check $?;  cmd1 && cmd2  (and);  cmd1 || cmd2 (or)
CRON         min hour day-of-month month day-of-week  command
             0 2 * * *  = 2 AM daily      */15 * * * *  = every 15 min
```

⚠️ **Bash top traps:** no spaces around `=`; spaces required inside `[ ]`; use `-gt` not `>` for numbers; quote your `"$variables"`.

8.7 Git cheat sheet ★★★★★

```
DAILY        git clone URL    git status    git add .    git commit -m "msg"
             git push    git pull    git log --oneline    git diff
BRANCHES     git checkout -b feature   git switch main   git merge feature
             git branch -d feature
FETCH/PULL   fetch = download only    pull = fetch + merge
UNDO         git stash / git stash pop        (shelve changes)
             git revert COMMIT   (safe: new undo commit — use on shared)
             git reset --soft HEAD~1  (undo commit, keep changes)
             git reset --hard HEAD~1  (undo + DISCARD — destructive!)
CONFLICTS    <<<<<<< mine  =======  theirs  >>>>>>>
             edit to correct → delete markers → git add → git commit

RULES:  Pull first, push last.   Never rebase shared/pushed history.
        Use revert (not reset) on shared branches.
```

8.8 Master diagrams (redraw these from memory)

The OSI model (top → bottom):

```
7 Application   ┐
6 Presentation  ├ "Application" in TCP/IP   ← HTTP, DNS, TLS
5 Session       ┘
4 Transport     ← TCP / UDP / ports
3 Network       ← IP / routers
2 Data Link     ┐ "Link" in TCP/IP        ← Ethernet, MAC, switches
1 Physical      ┘                          ← cables, Wi-Fi
Mnemonic: "All People Seem To Need Data Processing"
```

TCP 3-way handshake:

```
Client ── SYN ──▶ Server        "can we talk?"
Client ◀ SYN-ACK ─ Server       "yes, can you hear me?"
Client ── ACK ──▶ Server        "yes – go"
```

"Type google.com and press Enter" — the whole journey:

```
DNS resolve  →  TCP handshake (port 443)  →  TLS handshake (cert + key)
   →  HTTP GET  →  [load balancer → reverse proxy → app → DB]
   →  HTTP 200 + HTML  →  browser renders (fetches CSS/JS/images)
Chant: "Do The Task, Handle The Response, Render."
```

Process state machine:

```
NEW → READY ⇄ RUNNING → TERMINATED
        ↑        ↓
      WAITING ◀──┘   (I/O; returns to READY when done, not RUNNING)
```

Git's four areas:

```
Working Dir ──add──▶ Staging ──commit──▶ Local Repo ──push──▶ Remote
     ◀──────────────────────────────────────────────pull──────┘
```

Deadlock's 4 conditions (all required):

```
Mutual exclusion + Hold-and-wait + No preemption + Circular wait
"Mine, held, no-take-backs, in a circle."  Break ONE to prevent it.
```

Containers vs VMs:

```
VM:        [App+OS][App+OS] / Hypervisor / Host OS / HW   (heavy, minutes)
Container: [App][App][App]  / Docker / Host OS(shared) / HW (light, seconds)
```

8.9 Interview one-liners (steal these exact phrasings) ★★★★★

Memorize these crisp sentences — they make you sound senior:

- **Process vs thread:** "A process has its own private memory; threads share memory within a process — fast, but race-prone."
- **Deadlock:** "Four conditions must all hold — mutual exclusion, hold-and-wait, no preemption, circular wait. Break one and you're safe; ordering locks breaks circular wait."
- **Context switch:** "Pure overhead — save one process's state, load another's; the cache goes cold, so too many switches waste CPU."
- **TCP vs UDP:** "TCP is a phone call — reliable and acknowledged. UDP is shouting across a room — fast, but no guarantees."
- **DNS:** "It turns a name into an IP through a caching hierarchy — and it returns an IP, not the page."
- **HTTPS:** "Asymmetric crypto to agree a key once, then fast symmetric crypto for the data; the certificate proves identity."
- **401 vs 403:** "401 is 'who are you?', 403 is 'I know you, and no.'"
- **Idempotent:** "Doing it again has no extra effect — which is why you can safely retry a PUT but not a POST."
- **Load balancer:** "Spreads traffic for scalability and removes dead servers via health checks for availability."
- **Reverse proxy:** "Sits in front of servers — clients see it, not the backend. reveRse = seRveR."
- **kill -9:** "Last resort — SIGKILL can't be caught, so the process can't clean up. I try SIGTERM first."
- **Disk full:** "`df -h` to find the full mount, `du -sh *` to drill into the culprit — usually runaway logs in /var/log."
- **Merge vs rebase:** "Merge preserves history with a merge commit; rebase makes it linear but rewrites history — never on shared branches."
- **Docker vs VM:** "Containers share the host kernel and bundle just the app — megabytes and seconds, versus a VM's whole OS."
- **IaaS/PaaS/SaaS:** "How much the provider manages — you cook, you bake, or you just eat."
- **Least privilege:** "Grant each user or service only the access it actually needs — nothing more."

8.10 The 60-second self-test (do this the morning of)

Can you answer each in one breath? If not, flip back to the chapter.

1. Process vs thread? 2. Four deadlock conditions? 3. TCP vs UDP + one use each? 4. What DNS returns? 5. How HTTPS agrees on a key? 6. 401 vs 403? 7. What happens when you type google.com? 8. Find the full disk (which commands)? 9. kill vs kill -9? 10. Stack vs heap? 11. merge vs rebase + the golden rule? 12. Resolve a merge conflict — steps? 13. Container vs VM? 14. IaaS/PaaS/SaaS with examples? 15. The golden log pipeline?

> If you can answer these 15 out loud, you are genuinely interview-ready. Everything else is bonus.

> **Next:** Chapter 9 — 100 interview questions with tight, 1-minute model answers. Read them out loud.

Chapter 9 — 100 Interview Questions & Model Answers

> **How to use this:** Each answer is written to be spoken in about **one minute** — the length a real interviewer expects. **Read them out loud.** Cover the answer, say yours, then compare. Speaking is a different skill from knowing, and it's the one being tested. Questions are grouped by topic and roughly ordered by frequency.

Operating Systems (Q1–20)

Q1. What is an operating system? The layer between applications and hardware. It manages resources — CPU, memory, disk, devices — and shares them safely among many programs. It abstracts hardware so programs use simple interfaces instead of talking to devices directly, and it isolates programs so one crash doesn't take down the system.

Q2. What's the difference between a process and a thread? A process is an independent program with its own private memory. A thread is a lighter unit of execution inside a process, and threads share the process's memory. That sharing makes threads fast to create and easy to communicate between, but it also means they can corrupt shared data, so you need synchronization like mutexes. Processes are isolated and safer but heavier.

Q3. What is a context switch and why is it expensive? It's when the CPU swaps from one process or thread to another. The OS saves the current one's state — registers, program counter — and loads the next one's. It's expensive because it's pure overhead: no useful work happens during the switch, and the CPU cache is now full of the old process's data, so the new one runs slower until the cache warms up.

Q4. Explain the states a process goes through. New when created, Ready when waiting for the CPU, Running when on the CPU. If it needs I/O it moves to Waiting and gives up the CPU. When the I/O finishes it goes back to Ready — not straight to Running, since the CPU may be busy. Finally it Terminates. The OS constantly shuffles processes between Ready and Running to keep the CPU busy.

Q5. What is a deadlock? When two or more processes wait on each other in a cycle, each holding a resource the other needs, so none can proceed. It requires four conditions at once: mutual exclusion, hold-and-wait, no preemption, and circular wait. Break any one and deadlock can't happen.

Q6. How do you prevent a deadlock? Break one of the four conditions. The most practical is breaking circular wait by imposing a global lock order — always acquire lock A before lock B — so a cycle can't form. You can also avoid hold-and-wait by grabbing all resources at once, or use timeouts to detect and recover.

Q7. Difference between a mutex and a semaphore? A mutex is a lock allowing exactly one thread into a critical section, and only the thread that locked it can unlock it. A semaphore is a counter allowing up to N threads through, used both to limit access to a pool of resources and to signal between threads. A binary semaphore resembles a mutex but has no ownership.

Q8. What's the difference between the stack and the heap? Both live in a process's memory. The stack holds local variables and function-call frames, managed automatically in last-in-first-out order — fast but small. The heap is for dynamically allocated memory requested at runtime — large and flexible but slower, lasting until freed or garbage collected. Deep recursion overflows the stack; forgetting to free heap memory causes a leak.

Q9. What is virtual memory? An abstraction giving each process its own private address space that the OS maps to physical RAM. It provides isolation, lets the system use disk as overflow through swap so programs can exceed physical RAM, and simplifies programming since every process sees a clean space starting at zero.

Q10. What is paging and what's a page fault? Paging splits memory into fixed-size pages in virtual memory and frames in physical memory, with a page table mapping between them. Fixed sizes remove external fragmentation. A page fault happens when a program accesses a page not currently in RAM — the OS pauses it, loads the page from disk, updates the table, and resumes. Constant faulting is thrashing, meaning you're out of RAM.

Q11. What's a system call? The controlled way a user-space program requests a service from the kernel, like opening a file or a socket. The CPU switches from user mode to kernel mode, the kernel checks permissions and does the privileged work, then switches back and returns the result. It's the single doorway between unprivileged apps and the privileged kernel.

Q12. Difference between kernel space and user space? Kernel space is privileged — code there can access hardware and any memory. User space is where normal apps run, sandboxed. When an app needs something privileged it asks the kernel through a system call. That boundary keeps a buggy app from crashing the whole system.

Q13. What is a zombie process? A process that has finished but still has an entry in the process table because its parent hasn't collected its exit status with wait(). It uses no CPU or memory — just a table entry. A few are harmless, but if a parent never reaps them, zombies pile up and can exhaust the process table.

Q14. What is an orphan process? A process whose parent terminated while it's still running. It gets adopted by init, PID 1, which becomes its new parent and reaps it when it finishes. So unlike zombies, orphans are cleaned up properly.

Q15. Compare FCFS, SJF, and Round Robin scheduling. FCFS runs jobs in arrival order — simple, but a long job blocks everyone, the convoy effect. SJF runs the shortest job first, giving the best average wait, but long jobs can starve. Round Robin gives each process a fixed time slice and rotates — fair and responsive, ideal for interactive systems, at the cost of context-switch overhead.

Q16. Which scheduler for an interactive system, and why? Round Robin. Each process gets a small fixed time slice, so no one hogs the CPU and everyone gets a quick response — exactly what interactive workloads need. The trade-off is the slice size: too big degrades to FCFS, too small wastes CPU on context switching.

Q17. What's the difference between deadlock and starvation? Deadlock is when processes are stuck in a cycle forever, none able to proceed. Starvation is when a process keeps getting skipped — like a long job under SJF — but the system as a whole is still making progress. Deadlock is total; starvation is unfair scheduling.

Q18. What is the kernel? The core of the OS — always running with full hardware access. It manages memory, schedules processes, handles system calls, and drives devices. Everything else runs with limited privilege and must go through the kernel for anything sensitive. Note: Linux is technically just the kernel; a distribution adds the surrounding tools.

Q19. What is a race condition? When two threads access shared data at the same time and the result depends on timing, corrupting the data — like two threads both incrementing a balance and one update getting lost. You prevent it with synchronization, such as a mutex protecting the critical section.

Q20. What's the difference between paging and segmentation? Paging divides memory into fixed-size blocks, which eliminates external fragmentation and is what modern systems use. Segmentation divides it into variable-size logical units — code, stack, heap. Paging is by convenience; segmentation is by meaning. Pure segmentation is rare today.

Networking (Q21–47)

Q21. Explain the OSI model. A 7-layer framework for how data moves across a network: Application, Presentation, Session, Transport, Network, Data Link, Physical — "All People Seem To Need Data Processing." Each layer has one job. In practice the ones that matter are Layer 7 HTTP, Layer 4 TCP/UDP, and Layer 3 IP.

Q22. TCP vs UDP? Both are transport-layer protocols. TCP is connection-based, reliable, and ordered — it uses a handshake and acknowledgements to guarantee delivery, so it's used for web, email, and file transfer. UDP is connectionless and best-effort — no handshake or guarantees, but faster, so it's used for video calls, gaming, and DNS.

Q23. When would you choose UDP over TCP? For real-time traffic like live video or gaming. A packet that arrives late is useless there — you'd rather drop it and stay real-time than stall waiting for a retransmission. TCP's reliability guarantees are the wrong trade-off; a small glitch beats a delay.

Q24. Explain the TCP three-way handshake. It sets up a connection in three steps: the client sends SYN ("let's talk"), the server replies SYN-ACK ("okay, can you hear me?"), and the client sends ACK ("yes, go"). After that, data flows on the established connection. It ensures both sides are ready and synchronized before sending data.

Q25. What is DNS and how does it work? DNS turns a domain name into an IP address. Your browser checks its cache, then the OS cache; if it's not there, it asks a recursive resolver, which walks the hierarchy — root server, then the TLD server like .com, then the domain's authoritative nameserver, which returns the IP. The resolver caches it for its TTL. Importantly, DNS returns an IP, not the web page.

Q26. What's a port? A number identifying a specific service on a machine. The IP address gets you to the right computer; the port gets you to the right program on it — since one server runs many services on the same IP. For example, 80 is HTTP, 443 HTTPS, 22 SSH.

Q27. Name some common ports. 22 SSH, 25 SMTP, 53 DNS, 80 HTTP, 443 HTTPS, 3306 MySQL, 5432 PostgreSQL, 6379 Redis, 27017 MongoDB.

Q28. How does HTTPS work? HTTPS is HTTP over TLS. On connecting there's a handshake: the server presents a certificate the browser verifies against a trusted Certificate Authority, proving identity. They use asymmetric encryption to agree on a shared session key, then switch to fast symmetric encryption for the actual data. So you get authentication, encryption, and integrity without the cost of asymmetric crypto for everything.

Q29. Symmetric vs asymmetric encryption in HTTPS? Asymmetric uses a public/private key pair — secure but slow — and is used only during the handshake to safely exchange a shared key. Symmetric uses that single shared key — fast — and encrypts all the actual data. HTTPS combines them: asymmetric to agree the key once, symmetric for speed thereafter.

Q30. What does it mean that HTTP is stateless? Every request is independent — the server doesn't inherently remember previous requests from the same client. That keeps servers simple and scalable, but it's why we need cookies, sessions, or tokens to maintain things like a logged-in state across requests.

Q31. What is a cookie? A small piece of data the server tells the browser to store and send back on every request to that site. Since HTTP is stateless, cookies let the server recognize a returning user — classically holding a session ID after login. You flag them HttpOnly so JavaScript can't steal them, and Secure so they only travel over HTTPS.

Q32. Difference between a cookie and a session? A cookie is client-side storage in the browser; a session is server-side storage. Usually they work together — the server keeps the user's real data in a session and sends a cookie with just the session ID. Sessions keep sensitive data on the server, which is more secure, but they use server memory.

Q33. Session vs JWT? A session stores state on the server and gives the client an ID; a JWT is stateless — the token itself carries the user's identity, signed by the server so it can't be forged. JWTs suit distributed systems since any server can verify them without a shared store, but they're hard to revoke before expiry. Note a JWT is signed, not encrypted, so never put secrets in it.

Q34. What makes an API RESTful? It's organized around resources addressed by URLs and uses standard HTTP methods for actions — GET to read, POST to create, PUT to update, DELETE to remove. It's stateless, so each request carries all needed context, and it typically exchanges JSON. The idea is predictability.

Q35. Difference between PUT and POST? POST creates a new resource and isn't idempotent — sending it twice creates two resources, which is why double-clicking submit can double-charge you. PUT replaces a resource at a known URL and is idempotent — sending it twice leaves the same final state.

Q36. What does idempotent mean? Repeating the request has no additional effect beyond the first. GET, PUT, and DELETE are idempotent; POST isn't. It matters for safe retries — if a network call times out, you can safely resend an idempotent request without risking a duplicate action.

Q37. Difference between 401 and 403? 401 Unauthorized means you're not authenticated — the server doesn't know who you are, so log in. 403 Forbidden means you are authenticated but don't have permission for this resource. In short: 401 is "who are you?", 403 is "I know you, and no."

Q38. What does a 502 mean and how does it differ from 500 and 503? 500 is a generic internal server error. 502 Bad Gateway means a proxy or load balancer got an invalid response from the upstream server behind it. 503 Service Unavailable means the server is overloaded or down. 504 is a gateway timeout —

upstream didn't answer in time. In on-call, a wave of 502s or 504s usually means a backend is down or slow.

Q39. What happens when you type google.com and press Enter? DNS resolves the name to an IP. The browser opens a TCP connection on port 443 via the three-way handshake. Because it's HTTPS, a TLS handshake follows — the server sends a certificate the browser verifies, and they agree a session key. The browser sends an HTTP GET; on the server side it may pass through a load balancer and reverse proxy to an app server and database, returning 200 OK with HTML. The browser parses it and fetches CSS, JS, and images to render the page.

Q40. What is a load balancer? It sits in front of a group of servers and spreads requests across them using an algorithm like round robin or least connections. It gives scalability — add servers to handle load — and availability, since it health-checks backends and stops routing to failed ones. Layer 4 routes by IP/port; Layer 7 understands HTTP and can route by URL path.

Q41. Forward proxy vs reverse proxy? It's about which side it protects. A forward proxy sits in front of clients — the destination server sees the proxy, not the real client; used for caching, filtering, or anonymity. A reverse proxy sits in front of servers — clients talk to it and it forwards to backends; used for load balancing, SSL termination, and hiding the backend. Nginx is the classic reverse proxy.

Q42. What is NAT? Network Address Translation lets many devices share one public IP. Devices use private IPs internally; the router rewrites outgoing packets' source to its public IP and keeps a table to route replies back to the right device. It conserves the limited IPv4 space and is what lets a whole household browse through one public address.

Q43. What does a firewall do? It filters network traffic against rules — typically by IP, port, and protocol — allowing or blocking connections. Best practice is default-deny: block everything, then open only the ports you need, like 443 and 22. In the cloud, security groups are essentially firewalls on your instances.

Q44. What is SSH and how do you use it securely? SSH is an encrypted protocol for remote login and running commands, over port 22. Securely, you use key-based authentication rather than passwords: keep a private key locally, put the public key on the server, and authenticate cryptographically without sending a password. It's more secure and easy to automate.

Q45. What is DHCP? It automatically assigns a device an IP address, plus subnet mask, gateway, and DNS, when it joins a network. The handshake is DORA: the device broadcasts Discover, the server sends an Offer, the device sends a Request, and the server sends an Ack with a lease. It saves configuring every device by hand.

Q46. Difference between the OSI and TCP/IP models? OSI is the 7-layer teaching model; TCP/IP is the practical 4-layer model the internet runs on. They map onto each other — OSI's top three layers collapse into TCP/IP's Application layer, and its bottom two into the Link layer. TCP/IP is what's actually implemented.

Q47. What layer does a router work at versus a switch? A router works at Layer 3, the Network layer — it routes between networks using IP addresses. A switch works at Layer 2, the Data Link layer — it forwards frames within a local network using MAC addresses. So switches connect devices on one network; routers connect different networks.

Linux (Q48–67)

Q48. How do you view the last 50 lines of a log and watch it update live? `tail -50 file` for the last 50 lines. To watch it in real time, `tail -f file` — the `-f` follows the file and prints new lines as they're written. That's my go-to when debugging a live service during a deploy.

Q49. Difference between find and grep? `find` searches for files by attributes — name, type, size, modification time. `grep` searches for text inside files. So find answers "where is the file?" and grep answers "which lines contain this text?" You often combine them — find to locate files, grep to search within.

Q50. The disk is full — how do you find what's using the space? Start with `df -h` to see which filesystem is full. Then drill in with `du -sh *`, moving into the largest directory to find the culprit — often runaway logs in /var/log. A handy one-liner is `du -h /path | sort -rh | head`. Then I clear or rotate the files and check why they grew — maybe rotation is broken.

Q51. Difference between df and du? `df` shows free space per filesystem — the big picture of which disk is full. `du` shows usage of specific files and directories — the detail for hunting down the culprit. You use df to spot the problem and du to locate it.

Q52. A process is stuck at 100% CPU — what do you do? I'd run `top` to confirm the process and get its PID. To stop it I'd start with `kill PID`, which sends SIGTERM and lets it shut down gracefully. If it ignores that, `kill -9 PID`, which is SIGKILL and can't be caught — but that's a last resort since it can't clean up. Then I'd check logs to find why it spun up.

Q53. Difference between kill and kill -9? `kill` sends SIGTERM — a polite request to shut down gracefully, so the process can save state and close files. `kill -9` sends SIGKILL, which can't be caught or ignored and terminates immediately, but may leave corrupt files or leaked resources. Always try `kill` first; use `-9` only when it won't die.

Q54. What does chmod 755 mean? Owner gets read-write-execute — that's 4+2+1 — and group and others get read-execute, 4+1. It's standard for scripts and directories: the owner can modify and run it, everyone else can read and run. 644 — read-write for owner, read-only otherwise — is typical for plain files.

Q55. How do you make a script executable? `chmod +x script.sh` adds the execute permission. Then I can run it with `./script.sh`. Under the hood, execute is the "1" bit in the numeric permissions.

Q56. How do you check if a service is running and restart it? `systemctl status service` shows if it's active, when it started, and recent logs. To restart after a config change, `systemctl restart service`, or `reload` to avoid downtime. `enable` makes it start on boot. If it won't start, I check `journalctl -u service -e` for the error.

Q57. How do you see which ports are open on a server? `ss -tulnp` — or the older `netstat -tulnp` — lists listening ports and which process owns each. It's the first thing I check when a service won't accept connections, to confirm it's actually listening on the expected port.

Q58. What are pipes and redirection? A pipe `|` sends one command's output into the next command's input, letting you chain small tools — like `ps aux | grep nginx`. Redirection sends output to files: `>` overwrites, `>>` appends, `2>` captures errors. Together they're the glue that makes small Unix tools powerful.

Q59. How would you count how many times "error" appears in a log? `grep -c error app.log` counts matching lines directly. Or with a pipe, `grep error app.log | wc -l`. If I wanted case-insensitive, I'd add `-i` to catch Error and ERROR too.

Q60. What's the difference between absolute and relative paths? An absolute path starts from root, like `/var/log/app.log`, and is the same regardless of where you are. A relative path is from your current directory, like `../logs/app.log`. Absolute is unambiguous; relative is shorter but depends on your location.

Q61. What does the /var/log directory contain? System and application logs. It's the first place I look when troubleshooting — for example `/var/log/syslog` or `/var/log/nginx/error.log`. Config lives in `/etc`, logs in `/var/log`, and user files in `/home` — those three cover most of what you touch.

Q62. How do you find files larger than 100 MB? `find / -size +100M` searches from root for files over 100 megabytes. It's useful when hunting down what's filling a disk. I'd usually scope it to a directory rather than all of `/` to keep it fast.

Q63. What does the "everything is a file" philosophy mean? In Linux, most things — documents, directories, devices, even some kernel info under /proc — are represented as files. That uniformity means the same tools like cat, grep, and redirection work across all of them, which is a big part of why the command line is so composable.

Q64. How do you search for text recursively in a directory? `grep -r "text" .` searches every file under the current directory. Adding `-n` shows line numbers and `-i` makes it case-insensitive, so `grep -rni "text" .` is a common form when I don't know which file contains something.

Q65. What's the difference between a hard link and a soft link? A soft link, or symlink, is a pointer to another path — if the target is deleted, the link breaks. A hard link is another name for the same underlying file data, so it still works even if the original name is removed. Symlinks are more common day to day; you make one with `ln -s`.

Q66. What is sudo? "Superuser do" — it runs a single command with root privileges. You use it for system-level actions like restarting services, installing software, or editing files in /etc. It's safer than logging in as root because it's per-command and logged.

Q67. How do you view running processes? `ps aux` gives a detailed snapshot of all processes, and `ps aux | grep name` filters to one. For a live, updating view sorted by CPU, `top` or `htop`. I use ps for a quick check and top when watching resource usage in real time.

Bash Scripting (Q68–77)

Q68. What's the first line of a shell script for? The shebang, `#!/bin/bash`. It tells the OS which interpreter should run the script. Without it, the system doesn't know the file is a Bash script and how to execute it.

Q69. How does a script access command-line arguments? Positional parameters: `$1` is the first, `$2` the second, and so on. `$0` is the script name, `$@` is all arguments, and `$#` is the count. I check `$#` at the top to validate the user passed what's required, printing usage and exiting if not.

Q70. How do you check if a file exists in Bash? `if [ -f /path/file ]; then ... fi`. The `-f` test is true if it exists and is a regular file; `-d` checks a directory and `-e` checks either. I use this constantly in ops scripts before acting on a config or log.

Q71. What's a common mistake with variable assignment in Bash? Putting spaces around the equals sign. It must be `name="value"` with no spaces — `name = "value"` makes Bash treat name as a command. Also, you assign without a dollar sign but read with one, like `$name`.

Q72. How do you know if the last command succeeded? Check `$?` — it holds the exit code, where 0 means success and non-zero means failure. I can branch on it with an `if`, or chain: `cmd1 && cmd2` runs the second only if the first succeeds, `cmd1 || cmd2` only if it fails.

Q73. What are the numeric comparison operators in Bash? `-eq` equal, `-ne` not equal, `-gt` greater than, `-lt` less than, `-ge` and `-le` for greater/less-or-equal. You use these for numbers inside test brackets — not the symbols like `>`, which mean redirection. Strings use `=` and `≠`.

Q74. How would you loop over all files in a directory? `for file in *; do ... done`, or `for file in *.log` to match a pattern. Inside I reference `$file`. For reading a file line by line I'd use a while loop: `while read line; do ...; done < input.txt`.

Q75. How do you schedule a script to run every night? A cron job. `crontab -e` and add `0 2 * * * /path/script.sh` to run at 2 AM daily. The five fields are minute, hour, day of month, month, day of week. I'd redirect output to a log to confirm it ran and debug failures.

Q76. What does $(command) do? Command substitution — it runs the command and captures its output into a value. For example `today=$(date +%Y-%m-%d)` stores the date in a variable. It's how you use one command's result inside a script.

Q77. How do you capture output and errors from a command? `>` redirects standard output to a file, `2>` redirects standard error, and `> out.log 2>&1` sends both to the same file. The `2>&1` means "send stream 2, stderr, to wherever stream 1 is going." Useful for logging a script's full output.

Text Processing — grep/awk/sed/tr (Q78–85)

Q78. Find all lines containing "error", case-insensitive, and count them. `grep -ic error file`. The `-i` makes it case-insensitive so it catches Error and ERROR, and `-c` counts matching lines instead of printing them. To see them with line numbers instead, `grep -in error file`.

Q79. From a log, print only the IPs of requests that returned 500. `awk '$4 == 500 {print $1}' access.log`. awk splits each line into fields, so `$4` is the status and `$1` the IP. The condition filters to 500s and prints the IP. awk beats grep here because it understands columns, not just whole lines.

Q80. How would you replace every occurrence of "foo" with "bar" in a file? `sed -i 's/foo/bar/g' file`. The `s` substitutes, `g` makes it replace every occurrence per line rather than just the first, and `-i` edits in place. I'd often use `-i.bak` to keep a backup, since regex mistakes on a live file are easy and hard to undo.

Q81. Find the top 5 IPs making requests in a web log. `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5`. awk pulls the IP, sort groups identical IPs together, `uniq -c` counts them — which is why sort comes first — then `sort -rn` orders by count descending and `head -5` takes the top five.

Q82. What's the difference between grep and awk? grep finds and prints whole lines matching a pattern. awk understands structure — it splits lines into fields, so it can extract specific columns, filter by a field's value, and do arithmetic. Use grep to find lines, awk to work with columns.

Q83. How do you convert text to uppercase or remove specific characters? `tr 'a-z' 'A-Z'` uppercases, piping the input in. `tr -d '0-9'` deletes digits. tr operates on individual characters, so it's right for case changes, deleting or squeezing characters, or swapping delimiters — not word or pattern matching.

Q84. What does uniq -c do, and why sort first? `uniq -c` collapses repeated adjacent lines and prefixes each with its count. It only compares adjacent lines, so you must sort first to group identical lines together — otherwise duplicates scattered through the file won't be counted correctly. That's why the pattern is always `sort | uniq -c`.

Q85. Extract the second column of a CSV file. `awk -F',' '{print $2}' data.csv`. The `-F','` sets the field separator to a comma so awk splits on commas, then `$2` is the second field. The same trick with `-F':'` works on colon-separated files like /etc/passwd.

Git (Q86–95)

Q86. Walk me through your normal Git workflow. I pull the latest first so I'm on current code. I edit, then `git status` and `git diff` to review. I stage with `git add`, commit with a clear message about why, and push. For features I work on a branch and open a pull request so it's reviewed before merging to main.

Q87. Difference between git fetch and git pull? `fetch` downloads remote changes but leaves my working branch untouched, so I can review before integrating. `pull` is fetch plus merge — it downloads and immediately merges into my current branch. Pull is the quick everyday command; fetch is safer when I want to look first.

Q88. Difference between merge and rebase? Both integrate one branch into another. Merge creates a merge commit and preserves the branching history — safe and non-destructive. Rebase replays my commits on top of the target for a clean linear history, but it rewrites history. I rebase my own local branch to tidy it, merge for shared branches, and never rebase already-pushed history.

Q89. Why shouldn't you rebase a shared branch? Rebasing rewrites commit history, creating new commit IDs. If others have already pulled the old commits, their history no longer matches, which causes conflicts and confusion for the whole team. So rebase only your own local, un-pushed work.

Q90. How do you resolve a merge conflict? A conflict happens when two branches edit the same lines. Git marks the sections with `<<<<<<<`, `=======`, `>>>>>>>` showing both versions. I open each conflicted file, decide the correct final code — mine, theirs, or a mix — remove the markers, then `git add` and `git commit`. `git status` lists which files still need resolving.

Q91. Difference between git reset and git revert? Both undo, differently. `revert` creates a new commit that reverses an earlier one, keeping history intact — safe on shared branches. `reset` moves the branch pointer backward and effectively removes commits, rewriting history — risky if pushed. On shared branches I use revert; reset only for local, un-pushed commits.

Q92. What does git stash do? It shelves your uncommitted changes and gives you a clean working directory, so you can switch branches or pull without committing half-done work. `git stash pop` brings the changes back. It's like sweeping your desk into a drawer to handle something urgent, then tipping it back out.

Q93. What's the staging area? An intermediate area between your working directory and the repository. `git add` moves changes into staging, and `git commit` saves what's staged. It lets you choose exactly which changes go into each commit, rather than committing everything at once — so you can craft focused, logical commits.

Q94. How do you create and switch to a new branch? `git checkout -b feature-name` creates and switches in one step, or `git switch -c feature-name` in newer Git. Branches let me work in isolation without touching main, then merge back when ready.

Q95. What's the difference between git reset –soft and –hard? `--soft` moves the branch pointer back but keeps your changes staged, so you can re-commit them — good for redoing a commit message. `--hard` moves the pointer back and discards the working changes entirely, which is destructive and unrecoverable, so I use it carefully.

Cloud Computing (Q96–100)

Q96. What is cloud computing and why do companies use it? Renting computing resources — servers, storage, databases — over the internet instead of owning hardware. Companies use it because it turns big upfront costs into pay-as-you-go, and it's elastic: scale up when busy, down when quiet. It also offloads physical maintenance to the provider and enables fast global deployment.

Q97. Explain IaaS, PaaS, and SaaS with examples. Layers of how much the provider manages. IaaS, like AWS EC2, gives raw virtual machines — you manage the OS and app, most control. PaaS, like Heroku, manages the OS, runtime, and scaling — you just deploy code. SaaS, like Gmail, runs everything — you just use it. The trade-off is control versus convenience.

Q98. What's the difference between EC2 and S3? EC2 is compute — a virtual server you run your application on and control at the OS level. S3 is object storage — you put files like images, backups, or logs into buckets and retrieve them over HTTP. EC2 is a machine; S3 is a place to store files, not a disk for

the OS.

Q99. What is Docker and how is a container different from a VM? A container packages an app with all its dependencies so it runs identically everywhere, solving "works on my machine." Docker is the standard tool. Unlike a VM, which bundles a whole guest OS and is heavy — gigabytes, minutes to boot — a container shares the host kernel and bundles just the app, so it's megabytes and starts in seconds.

Q100. What is Kubernetes and why use it? It's a container orchestrator — it manages containers at scale across a cluster. Docker runs individual containers, but with hundreds across many servers you need automation to deploy, scale, restart crashed ones, and route traffic. Kubernetes is self-healing, auto-scaling, load-balancing, and does zero-downtime rolling updates. Docker packs the box; Kubernetes runs the fleet.

> **You've now rehearsed the 100 questions that cover the vast majority of what you'll be asked.** Say the ones you fumbled out loud three more times. Then move to Chapter 10 — the mock interviews, where you'll practice stringing these together under pressure.

Chapter 10 — 25 Mock Interview Scenarios

> **How to use this chapter:** These are *scenario* questions — the "here's a situation, what do you do?" style that dominates on-call and support interviews. For each one: **read the scenario, then genuinely pause and answer out loud for about 5 seconds of thinking before you speak** (the ⏱️ marks that pause). Only then read the ideal answer and the explanation of *why* it's strong. The goal is to practice the *thinking process* interviewers score, not just the fact.

> 🎯 **The universal troubleshooting framework** (use it whenever you're unsure): **Observe → Isolate → Fix → Verify → Prevent.** Say what you'd *check* before what you'd *do*. Interviewers care more about your method than the exact command.

Scenario 1 — "The website is returning 502 errors. Walk me through what you'd do."

⏱️ *(pause, think, then answer)*

✅ **Ideal answer:** "A 502 Bad Gateway means the load balancer or reverse proxy got an invalid response from the backend — so the problem is likely the backend, not the proxy itself. First I'd confirm the scope: is it every request or intermittent? I'd check whether the backend app servers are actually running with `systemctl status`, and look at their logs with `journalctl` or `tail -f` for crashes or errors. I'd check resources — `top` for CPU, `free -h` for memory, `df -h` for disk, since a full disk or OOM can take a backend down. If a backend crashed, I'd restart it and watch the 502s clear, then investigate the root cause from the logs so it doesn't recur."

💡 **Why it's strong:** It correctly diagnoses that 502 points *downstream* to the backend, lays out an ordered checklist, and ends with root-cause and prevention — not just "restart it."

Scenario 2 — "A server's disk is 100% full. What do you do?"

⏱️

✅ **Ideal answer:** "I'd start with `df -h` to confirm which filesystem is full. Then I'd drill in with `du -sh /*` and follow the biggest directory down — `du -sh /var/*` and so on — to find the culprit, which is very often runaway logs in /var/log. Before deleting anything I'd check what it is; if it's logs, I'd truncate or rotate them rather than delete files a running process still holds open — deleting an open file doesn't free the space until the process releases it. Then I'd fix the root cause: maybe log rotation is broken or a process is logging in a loop. I'd also set up an alert at, say, 80% so we catch it before 100% next time."

💡 **Why it's strong:** Uses the df→du drill, shows the subtle "deleting an open file doesn't free space" knowledge, and adds prevention (rotation + alert).

Scenario 3 — "Users say the app is slow. How do you investigate?"

⏱️

✅ **Ideal answer:** "'Slow' is vague, so first I'd quantify it — which action, how slow, when it started, everyone or some users. Then I'd work up the stack. On the server, `top` for CPU and load average, `free -h` for memory and swap — swapping kills performance — and `df -h` for disk. I'd check the app logs for slow queries or errors. Networking-wise, is it slow to connect or slow to respond? I'd look at whether a database is the bottleneck, since slow queries are a common cause. The key is to measure and isolate one layer at a time rather than guess."

💡 **Why it's strong:** Refuses to guess, quantifies the vague word "slow," and shows a systematic layer-by-layer method.

Scenario 4 — "You can't SSH into a server. How do you troubleshoot?"

⏱️

✅ **Ideal answer:** "I'd isolate where it breaks. First, is it network or the service? I'd `ping` the server — if that fails it's a network or the host is down. If ping works, I'd check the SSH port with something like `telnet host 22` or `nc -zv host 22` to see if port 22 is open — if it's refused, sshd may be down or a firewall is blocking it. I'd check whether it's just me by asking a colleague. If I have console access, I'd check `systemctl status sshd` and the firewall rules. Common causes are the SSH service being down, a firewall blocking port 22, or the server being fully out of resources so it can't accept connections."

💡 **Why it's strong:** Classic isolate-the-layer approach — network → port → service — and names concrete tools and common causes.

Scenario 5 — "A process is using 100% CPU. How do you handle it?"

⏱️

✅ **Ideal answer:** "I'd run `top` to identify the process and note its PID. I'd decide whether it's legitimately busy or genuinely stuck — a batch job at 100% might be fine. If it needs stopping, I'd try `kill PID` first, which sends SIGTERM and lets it shut down gracefully. If it ignores that, `kill -9 PID` as a last resort, since SIGKILL can't be caught but also can't clean up. Crucially, I'd capture logs and details first so I can find *why* it spiked — killing it treats the symptom, not the cause."

💡 **Why it's strong:** Distinguishes busy vs stuck, uses SIGTERM before SIGKILL, and prioritizes root cause.

Scenario 6 — "How would you find which files are eating disk space in /var?"

⏱️

✅ **Ideal answer:** "`du -sh /var/*` to see the size of each subdirectory, then I follow the biggest one down — `du -sh /var/log/*` and so on. A quick way to rank everything is `du -h /var | sort -rh | head -20`, which lists the twenty largest paths. That combination — du to measure, sort -rh to rank, head

to trim — pinpoints the culprit fast, usually oversized logs."

💡 **Why it's strong:** Gives the exact drill-down commands plus the ranking one-liner, showing real hands-on familiarity.

Scenario 7 — "Explain what happens when you type a URL and press Enter — as much detail as you can."

⏱️

✅ **Ideal answer:** "DNS resolves the domain to an IP — browser cache, OS cache, then a recursive resolver walking root, TLD, and authoritative servers. The browser opens a TCP connection on port 443 with the three-way handshake. Since it's HTTPS, a TLS handshake follows: the server sends a certificate the browser verifies against a CA, and they agree a symmetric session key. The browser sends an HTTP GET; server-side it may pass through a load balancer and reverse proxy to an app server and database, returning 200 OK with HTML. The browser parses the HTML and fetches CSS, JS, and images — each possibly repeating the process — then renders the page."

💡 **Why it's strong:** Hits every layer in order — DNS, TCP, TLS, HTTP, server, render — which is exactly what this flagship question tests.

Scenario 8 — "A cron job was supposed to run last night but didn't. How do you debug it?"

⏱️

✅ **Ideal answer:** "First I'd verify the schedule with `crontab -l` — a wrong field is the most common cause. Then I'd check the cron logs, usually in /var/log/syslog or via `journalctl -u cron`, to see if it even attempted to run. Common gotchas: cron runs with a minimal environment, so a script relying on custom PATH or variables can fail — I'd use absolute paths. I'd also check the script has execute permission and that I redirected its output to a log so I can see errors. I'd test the script manually to confirm it works outside cron, which isolates whether it's the schedule or the script."

💡 **Why it's strong:** Knows the top real cron pitfalls — schedule syntax, minimal environment/PATH, permissions, output logging — which only comes from experience.

Scenario 9 — "How do you check if a remote port is open?"

⏱️

✅ **Ideal answer:** "From another machine, `nc -zv host 443` or `telnet host 443` tests whether that port accepts a connection. On the server itself, `ss -tulnp` — or `netstat -tulnp` — shows what's listening and which process owns it. So I'd confirm from outside that the port is reachable, and from inside that the service is actually listening on it. If it's listening locally but not reachable remotely, that points to a firewall."

💡 **Why it's strong:** Tests from both sides and interprets the result (local-but-unreachable = firewall).

Scenario 10 — "Difference between 401 and 403 — and give a real example of each."

⏱️

✅ **Ideal answer:** "401 Unauthorized means you're not authenticated — the server doesn't know who you are. Example: hitting an API without a login token; you need to log in. 403 Forbidden means you *are* authenticated but lack permission. Example: a logged-in regular user trying to open an admin-only page — the server knows who they are and is refusing. Short version: 401 is 'who are you?', 403 is 'I know you, and no.'"

💡 **Why it's strong:** Nails the distinction *and* gives a concrete example of each — examples are what make it land.

Scenario 11 — "Your teammate force-pushed and now your local branch is broken. What happened and what do you do?"

⏱️

✅ **Ideal answer:** "They probably rebased or reset a shared branch and force-pushed, which rewrote history — so the commits I have no longer match the remote. That's exactly why you avoid rebasing shared branches. To recover, I'd first make sure my own unpushed work is safe, maybe with a branch or stash. Then I'd fetch the new remote state and reconcile — often `git reset --hard origin/branch` to match the remote if I have no local work to keep, or carefully rebase my local commits onto the new history if I do. The lesson I'd raise with the team is to not force-push shared branches."

💡 **Why it's strong:** Correctly diagnoses rewritten history, protects local work first, and turns it into a team-process lesson.

Scenario 12 — "Design a simple script to alert when memory usage is too high."

⏱️

✅ **Ideal answer:** "I'd write a Bash script that reads memory usage, compares it to a threshold, and alerts if exceeded. Something like: capture used-percentage with `free` piped through `awk`, then `if [ $usage -gt 90 ]; then` send an alert — email, Slack webhook, or just log it. I'd schedule it with cron every few minutes. Key touches: use a configurable threshold variable, and make the alert actionable — include the hostname and the actual number. I'd also avoid alert spam by only alerting on state changes if this were production."

💡 **Why it's strong:** Sketches real Bash (free + awk + if + threshold + cron) and shows production polish — actionable alerts, anti-spam.

Scenario 13 — "The database connection is failing from the app. Where do you start?"

⏱️

✅ **Ideal answer:** "I'd isolate the layer. Is the database process running? `systemctl status` on the DB host. Is it reachable on its port — 3306 for MySQL, 5432 for Postgres — via `nc -zv dbhost 5432`? If the port's closed, it's the DB service or a firewall. If it's open, I'd check credentials and connection limits — 'too many connections' is a classic. I'd read both the app logs and the database logs for the exact error. I'd also check whether the DB host is out of disk or memory, since that can refuse connections. The error message usually tells you which of these it is, so I'd start there."

💡 **Why it's strong:** Systematic — process, port, credentials, limits, resources — and anchors on reading the actual error.

Scenario 14 — "How would you count the number of unique visitors in a web access log?"

⏱️

✅ **Ideal answer:** "Assuming the client IP is the first field, `awk '{print $1}' access.log | sort | uniq | wc -l`. awk extracts the IPs, sort groups them, `uniq` collapses duplicates to one per IP, and `wc -l` counts the unique lines. If I wanted to see the busiest ones instead, I'd swap in `uniq -c | sort -rn | head` to rank them by request count."

💡 **Why it's strong:** Builds the exact pipeline and shows the flexible variant, demonstrating command fluency.

Scenario 15 — "A deployment went out and errors spiked. What's your immediate action?"

⏱️

✅ **Ideal answer:** "In on-call, the priority is stopping the bleeding, not diagnosing first. If errors spiked right after a deploy, the deploy is the prime suspect, so my immediate action is to **roll back** to the last known-good version to restore service. Once users are okay, *then* I investigate calmly — compare the logs and the diff between versions to find what broke. Recover first, root-cause second. I'd also communicate status to stakeholders throughout."

💡 **Why it's strong:** Shows the on-call instinct — mitigate/rollback first, diagnose after — plus communication. This mindset is exactly what these roles hire for.

Scenario 16 — "Explain TCP vs UDP to a non-technical person."

⏱️

✅ **Ideal answer:** "TCP is like a phone call — you confirm the other person is there, and if they miss something you repeat it, so nothing is lost but it takes a bit more effort. UDP is like shouting across a noisy room — it's fast and you don't wait for confirmation, but if a word gets lost, you just keep going. So you use TCP when every piece must arrive, like a web page or a file, and UDP when speed matters more than perfection, like a live video call."

💡 **Why it's strong:** Uses a clean analogy and connects it to *when* you'd use each — the "explain simply" skill support roles prize.

Scenario 17 — "You see thousands of requests from one IP hammering the server. What do you do?"

⏱️

✅ **Ideal answer:** "First confirm it with the logs: `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head` to see the top IPs by request count. If one IP is clearly abusive — far above normal — it could be a bad actor, a misbehaving client, or a scraper. Short term, I'd rate-limit or block that IP at the firewall or load balancer to protect the service. Then I'd check whether it's malicious or just a buggy client, and consider a longer-term fix like rate limiting per IP. I'd be careful not to block a legitimate shared IP, like a corporate NAT, without checking."

💡 **Why it's strong:** Confirms with the golden pipeline, mitigates, then reasons about false positives (NAT) — nuance impresses.

Scenario 18 — "What's the difference between a container and a VM, and when would you use each?"

⏱️

✅ **Ideal answer:** "A VM bundles a full guest OS, so it's heavy — gigabytes and minutes to boot — but strongly isolated. A container shares the host's kernel and bundles just the app and its dependencies, so it's megabytes and starts in seconds, letting you pack many more per machine. I'd use containers for deploying and scaling modern applications, especially microservices, because they're fast and portable. I'd use VMs when I need stronger isolation, a different OS kernel, or to run legacy software that expects a full machine."

💡 **Why it's strong:** Contrasts on the key axes and gives clear "when to use each," which is the real question.

Scenario 19 — "How would you securely give a new engineer access to a server?"

⏱️

✅ **Ideal answer:** "I'd use SSH key-based access, not shared passwords. They generate a key pair, send me the public key, and I add it to their user's authorized_keys on the server — the private key never leaves their machine. I'd give them their own account, not a shared one, so actions are auditable, and grant only

the permissions they need — least privilege — using sudo for specific commands rather than full root. When they leave, I just remove their key and account. Individual keys plus least privilege is the secure, auditable approach."

💡 **Why it's strong:** Hits SSH keys, individual accounts, least privilege, and offboarding — a complete security mindset.

Scenario 20 — "Logs show 'out of memory' and a process was killed. What happened?"

⏱️

✅ **Ideal answer:** "That's the OOM killer. When the system runs out of memory and swap, the Linux kernel steps in and kills a process to free memory — usually the one using the most — and logs it as 'Out of memory: Killed process.' So a process being OOM-killed means the machine ran out of RAM. I'd check `free -h` and the memory trend, identify what consumed it — a leak, an under-provisioned box, or a spike in load — and fix accordingly: add memory, tune the app's limits, or fix the leak. It's the kernel protecting the system from total collapse."

💡 **Why it's strong:** Correctly names the OOM killer, explains *why* the kernel does it, and moves to root cause and fixes.

Scenario 21 — "Walk me through resolving a Git merge conflict."

⏱️

✅ **Ideal answer:** "A conflict means two branches changed the same lines and Git can't decide automatically. `git status` shows the conflicted files. In each, Git inserts markers — `<<<<<<<` for my version, `=======`, then `>>>>>>>` for the incoming version. I read both sides, decide the correct final result — sometimes mine, sometimes theirs, often a combination — edit the file, and delete the markers. Then `git add` the resolved files and `git commit` to complete the merge. The important part is understanding both changes rather than blindly picking a side."

💡 **Why it's strong:** Clear step-by-step with the marker knowledge and the "understand both sides" judgment.

Scenario 22 — "A service keeps crashing and restarting. How do you find out why?"

⏱️

✅ **Ideal answer:** "The crash-loop itself is a symptom, so I'd go to the logs for the cause: `journalctl -u service -e` for the most recent entries, or `systemctl status service` which shows the last error and exit code. I'd look for what happens right before each crash — a config error, a missing dependency, a port already in use, or running out of memory. If it's under systemd with auto-restart, that explains the looping. Once I see the actual error, I fix that — the restart is just systemd doing its job; my job is the underlying failure."

💡 **Why it's strong:** Goes straight to logs for root cause, lists realistic causes, and understands why it's *looping* (auto-restart).

Scenario 23 — "How does HTTPS keep my data safe? Explain simply."

⏱️

✅ **Ideal answer:** "Three things. First, encryption — your data is scrambled so anyone intercepting it just sees gibberish, not your password. Second, identity — the website presents a certificate, like an ID card verified by a trusted authority, so you know you're really talking to your bank and not an impostor. Third, integrity — the data can't be secretly altered in transit. Technically, it uses a slow but secure method once to agree on a secret key, then fast encryption with that key for everything after — best of both worlds."

💡 **Why it's strong:** Covers all three guarantees plus the asymmetric-then-symmetric insight, explained accessibly.

Scenario 24 — "You get paged at 3 AM that the site is down. Describe your first five minutes."

⏱️

✅ **Ideal answer:** "First, acknowledge the page so the team knows it's being handled. Then confirm the impact — is the whole site down or one feature, all users or some? I'd check the obvious health signals: is the service running, are the servers up, what do the dashboards and error rates show. I'd look at what changed recently — a deploy, a config change, a traffic spike — since that's the usual trigger. My priority is restoring service, so if a recent change caused it I'd roll back rather than debug live. And I'd communicate status early and often, even if it's just 'investigating.' Recover first, root-cause in the morning."

💡 **Why it's strong:** Demonstrates real on-call discipline — acknowledge, assess impact, check recent changes, mitigate over debug, communicate — which *is* the job.

Scenario 25 — "How would you approach a problem you've never seen before?"

⏱️

✅ **Ideal answer:** "I'd stay systematic instead of panicking. Start by gathering information — read the actual error message carefully, check the logs, and reproduce it if I can. Then form a hypothesis and test it, changing one thing at a time so I know what fixed it. I'd use the resources available — documentation, runbooks, and asking a teammate rather than staying stuck too long, since knowing when to escalate is part of the job. And I'd take notes so we can document it afterward for next time. The method matters more than knowing every answer up front."

💡 **Why it's strong:** This is really a *behavioral* question testing temperament. The answer shows composure, method, appropriate escalation, and a learning mindset — exactly what interviewers hope to hear from a junior.

How to practice these

1. **Cover the answer.** Read only the scenario.
2. **Speak for 60–90 seconds.** Record yourself if you can.
3. **Compare** against the ideal answer — not word-for-word, but did you hit the *method* and the *key facts*?
4. **Re-do the ones you fumbled** until the framework — Observe → Isolate → Fix → Verify → Prevent — comes automatically.

> **The meta-lesson across all 25:** interviewers aren't checking whether you memorized a command. They're checking whether you *think* like an engineer under pressure — measure before you act, mitigate before you debug, find the root cause, and communicate. Show that, and you'll pass.

> **Next:** Chapter 11 — the Final Exam. Time to test yourself for real.

Chapter 11 — Final Exam

> **How to use this:** Do the exam **closed-book** first — no peeking. Then check the answer keys at the end of each section. Score yourself. Anything you miss, flip back to the relevant chapter *the same day*. Aim for 80%+ before your interview.
>
> **Sections:** 100 Multiple-Choice · 50 Short Questions · 20 Practical Linux · 20 Networking Scenarios · Answer Keys.

Part A — 100 Multiple-Choice Questions

(Answer key at the end of Part A.)

Operating Systems (1–20)

1. Threads of the same process each have their own: a) Heap b) Global variables c) Stack d) Code segment

2. How many of the four Coffman conditions must hold for a deadlock? a) All four b) Any one c) Exactly three d) At least two

3. A process that has finished but whose exit status hasn't been read is a: a) Zombie b) Orphan c) Daemon d) Thread

4. Which scheduling algorithm is best for interactive systems? a) FCFS b) SJF c) Priority (non-preemptive) d) Round Robin

5. The stack is characterized by: a) LIFO, automatic, fast b) Manual allocation c) Unlimited size d) Garbage collection

6. A page fault occurs when: a) The disk is full b) A referenced page isn't in RAM c) A page is corrupted d) The CPU overheats

7. A mutex allows how many threads in a critical section at once? a) Up to N b) Zero c) Exactly one d) Unlimited

8. Which is TRUE of virtual memory? a) It's the same as the CPU cache b) It replaces RAM entirely c) It gives each process a private address space d) It only works with SSDs

9. An orphan process is adopted by: a) init / PID 1 b) The kernel scheduler c) The nearest parent d) No one; it dies

10. A context switch is expensive mainly because: a) It uses network bandwidth b) It needs root privileges c) It's pure overhead and cools the cache d) It requires disk writes always

11. Which lives in kernel space? a) Your web browser b) A Python script c) A text editor d) The process scheduler

12. The transition from Running to Waiting typically happens because: a) It needs I/O b) The process finished c) The CPU failed d) It was created

13. SJF scheduling's main risk is: a) Convoy effect b) Deadlock c) Starvation of long jobs d) Too many context switches

14. Paging eliminates which kind of fragmentation? a) Internal b) External c) Both d) Neither

15. A system call causes: a) A reboot b) A switch from user mode to kernel mode c) A network request d) A page fault always

16. Two threads incrementing a shared counter without locks can cause a: a) Deadlock b) Page fault c) Context switch d) Race condition

17. Which is NOT one of the four deadlock conditions? a) Mutual exclusion b) Preemption allowed c) Circular wait d) Hold and wait

18. Heap memory in a garbage-collected language is freed: a) When the function returns b) By the GC when unreachable c) Never d) At compile time

19. "Linux" technically refers to the: a) Whole operating system b) Desktop environment c) Kernel d) Package manager

20. A binary semaphore has a count of: a) 0 b) 2 c) 1 d) N

Networking (21–50)

21. TCP provides all EXCEPT: a) The lowest possible latency b) Reliability c) Ordering d) Acknowledgements

22. DNS primarily returns: a) The web page b) A MAC address c) An IP address d) A port number

23. HTTPS default port is: a) 443 b) 80 c) 22 d) 8080

24. The TCP handshake order is: a) ACK, SYN, SYN-ACK b) SYN, SYN-ACK, ACK c) SYN, ACK, SYN d) SYN-ACK, SYN, ACK

25. A 403 status code means: a) Not authenticated b) Server error c) Authenticated but not allowed d) Not found

26. A 401 status code means: a) Not authenticated b) Forbidden c) Bad gateway d) Too many requests

27. Which protocol is connectionless? a) TCP b) HTTP c) UDP d) FTP

28. SSH uses port: a) 21 b) 22 c) 23 d) 25

29. In TLS, the shared session key is used for: a) Verifying the certificate b) DNS lookups c) Fast symmetric encryption of data d) Nothing

30. A reverse proxy sits in front of: a) Clients b) Servers c) Routers d) DNS

31. A 5xx status code indicates: a) Client error b) Redirect c) Success d) Server error

32. Which is idempotent? a) POST b) PUT c) PATCH (always) d) None

33. DHCP's four-step process is: a) SYN-ACK-FIN-RST b) Get-Post-Put-Delete c) Discover-Offer-Request-Ack d) Root-TLD-Auth-Cache

34. NAT primarily solves: a) Slow DNS b) IPv4 address shortage c) Weak encryption d) Deadlocks

35. A cookie is stored: a) On the server b) In the browser (client-side) c) In DNS d) On the router

36. A session's data is stored: a) Client-side b) In the URL c) Server-side d) In the cookie itself

37. Which OSI layer is TCP? a) 3 Network b) 7 Application c) 2 Data Link d) 4 Transport

38. IP addressing happens at OSI layer: a) 2 b) 3 c) 4 d) 7

39. A JWT is: a) Encrypted by default b) Signed but not encrypted by default c) Stored server-side d) A type of cookie only

40. A load balancer improves: a) Only security b) DNS speed only c) Scalability and availability d) Encryption strength

41. UDP is preferred for: a) File downloads b) Live video calls c) Banking transactions d) Email

42. MySQL's default port is: a) 5432 b) 6379 c) 3306 d) 27017

43. A 301 status code means: a) Temporary redirect b) Permanent redirect c) Not modified d) Created

44. The mnemonic "All People Seem To Need Data Processing" is for: a) TCP handshake b) HTTP methods c) DHCP d) OSI layers

45. A firewall's best-practice default policy is: a) Allow all b) Deny all, then allow specifics c) Allow all internal d) No policy

46. A 502 Bad Gateway usually means: a) DNS failed b) The client sent bad data c) The upstream/backend gave an invalid response d) Rate limited

47. POST is used to: a) Read a resource b) Create a resource c) Delete a resource d) Replace a resource

48. A forward proxy is used for: a) Load balancing servers b) SSL termination c) Client caching/filtering/anonymity d) Hiding backends

49. The recursive resolver in DNS asks servers in this order: a) Authoritative, TLD, Root b) Root, TLD, Authoritative c) TLD, Root, Authoritative d) Cache only

50. DNS most commonly runs over: a) TCP only b) UDP (port 53) c) HTTP d) ICMP

Linux (51–75)

51. To watch a log file update live: a) cat file b) head file c) tail -f file d) less file

52. Which finds text *inside* files? a) grep b) find c) ls d) locate

53. `chmod 755` gives others: a) rwx b) rw- c) r-x d) —

54. Which shows free disk space per filesystem? a) du b) ls c) df d) free

55. Which shows usage of specific directories? a) df b) top c) du d) ps

56. `kill -9` sends: a) SIGTERM b) SIGHUP c) SIGKILL d) SIGINT

57. The polite signal to stop a process is: a) SIGKILL b) SIGTERM c) SIGSTOP d) SIGSEGV

58. To see memory usage: a) free -h b) df -h c) du -sh d) ps aux

59. Which lists all processes in detail? a) ps aux b) ls -la c) jobs d) top -n1 only

60. The permission value for read is: a) 1 b) 2 c) 4 d) 7

61. `mv old.txt new.txt` does what? a) Renames/moves b) Copies c) Deletes d) Links

62. To make a script executable: a) chmod +r b) chmod 000 c) chmod +x d) chown

63. Which restarts a systemd service? a) systemctl restart X b) service restart c) kill -HUP d) reboot

64. To read a systemd service's logs: a) tail -f service b) dmesg only c) journalctl -u X d) cat /service

65. The pipe `|` does what? a) Overwrites a file b) Appends to a file c) Sends one command's output to another d) Runs in background

66. `>>` does what? a) Overwrites b) Reads input c) Appends d) Pipes

67. Logs typically live in: a) /var/log b) /etc c) /home d) /bin

68. Config files typically live in: a) /var b) /tmp c) /etc d) /dev

69. `find / -size +100M` finds: a) Files modified in 100 min b) 100 files c) Files larger than 100 MB d) Directories only

70. `grep -v error` shows: a) Lines without "error" b) Only error lines c) Line count d) Case-insensitive matches

71. `grep -i` means: a) Invert b) Recursive c) Ignore case d) Line numbers

72. `ps aux | grep nginx` does what? a) Lists processes, filters to nginx b) Kills nginx c) Starts nginx d) Restarts nginx

73. `cd ~` goes to: a) Root b) Parent dir c) Home directory d) Previous dir

74. Which command shows open listening ports? a) ss -tulnp b) ls -l c) df -h d) top

75. `sudo` means: a) Shutdown b) Superuser do c) Sudo directory d) System update

Bash / Text / Git / Cloud (76–100)

76. The shebang line is: a) # bash b) #!/bin/bash c) //bash d) @bash

77. In Bash, `$1` is: a) The script name b) The exit code c) The first argument d) All arguments

78. `$#` gives: a) The count of arguments b) The script name c) The last exit code d) The PID

79. The exit code for success is: a) 1 b) -1 c) 0 d) 200

80. Correct variable assignment: a) `x=5` b) `x = 5` c) `$x=5` d) `let x = 5`

81. To test if a file exists: a) `[ -f file ]` b) `[ -e = file ]` c) `[ file ]` d) `[ exists file ]`

82. Numeric "greater than" in Bash test is: a) `>` b) `gt` c) `-gt` d) `>>`

83. A cron field order is: a) hour min day month weekday b) sec min hour day month c) min hour day-of-month month day-of-week d) weekday month day hour min

84. `0 2 * * *` runs: a) Every 2 hours b) At 2 PM on the 2nd c) Every 2 minutes d) At 2 AM daily

85. Which tool is best for extracting a column? a) grep b) tr c) awk d) cat

86. `sed 's/a/b/g'` — the `g` means: a) Global (all matches per line) b) Greedy c) Grep mode d) Go

87. `tr 'a-z' 'A-Z'` does: a) Deletes letters b) Counts letters c) Reverses d) Uppercases

88. The "golden pipeline" for top-N counts is: a) `grep | wc` b) `cat | tail` c) `awk | sed` d) `... | sort | uniq -c | sort -rn | head`

89. `awk '{print $NF}'` prints: a) The first field b) The number of fields c) The last field d) The whole line

90. `git pull` equals: a) fetch only b) push + merge c) fetch + merge d) clone

91. Which safely undoes a commit on a shared branch? a) git reset –hard b) git rebase c) git commit –amend d) git revert

92. Merge conflict markers include: a) `<<<<<<< ======= >>>>>>>` b) `### **` c) `/* */` d) `-- ++`

93. You should never rebase: a) A local branch b) Shared/pushed history c) Your first commit d) A feature branch ever

94. `git stash` does what? a) Deletes changes b) Pushes to remote c) Temporarily shelves uncommitted changes d) Creates a branch

95. In IaaS, you manage: a) Nothing b) The OS and app c) Only hardware d) Only the network cables

96. AWS S3 is: a) A virtual server b) A database engine c) Object/file storage d) A load balancer

97. AWS EC2 is: a) A virtual server (compute) b) Object storage c) Identity management d) A network

98. A container differs from a VM because it: a) Includes a full guest OS b) Is always slower c) Can't be networked d) Shares the host kernel, no full OS

99. Kubernetes primarily provides: a) Container orchestration at scale b) A programming language c) A database d) DNS resolution

100. CI/CD's "CI" means: a) Continuous Isolation b) Cloud Infrastructure c) Continuous Integration (auto build+test) d) Container Image

---

Part A — Answer Key

```
1 c   2 a   3 a   4 d   5 a   6 b   7 c   8 c   9 a   10 c
11 d  12 a  13 c  14 b  15 b  16 d  17 b  18 b  19 c  20 c
21 a  22 c  23 a  24 b  25 c  26 a  27 c  28 b  29 c  30 b
31 d  32 b  33 c  34 b  35 b  36 c  37 d  38 b  39 b  40 c
41 b  42 c  43 b  44 d  45 b  46 c  47 b  48 c  49 b  50 b
51 c  52 a  53 c  54 c  55 c  56 c  57 b  58 a  59 a  60 c
61 a  62 c  63 a  64 c  65 c  66 c  67 a  68 c  69 c  70 a
71 c  72 a  73 c  74 a  75 b  76 b  77 c  78 a  79 c  80 a
81 a  82 c  83 c  84 d  85 c  86 a  87 d  88 d  89 c  90 c
91 d  92 a  93 b  94 c  95 b  96 c  97 a  98 d  99 a  100 c
```

Scoring: 90–100 excellent · 75–89 solid, review misses · 60–74 study weak chapters again · <60 re-read Chapters 1–7 before interviewing.

Part B — 50 Short-Answer Questions

Write a one- or two-sentence answer, then check against the key.

1. Define a process vs a thread.
2. Name the four deadlock conditions.
3. What's the difference between a zombie and an orphan process?
4. What does a context switch do?
5. Compare mutex and semaphore.
6. What is virtual memory for?
7. Stack vs heap — one difference each.
8. What is a page fault?
9. What's the difference between SJF and Round Robin?
10. What is a system call?
11. TCP vs UDP — one-line difference.
12. What does DNS return?
13. What port is HTTPS? SSH? MySQL?
14. What are the three TCP handshake messages?
15. 401 vs 403?
16. What does a 502 indicate?
17. Symmetric vs asymmetric encryption in HTTPS?
18. Why does HTTP need cookies?
19. Cookie vs session?
20. Session vs JWT?
21. Is a JWT encrypted?
22. What makes a method idempotent? Give one that is and one that isn't.
23. What does a load balancer provide?
24. Forward vs reverse proxy?
25. What problem does NAT solve?
26. What's a firewall's default-deny policy?
27. What does DHCP do (and the acronym for its steps)?
28. What OSI layer is IP? TCP?
29. Command to watch a log live?
30. df vs du?
31. kill vs kill -9?
32. What does chmod 755 mean?
33. find vs grep?
34. How do you check a service's status and logs?
35. What does the pipe `|` do?
36. What's the shebang line?
37. How does a Bash script read its arguments?
38. How do you check the last command succeeded?
39. Cron field order?
40. Which tool extracts a column — grep, awk, or sed?
41. What does `sed 's/x/y/g'` do, and why the `g`?
42. What does `tr -d '0-9'` do?

43. The golden log pipeline for "top N"?
44. fetch vs pull?
45. merge vs rebase, and the golden rule?
46. Steps to resolve a merge conflict?
47. reset vs revert?
48. IaaS vs PaaS vs SaaS?
49. Container vs VM?
50. Docker vs Kubernetes?

Part B — Answer Key (concise)

1. Process = own private memory; thread = shares the process's memory. 2. Mutual exclusion, hold-and-wait, no preemption, circular wait. 3. Zombie = child finished, not reaped; orphan = parent died, child adopted by PID 1. 4. Saves one process's CPU state and loads another's. 5. Mutex = 1 at a time, owned; semaphore = up to N, not owned. 6. Private address space per process + use disk as RAM overflow (swap). 7. Stack = auto/fast/small/LIFO; heap = manual-or-GC/large/flexible. 8. Accessing a page not currently in RAM; OS loads it from disk. 9. SJF = shortest first (best wait, starvation); RR = fixed time slices, fair/interactive. 10. Controlled request from user space to the kernel for a privileged service. 11. TCP = reliable/ordered; UDP = fast/best-effort. 12. An IP address. 13. 443, 22, 3306. 14. SYN, SYN-ACK, ACK. 15. 401 = not authenticated; 403 = authenticated but not allowed. 16. Backend/upstream returned an invalid response to the proxy/LB. 17. Asymmetric agrees the key once; symmetric encrypts the data fast. 18. HTTP is stateless, so cookies carry state (like a session ID) across requests. 19. Cookie = client-side; session = server-side (cookie holds the ID). 20. Session = server-stored; JWT = stateless signed token. 21. No — signed, not encrypted by default. 22. Repeating has no extra effect; PUT is idempotent, POST isn't. 23. Scalability + availability (spread load, health-check backends). 24. Forward = for clients; reverse = for servers. 25. IPv4 address shortage — many private IPs behind one public. 26. Block everything, then explicitly allow only needed ports. 27. Auto-assigns IPs; DORA = Discover, Offer, Request, Ack. 28. IP = Layer 3; TCP = Layer 4. 29. `tail -f file`. 30. df = free space per filesystem; du = usage of files/dirs. 31. kill = SIGTERM (graceful); kill -9 = SIGKILL (forced). 32. Owner rwx (7), group and others r-x (5). 33. find = locate files; grep = find text in files. 34. `systemctl status X` and `journalctl -u X`. 35. Sends one command's output as the next command's input. 36. `#!/bin/bash`. 37. Positional params: `$1 $2 …`, `$@` all, `$#` count. 38. Check `$?` (0 = success). 39. min, hour, day-of-month, month, day-of-week. 40. awk. 41. Substitutes x with y; `g` = all matches per line, not just the first. 42. Deletes all digit characters. 43. `… | sort | uniq -c | sort -rn | head`. 44. fetch = download only; pull = fetch + merge. 45. Merge keeps branchy history; rebase makes it linear but rewrites it — never rebase shared/pushed history. 46. Edit conflict markers to the correct version, delete markers, `git add`, `git commit`. 47. reset = rewrites history (local); revert = new undo commit (safe on shared). 48. Increasing provider management: EC2 / Heroku / Gmail. 49. Container shares host kernel, light, seconds; VM has full OS, heavy, minutes. 50. Docker builds/runs a container; Kubernetes orchestrates many.

Part C — 20 Practical Linux Questions

Write the command(s). Key follows.

1. Show the last 100 lines of `/var/log/syslog`.
2. Watch `/var/log/nginx/error.log` update in real time.
3. Find all `.conf` files under `/etc`.
4. Count how many lines contain "error" (case-insensitive) in `app.log`.
5. Show disk usage of every filesystem in human-readable form.
6. Find the 10 largest directories under `/var`.
7. Show memory usage in human-readable form.
8. List all running processes and filter to `java`.
9. Gracefully stop process with PID 4321, then force it if needed.
10. Make `deploy.sh` executable.
11. Give a file permissions rwxr-xr-x using numbers.
12. Check whether the `nginx` service is running.
13. View the logs for the `nginx` service, following live.
14. Find files larger than 500 MB starting from `/`.
15. Print only the first column (IP) of `access.log`.
16. Replace all "http://" with "https://" in `urls.txt`, in place.
17. Show the top 5 IPs by request count in `access.log`.
18. Show which process is listening on port 8080.
19. Recursively search for "TODO" in the current directory with line numbers.
20. Delete all `.tmp` files under `/tmp` older than 3 days.

Part C — Answer Key

1. `tail -100 /var/log/syslog`
2. `tail -f /var/log/nginx/error.log`
3. `find /etc -name "*.conf"`
4. `grep -ic error app.log`
5. `df -h`
6. `du -h /var | sort -rh | head -10` (or `du -sh /var/* | sort -rh | head`)
7. `free -h`
8. `ps aux | grep java`
9. `kill 4321` then, if needed, `kill -9 4321`
10. `chmod +x deploy.sh`
11. `chmod 755 file`
12. `systemctl status nginx`
13. `journalctl -u nginx -f`
14. `find / -size +500M`
15. `awk '{print $1}' access.log`
16. `sed -i 's|http://|https://|g' urls.txt` (using `|` as delimiter to avoid escaping slashes)
17. `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5`
18. `ss -tulnp | grep 8080` (or `lsof -i :8080`)
19. `grep -rn "TODO" .`

20. `find /tmp -name "*.tmp" -mtime +3 -delete`

Part D — 20 Networking Scenarios

Give a short diagnosis/answer. Key follows.

1. Users get "connection refused" on port 443. First two things to check?
2. A page loads but images don't. The image server returns 403. What does that mean?
3. `ping` to a server works, but the website times out. What layer is likely the problem?
4. You see many 504 errors. What's the likely cause?
5. DNS for your domain resolves to the wrong IP. What would you check?
6. Your app can't reach the database at 10.0.1.5:5432 but the DB is running. Likely cause?
7. A user reports the site is slow only on their office network. What might it be?
8. After enabling a firewall, SSH stopped working. Likely cause?
9. Two devices on the same subnet can't communicate, but each reaches the internet. What layer?
10. You want to test if port 22 is reachable on a remote host. Command?
11. A REST API returns 401 for all requests. What's missing?
12. A POST request accidentally sent twice created two orders. Why?
13. A video call is choppy; a file download is fine. Which protocol issue and why acceptable?
14. Your browser shows "certificate not trusted." What failed?
15. Requests to `/api` should go to one server pool and `/static` to another. What device/layer?
16. All users behind one office IP got blocked by your rate limiter. What went wrong?
17. A service works via IP but not via its domain name. What's broken?
18. You need many devices to share one public IP. What technology?
19. A 301 vs a 302 redirect — which should you use for a permanent move?
20. Traffic to your site should be encrypted end to end. What must the server present, and on what port?

Part D — Answer Key

1. Is the service actually listening on 443 (`ss -tulnp`), and is a firewall/security group blocking the port. "Refused" usually means nothing is listening or it's blocked.
2. 403 Forbidden = authenticated (or reachable) but not permitted — likely a permissions/hotlink-protection issue on the image server, not a missing file.
3. Layer 4/7 — ping (ICMP, layer 3) works, so the network path is fine; the issue is the service/application not responding (service down, port blocked, or app hung).
4. 504 Gateway Timeout = the upstream/backend didn't respond in time — a slow or overloaded backend or database.
5. DNS records (the A record) at the authoritative nameserver, plus caching/TTL — the old IP may be cached; check `dig` and propagation.
6. A firewall/security group blocking port 5432, or the DB only listening on localhost — check the port reachability with `nc -zv 10.0.1.5 5432` and the DB's bind address.
7. Their network — a proxy, firewall, DNS, or bandwidth issue on that office network, since it's location-specific.
8. The firewall's default-deny blocked port 22; you need to explicitly allow SSH (22).
9. Layer 2 (Data Link) — same subnet means no routing needed; likely a switch, VLAN, or ARP issue.
10. `nc -zv host 22` (or `telnet host 22`).
11. Authentication — a valid token, session, or credentials; the server doesn't know who you are.

12. POST isn't idempotent — sending it twice creates two resources. Use idempotency keys or PUT semantics to prevent duplicates.
13. UDP for the video call — a late packet is useless in real time, so dropping it is acceptable; the file download (TCP) needs every byte.
14. Certificate validation — expired, self-signed, wrong domain, or an untrusted CA; the TLS identity check failed.
15. A Layer 7 (application) load balancer or reverse proxy — it can route by URL path.
16. That office is behind NAT sharing one public IP, so many users appear as one IP; rate-limiting by IP punished them all. Account for shared IPs.
17. DNS resolution — the name isn't resolving to the IP; check the DNS record and local resolver.
18. NAT (Network Address Translation).
19. 301 Moved Permanently for a permanent move; 302 is temporary.
20. A valid TLS certificate (verified by a trusted CA), served over HTTPS on port 443.

Final words

If you've worked through this exam and can explain your wrong answers, you are genuinely prepared for a junior on-call, support, or cloud interview. Remember on the day:

- **Think out loud.** Interviewers score your reasoning, not just the answer.
- **Measure before you act; mitigate before you debug; find the root cause.**
- **When you don't know, say how you'd find out** — that's a senior habit and it beats bluffing.
- **Breathe.** You've done the work. Now go show them.

> **Good luck. You've got this.** 🚀